A FRIENDLY GUIDE

Individual Guide

Start here, learn by doing, and find the task you need

Beginner edition · Release 0.7.0 · 23 October 2026

58 task types · 22 real projects · No R experience needed

Contents

1 Welcome to Individual Guide
- 1.1 Find your starting point
- 1.2 Six words you will see often
- 1.3 How to follow this guide

2 Installation and first launch
- 2.1 Download the public Windows release
- 2.2 Open the app each day
- 2.3 Choose or change your language
- 2.4 Update the app
- 2.5 Before your first project

3 Your first project
- 3.1 Open the flower example
- 3.2 Look around before running anything
- 3.3 Make one small change
- 3.4 Find the report and log
- 3.5 Build a small flow yourself

4 Find your way around
- 4.1 Open a task or result
- 4.2 Set up and keep a task result
- 4.3 Understand the arrows
- 4.4 Choose useful names

5 Save, reopen, and share projects
- 5.1 Save your project
- 5.2 Move work to another computer
- 5.3 Recover interrupted work
- 5.4 Share a project with care

6 Run tasks and check results
- 6.1 Run, Preview, and Save do different jobs
- 6.2 Check the task status
- 6.3 Run the right part of a project
- 6.4 Stop a run

7 Importing your data
- 7.1 Import Data
- 7.2 Work with a database

8 Select, join, and calculate data
- 8.1 Query Builder
- 8.2 Filter and Sort
- 8.3 Derive Columns

9 Optional: saved code steps
- 9.1 Script

10 Clean and reshape your data
- 10.1 Append Table
- 10.2 Compare Data
- 10.3 Stack Columns
- 10.4 Split Columns
- 10.5 Transpose
- 10.6 Rank
- 10.7 Standardize Data
- 10.8 Random Sample
- 10.9 Remove Duplicates
- 10.10 Keep / Drop Columns
- 10.11 Rename Columns
- 10.12 First / Last N Rows
- 10.13 Replace Missing Values
- 10.14 Recode Values
- 10.15 Change Column Types

11 Get to know your data
- 11.1 Characterize Data
- 11.2 List Data
- 11.3 One-Way Frequencies
- 11.4 Summary Statistics
- 11.5 Summary Tables
- 11.6 Table Analysis
- 11.7 Distribution Analysis

12 Make charts people can read
- 12.1 Keep the output you need
- 12.2 Bar Chart
- 12.3 Bar-Line Chart
- 12.4 Box Plot
- 12.5 Bubble Plot
- 12.6 Contour Plot
- 12.7 Histogram
- 12.8 Line Plot
- 12.9 Pie Chart
- 12.10 Scatter Plot
- 12.11 Scatter Plot Matrix

13 Compare groups and fit models
- 13.1 One-Way ANOVA
- 13.2 t Test
- 13.3 Linear Models
- 13.4 Linear Regression
- 13.5 Logistic Regression
- 13.6 Generalized Linear Models
- 13.7 Nonlinear Regression
- 13.8 Correlations
- 13.9 Principal Components
- 13.10 Cluster Analysis

14 Check patterns and work with time
- 14.1 CDF Plots
- 14.2 Probability Plots
- 14.3 Control Charts
- 14.4 ARIMA / Basic Forecasting

15 Time until an event
15.1 Life Tables
15.2 Proportional Hazards
16 Run steps conditionally or repeatedly
16.1 Branch
16.2 Loop
16.3 Choose a loop source
16.4 Start small
17 Keep a useful log
17.1 Choose how much to record
17.2 Customize a task's message
17.3 Add a message inside a Script
17.4 Export Log
17.5 Put the log in a report
17.6 Read dates as well as messages
17.7 Use startup logs for startup problems
18 Share reports and files
18.1 Report
18.2 Choose HTML or Word
18.3 Pick a content style
18.4 Export
18.5 Choose an export format
18.6 Check a finished delivery
19 Optional: try the R Console
19.1 Open the console and run a command
19.2 Read a command before running it
19.3 Find and reuse a command
19.4 Clear, Stop, and Reset
19.5 Keep useful code in a task
20 Optional: export the flow as R code
20.1 Share code that someone can run
21 Explain and review your work
21.1 Add notes at decision points
21.2 Check before sharing
21.3 Find something quickly
21.4 A simple review checklist
22 Make the app comfortable to use
22.1 Global defaults and project overrides
22.2 Package versions
22.3 Display and navigation
22.4 Saving and confirmation choices
22.5 Trust and private information
23 Open other Shiny apps
23.1 What you need
23.2 Try the practice apps
23.3 Open and run an app
23.4 Return to your project
23.5 Close an app or open another
23.6 If an app will not start
24 Keep work running smoothly
24.1 When editing feels slow
25 Troubleshooting
25.1 The app seems not to open
25.2 The app disconnects or the R server stops
25.3 A button or menu item is disabled
25.4 A file will not import
25.5 A table or column is missing from a choice list
25.6 A result is old, empty, or different from the preview
25.7 A model fails or looks strange
25.8 A report is missing a chart
25.9 An exported file is missing or will not update
25.10 A log seems empty
25.11 Ask for help
26 Optional: install with R
26.1 Step 1: install R from its public website
26.2 Step 2: install the app from GitHub
26.3 Step 3: add the features you need
26.4 Step 4: open the app
26.5 Open it next time
26.6 If package installation fails
27 Choose an example project
27.1 Find an example by task
27.2 A simple learning route
27.3 Before you rerun an example
27.4 Iris exploration
27.5 Air quality - clean and compare
27.6 Fuel economy model
27.7 Cars - prepare an analysis table
27.8 US states - import, join and filter
27.9 Iris - reshape and compare
27.10 Cars - rank, scale and sample
27.11 Tooth growth - descriptive analysis
27.12 Flowers - categorical chart gallery
27.13 Flowers - numeric chart gallery
27.14 Nottingham temperatures - time charts
27.15 Plant growth - compare experimental groups
27.16 Cars - binary outcome model
27.17 Loom breaks - a count model
27.18 Puromycin - fit a nonlinear curve
27.19 Flowers - multivariate structure
27.20 Lung cancer - survival analysis
27.21 Air passengers - forecast a monthly series
27.22 Nile flow - distribution and stability
27.23 Flowers - conditional quality gate
27.24 Flowers - repeat by species
27.25 Cars - a report with an execution audit
28 Task finder
29 Shortcuts and everyday terms
29.1 Useful Windows desktop shortcuts
29.2 A few more terms
29.3 Public downloads and help

1 Welcome to Individual Guide

You can use Individual Guide without knowing R. The app helps you bring in data, clean it, make charts, and share results. You choose settings in forms. The app does the calculations for you.

Think of a project as a recipe. Your data are the ingredients. Each **task** is one step, such as sorting rows or drawing a chart. A **flow**, also called a pipeline, puts those steps together. You can save the recipe and use it again.

This guide covers **Individual Guide 0.7.0**, with **58 task types** and **22 example projects**. Start with installation and the first project. After that, read the task pages you need. You do not need to read the book from start to finish.

1.1 Find your starting point

What you want to do	Where to go
Install the Windows app	Installation and first launch
Try the app with ready-made data	Your first project
Bring in your own spreadsheet or CSV	Importing your data
Find a task and a matching example	Task finder
Make a report with a log	Reports and files
Fix a problem	Troubleshooting
Run the app through R instead	Optional: install with R

1.2 Six words you will see often

Word	Meaning in the app
Project	Your saved work: data, tasks, results, and notes
Table	Data arranged in rows and columns, like a spreadsheet
Task	One step that does a job
Flow	A group of connected tasks
Output	A result from a task, such as a table or chart
Log	A record of what happened when tasks ran

For example, a flower project can start with measurements, calculate a mean, draw a chart, and make a report. The arrows show which steps need results from earlier steps.

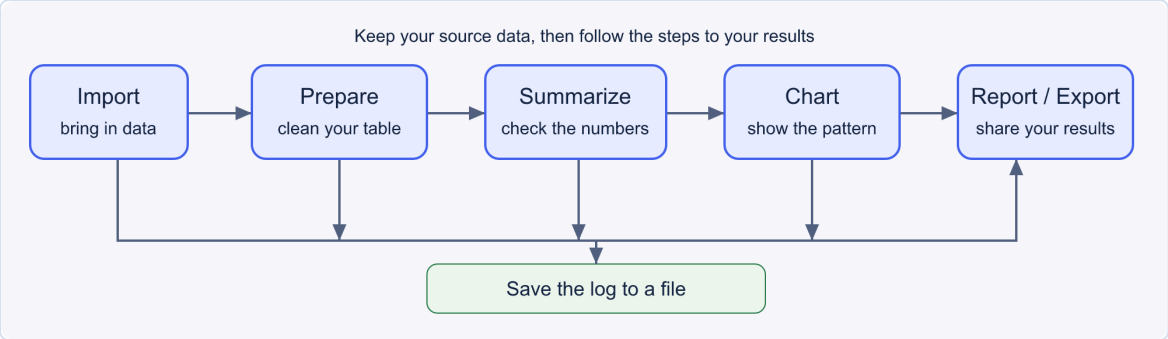


Figure 1. A flow takes data through tasks to results, reports, and a saved log.

1.3 How to follow this guide

Bold words, such as **Run**, name buttons or choices. **File > Save project as...** means open the File menu, then choose Save project as. The dots mean that another window will open. Text such as `flowers` is a name to type exactly.

The Windows app has menus across the top. If you open the app through R in a web browser, use the buttons inside the app for opening and saving projects. Some desktop features, such as restarting the R server from a menu, are only in the Windows app.

Some browser controls are icons. Point to an icon to see its name, such as **Save project** or **Help**.

The HTML book works offline after you download it. Use **Find a chapter or task** in its contents, or press Ctrl+F to search the page. The PDF is useful for printing. EPUB works in an e-reader, and Word lets you add your own notes.

Read the same guide in English or Français (Canada). Both editions have the same chapters, examples, and steps, with screenshots in their own language. Use the same language for the app and the guide so the buttons match the pictures.

2 Installation and first launch

For most Windows users, the Setup installer is the easiest choice. It includes R and the extra software the app needs. **You do not need to install R or RStudio separately.**

2.1 Download the public Windows release

You need a 64-bit Windows 10 or Windows 11 computer. Allow about 1.5 GB for the program, plus space for your projects. You need an internet connection to download Setup. The installed app and its examples can then work offline. Tasks that use an online database still need that connection.

1. Open the public releases page.
2. Under **Assets**, download **IndividualGuide-0.7.0-Setup-Windows-x64.exe** for this edition. If a newer release is shown, read its notes before choosing it. Find this book's matching installer on the 0.7.0 release page.
3. Close any running copy of Individual Guide. You do not need the files called **Source code** to use the desktop app.
4. Double-click the downloaded Setup file. Choose **English** or **Français (Canada)** when it asks for a language, then follow its instructions. The usual installation is for your own Windows account.
5. Open **Individual Guide** from the Start menu or desktop shortcut.

The installer is unsigned. Windows may show an unknown-publisher warning. Check that you downloaded it from **woodtho/IndividualGuide** before deciding whether to continue. If your work or school computer blocks it, ask your support team for help.

2.2 Open the app each day

Use the Start menu or desktop shortcut. Setup installs or updates the program; you do not need to run Setup each time.

The first launch may take longer than later launches. Wait while the startup screen shows its progress. The welcome screen then offers three choices. For the first exercise, choose **Start with a blank canvas**. This opens the workspace and makes the Help menu available.

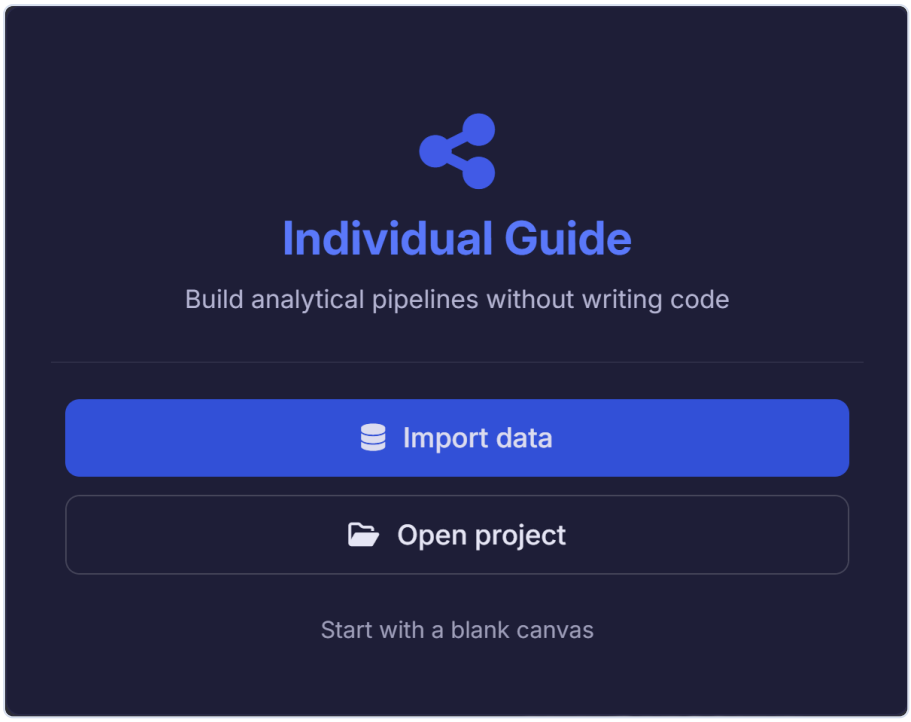


Figure 2. At the welcome screen, choose Start with a blank canvas to reach the workspace and its Help menu.

If startup stops with an error, choose **Open startup log**. See Troubleshooting for the next step. Repeatedly opening more copies usually does not help.

2.3 Choose or change your language

The app supports English and Canadian French. The Windows installer passes your choice to the app. You can change it later without restarting R or reopening the project.

1. Choose **English** or **Français (Canada)** in the language list on the welcome screen or beside the app name.
2. In the desktop app, you can also use the **Language** menu. In **Settings**, search for **Interface language**, then select **Edit** and **Save option** for a global default or project override.
3. Check that the buttons and menus use your chosen language. The app remembers it on this computer.

The Language menu sets your global language. A project can override it in Settings. Your data, column names, file paths, formulas, and names you entered stay as you typed them. R commands and technical identifiers keep the spelling that R needs.

2.4 Update the app

1. Save your project and close the app.
2. Visit the public releases page.
3. Download and run the newer Setup installer.
4. Open the app from its shortcut. Check **Help > About Individual Guide...** to see the installed version.

The app does not update itself. Updating keeps saved projects and settings, but keep your own backup of important project files. Uninstalling through Windows removes the program and keeps separately stored user data.

2.5 Before your first project

Create a folder you can find again, such as **Documents > Individual Guide Projects**. Use it for saved projects and files you export. Give each project a clear name, such as `flower-practice.igpx`.

The file ending **.igpx** means an Individual Guide project. Open it through **File > Open project...**, or double-click it in Windows after installing the app.

You are ready to try your first project. The optional R installation steps are near the end of this guide; the Windows app does not need them.

3 Your first project

Start with an example that already works. You will open flower data, find a chart, check a report, and save your own copy. No typing of R code is needed for this first part.

3.1 Open the flower example

1. If the welcome screen is still open, choose **Start with a blank canvas**.
2. In the desktop app, choose **Help > Application help**. In a browser, select the **Help** button in the app header.
3. Open **Example projects** and search for **Iris exploration**.
4. Choose **Open example**. If an unsaved-work prompt appears, choose **Cancel** and save your current work before trying again. In the desktop prompt, **Discard** replaces the current work; the browser app calls this **Discard and open example**.
5. Choose **File > Save project as...**. Save the project as `flower-practice.igpx` in your projects folder. In a browser, select **Save project** and choose the name and folder in the file picker that opens for this unsaved example.

Iris is a kind of flower. The example has measurements from **150 flowers** in **three species**, with **50 flowers per species**. Each row is one flower. The four measurement columns use centimetres. `Species` names the flower's group.

Opening an example gives you a separate working copy. You can change it without changing the example that ships with the app. Save your copy if you want to keep those changes.

An example may show a software-version notice because its saved run came from another setup. You can inspect the saved results first. Before rerunning, check the export folders, then use the new task logs to check how the work runs on your computer. The review tools explain run records if you need to compare versions.

3.2 Look around before running anything

The left side lists the project's tasks and data. The **Flow** tab shows connected steps. Select the source task and read its notes. Open its flower table and look at the column names.

The example uses labels that describe each step's job. Select **Summarize measurements**, **Compare mean petal length by species**, and **Explore relationships** to open the Summary Statistics, Summary Tables, and Scatter Plot tasks. The task type appears in each panel. Open **Results** to inspect the saved result before running anything again. Names given to steps within a project stay in their original language.

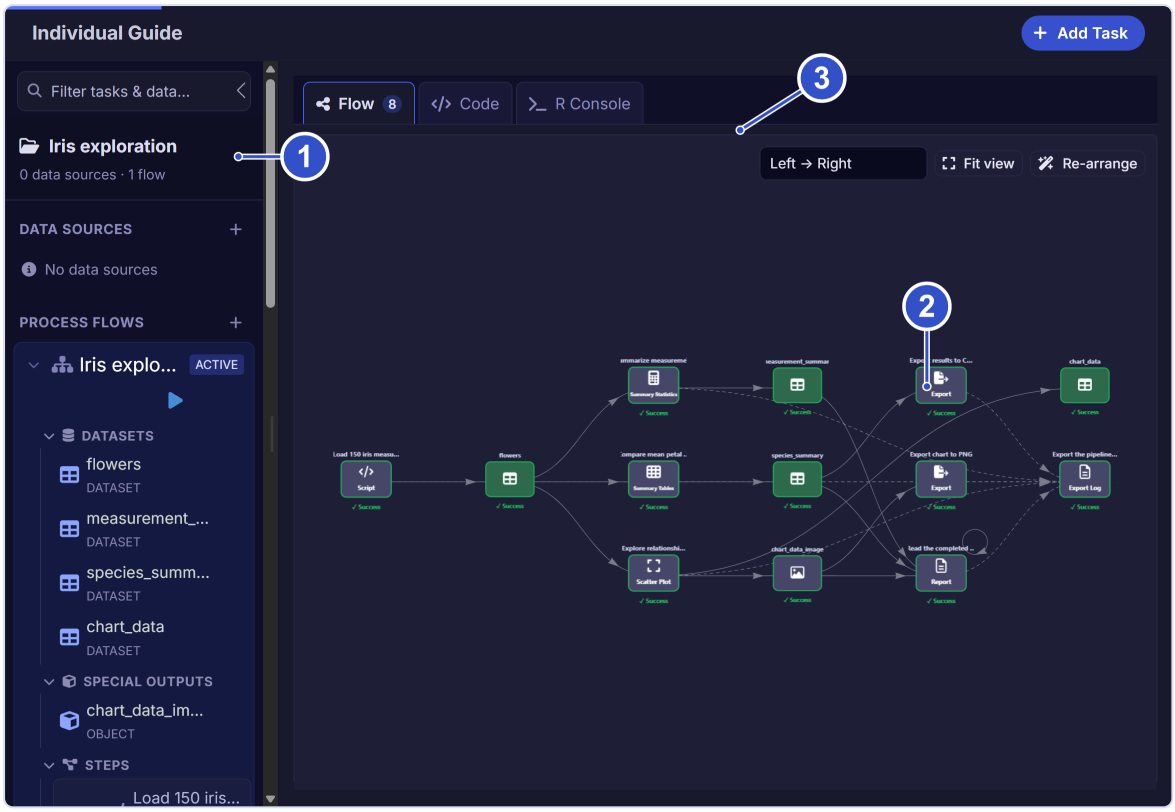


Figure 3. Callout 1 marks the project tree; 2 marks the connected flow; 3 marks the Flow, Code, and R Console tabs. Select a task to open its settings.

For the species summary, the mean petal lengths should be about:

Species	Mean petal length, in centimetres
setosa	1.462
versicolor	4.260
virginica	5.552

You do not need to memorize these values. They give you a way to check that you are viewing the right table and settings.

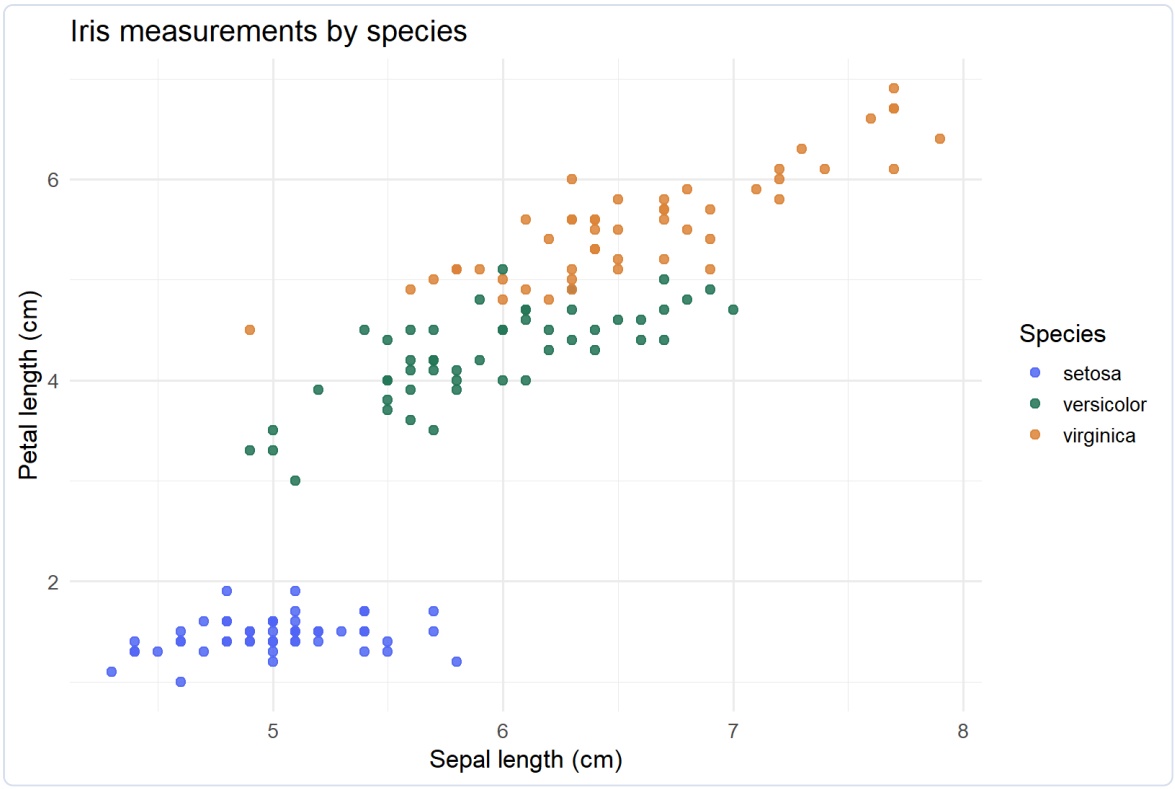


Figure 4. The flower example compares sepal length and petal length. Colour shows the species.

3.3 Make one small change

1. Open the **Scatter Plot** task.
2. Under **Setup**, change **Y / value** from `Petal.Length` to `Petal.Width`.
3. Scroll down within **Setup**. Change **Title** to **Petal width by sepal length**. Set **Y label** to **Petal width (cm)**.
4. Click **Run** in the task header. Open Results and check the new chart.
5. Open the **Report** task and select **Generate Report** so its chart matches your change.
6. Choose **File > Save project** or press Ctrl+S.

Check that the vertical axis is labelled **Petal width (cm)**. The source petal widths range from 0.1 to 2.5 centimetres.

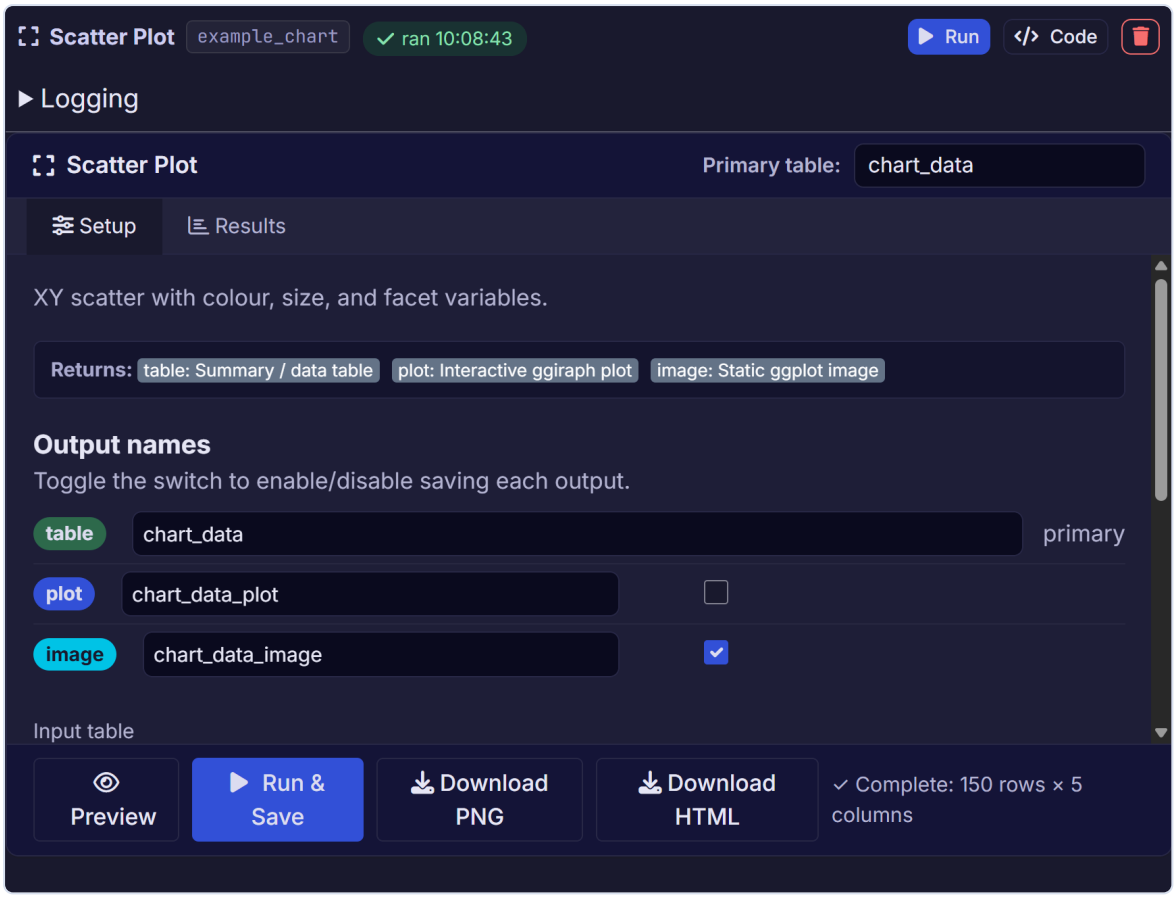


Figure 5. Start from these Scatter Plot settings and change Y / value to Petal.Width. Save Static ggplot image keeps a picture for the report.

Changing a setting does not update a saved report by itself. Run the changed task, then the later tasks that use its result. If you want to run everything again, use the flow's play button. Before doing that, check the Export tasks' folders and Overwrite settings.

3.4 Find the report and log

1. Open the **Report** task. Under **Appearance**, select **Include pipeline log in the report**.
2. Select **Generate Report**, then **Download HTML**. Save the file in your projects folder and open it in a browser.
3. Check its headings, tables, chart, and **Pipeline execution log** section. The downloaded report is a separate file from the project.

Open **Export Log** to see how the example writes its log to a file. Before running this task, choose a destination you can find. Logs can include earlier runs as well as the latest one. Dates and times help you tell them apart.

3.5 Build a small flow yourself

This optional exercise builds the same kinds of results from a new project. It includes one short piece of R code that you can copy exactly. You do not need to understand the code to finish the exercise. Stay with the ready-made examples if you prefer.

Start with an empty project. In the Windows app, choose **File > New project** and save it as `first-iris.igpx`. In a browser, save your current work, refresh the page, and choose **Start with a blank canvas**. Then use **Save project** to create `first-iris.igpx`.

Use **Add Task** to find each task below. Task labels help you recognize a step. Output names tell later steps which result to use, so type those names carefully.

3.5.1 Step 1: load the practice data

1. Select **Add Task**, open **General**, and choose **Script**.
2. In the task header, set **Primary table** to `flowers`.
3. Under **Setup**, leave **Input tables / special outputs** blank. This step supplies its own data.
4. In **Outputs**, type `flowers: table` on one line. This tells the app to keep a table named `flowers`.
5. Paste the following into **R code**, not the R Console:

```
flowers <- datasets::iris
log_event(sprintf("Loaded %d flowers", nrow(flowers)), "INFO")
```



Figure 6. In Outputs, flowers: table declares the table to keep. Its name matches flowers in the copied code.

Click **Run**. Select `flowers` under **DATASETS** in the project tree and check for 150 rows and five columns. If it is missing, make sure Outputs contains `flowers: table`. The name must match the name in the copied code.

3.5.2 Step 2: summarize the measurements

1. Select **Add Task > Describe > Summary Statistics**.
2. Under **Setup**, select `flowers` in **Input table**.
3. In **Numeric columns**, choose `Sepal.Length`, `Sepal.Width`, `Petal.Length`, and `Petal.Width`.
4. Leave **Save** checked and enter `measurement_summary` in **Output table**.
5. Select **Analyze**, then open **Results**.

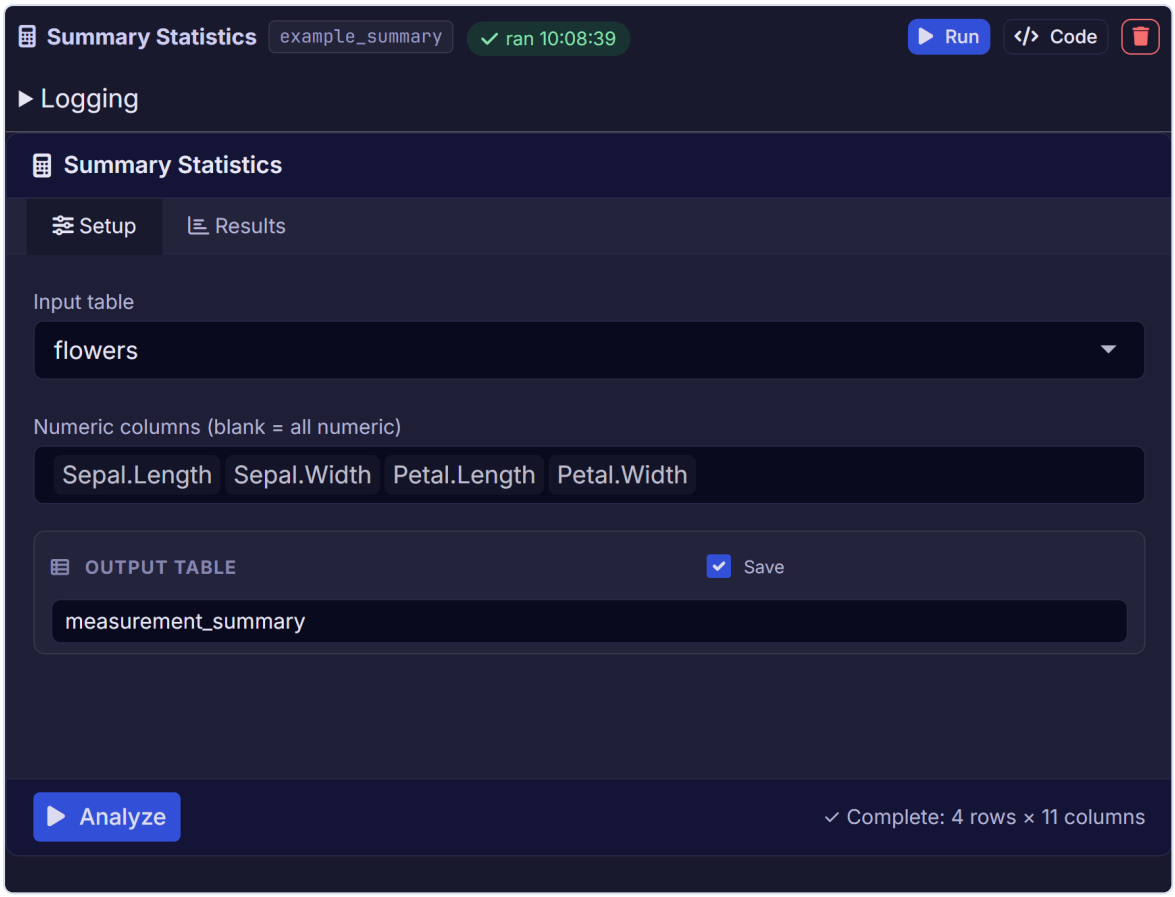


Figure 7. Summary Statistics needs an input table and numeric columns. The Save checkbox keeps its Output table for later tasks.

Under **Results**, select **Data** to read the saved table. Check the mean for `Sepal.Length`. It should be about **5.8433**. A mean is the total divided by the number of values. The app may show fewer decimal places.

3.5.3 Step 3: compare the three species

1. Select **Add Task > Describe > Summary Tables**.
2. Under **Setup**, choose these settings:

Setting	Choice
Input table	`flowers`
Row variable	`Species`
Value variable	`Petal.Length`
Summary statistic	Mean
Output table	`species_summary`

3. Leave **Save** checked and select **Analyze**.
4. Open **Results > Data** and compare the three means with the table earlier in this chapter.

3.5.4 Step 4: draw the chart

1. Select **Add Task > Graph > Scatter Plot** and choose `flowers` in **Input table**.
2. Set **X / category / date** to `Sepal.Length`, **Y / value** to `Petal.Length`, and **Group / colour** to `Species`.
3. Set **Title** to **Iris measurements by species**.
4. Near the top of the panel, check **Save Static ggplot image**. Set **Output name for Static ggplot image** to `iris_scatter_image`.
5. Select **Run** and open **Results**. Check for a chart like Figure 4.

A static image is a still chart that you can put in a report or export as PNG. The word ggplot names the chart tool; you do not need to learn it.

3.5.5 Step 5: save a table and chart as files

1. Select **Add Task > Output > Export**. Choose `species_summary` in **Output to export** and CSV in **Format**.
2. Select **Choose folder...** and choose your projects folder. In **File name (without extension)**, type `species-summary`.
3. Select **Export** and check for `species-summary.csv` in that folder.
4. Add another Export task. Select `iris_scatter_image`, choose PNG, and use the same folder.
5. Enter `iris-scatter` without the file ending. Set **DPI** to 150, select **Export**, and open `iris-scatter.png`.

DPI controls how many dots are used per inch. Check the text in the finished image before sharing it.

An existing file can block an export. Choose a new name, or enable **Overwrite if the file already exists** when you mean to replace it. An exported file is separate from your saved project.

3.5.6 Step 6: make a report

1. Select **Add Task > Output > Report**. Under **Content**, add `measurement_summary`, `species_summary`, and `iris_scatter_image`. Give the items clear section headings.
2. Under **Appearance**, choose HTML in **Output format** and set **Report title** to **First Iris report**.
3. Select **Include pipeline log in the report** and leave **Minimum log level** at INFO.
4. Select **Generate Report**, then **Download HTML**. Open the downloaded file and check for the two tables, the chart, and **Pipeline execution log**.

3.5.7 Step 7: save the log

A full path includes the folder address and filename together. Use your own projects folder; the practice folder shown in screenshots is only an example.

1. Select **Add Task > Output > Export Log**.
2. In **Run after and include these tasks**, select the preceding tasks, including Report, so it waits for them.

3. Choose CSV. In **Destination file path**, enter the full path to `iris-log.csv` inside your projects folder. For example, use the folder's address from Windows File Explorer, followed by `/iris-log.csv`.
4. Run the active flow. Open the CSV and check for **Loaded 150 flowers** and the Report completion entry.

The report's own log section is made before the report finishes. The final Export Log can record that later completion.

3.5.8 Step 8: close and reopen your work

1. Choose **File > Save project** to save `first-iris.igpx`.
2. Choose **File > Close project**, then **File > Open project...** and select that file. In a browser, use **Open project** in the app header after saving.
3. Check that its table, chart, and report are still there. You have made a reusable flow with data, results, a report, and a log.

4 Find your way around

The project list, flow diagram, and task panels show the same work in different ways. Use whichever makes the next step easiest to find.

4.1 Open a task or result

Select a task in the project list or flow to open its panel. Its settings control what it does. Results show what happened when it last ran. Expand **Logging** to read recorded entries and set messages. **Code**, where offered, shows the R instructions; reading it is optional.

Closing a task tab hides that panel. It does not delete the task. Use the task's delete action only when you want to remove the step itself.

If the diagram becomes crowded, choose **View > Task list** in the desktop app. In a browser, select **Project tools**, then **Task list**. Search for the step you need and open it from the list. Use **Fit view** to bring the whole flow back into view.

4.2 Set up and keep a task result

Most task pages in this guide start after you have opened a project and run the step that supplies the input table. To add a task, select **Add Task**, choose its category, then select its name. The Task finder lists those categories.

1. Open **Setup** when the task has that tab. Choose the input table before choosing columns. A column list can stay empty until its input has run.
2. Enter a unique output name. Keep any **Save** checkbox beside a wanted result checked. A result that is only displayed may not be available to a later task.
3. Select **Run** in the task header. Some forms also have a more specific button, such as **Analyze**, **Load**, **Export**, or **Generate Report**. This guide uses that button name when helpful.
4. Open **Results**, **Data**, or **Preview**, as directed. Check values and messages before moving on.
5. Save the project to keep your work on disk. A task's **Save** checkbox and **Save project** do different jobs.



Figure 8. Under Results, the Data tab shows one row per numeric measurement. Check the mean and missing count before using the table in a report.

A form may scroll. If you cannot find an output control, look near the top of the task panel. If you cannot find its action button, look near the bottom. Screenshot colours may differ from your chosen theme; the labels and steps are the same.

Screenshots use a separate practice folder. Choose the folder you created during installation when your file address differs from the picture.

4.3 Understand the arrows

A later task needs the right input name. For example, a summary that uses `flowers` must wait for the task that creates `flowers`. The app draws the connection from those choices.

Dragging a task to a new place changes the layout. It does not change which tasks run first. To change that, edit its inputs or its branch and loop settings.

4.4 Choose useful names

Give tasks clear labels, such as **Remove repeated customer rows**. Use short, unique output names, such as `customers_clean`. After changing an output name, check later tasks' inputs and rerun the flow.

5 Save, reopen, and share projects

5.1 Save your project

Use **File > Save project** or Ctrl+S in the desktop app. The first save asks for a name and folder. Use **File > Save project as...** to keep a separate copy before a large change.

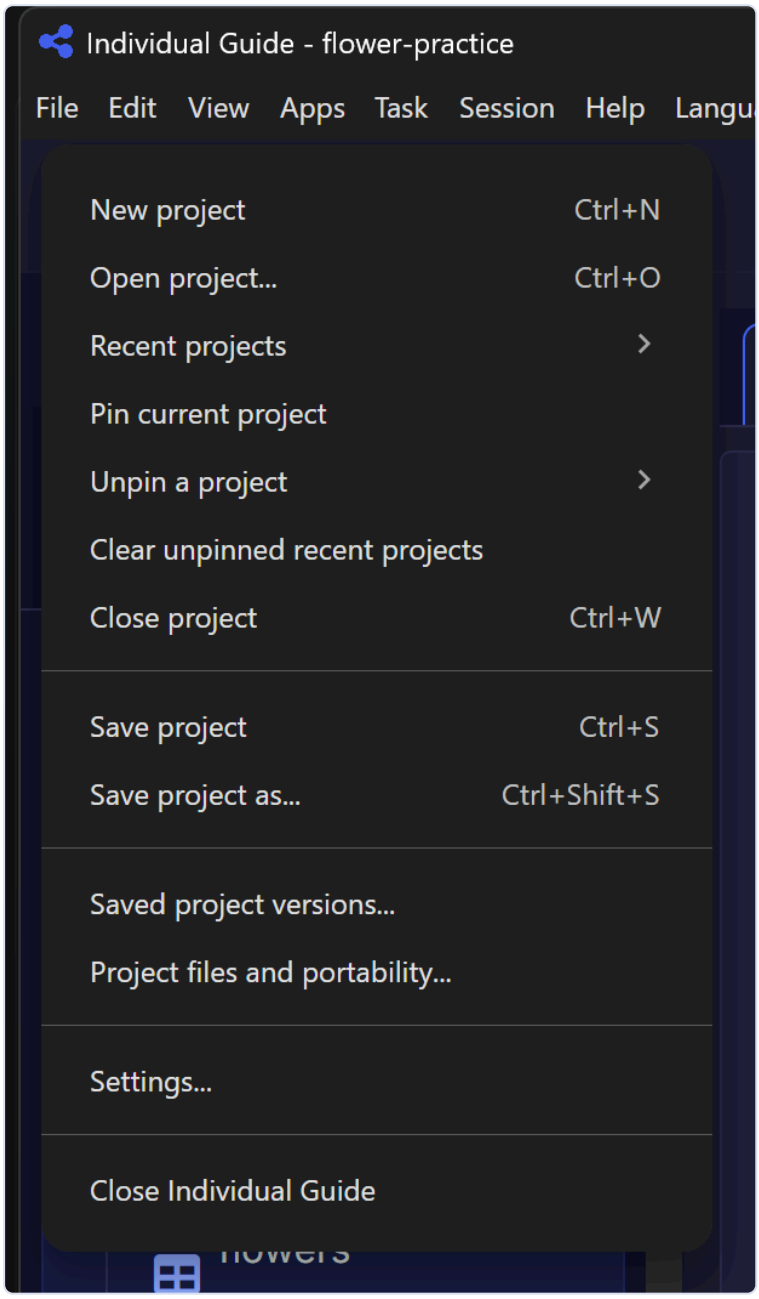


Figure 9. The File menu groups project saving, saved versions, portability, and settings. Use Save project as to make a separate copy.

In a web browser, use the app's Save controls. Follow its file picker or download prompt and check where the file was saved.

A saved project keeps tasks, settings, available results, notes, and retained task logs. It does not automatically copy every outside file used by your tasks. Keep important input files in a stable folder.

5.2 Move work to another computer

1. Save the project, then choose **File > Project files and portability…** in the desktop app. In a browser, open **Project tools**, then its project-files option.
2. Check the list of outside files. Resolve missing files before making a portable copy.
3. Select **Save portable project** to bundle supported inputs with the project.
4. Open the copy on the other computer and check its inputs before running it.

Database connections still need access to the database. A Script may refer to a file that the portable export cannot find on its own. If so, include that file separately and update the path. A path is the address of a file or folder on your computer.

Reports and files that you already exported are separate copies. Saving the project again does not update those copies. Run the relevant Report or Export task when results change.

5.3 Recover interrupted work

The app keeps recovery information for unsaved work. After a crash or restart, it may offer a recovery copy. Read the prompt, recover the work you need, then save it with a clear filename.

Recovery helps after a problem. Keep using Save during normal work. If you need an earlier saved copy, choose **File > Saved project versions…** in the desktop app, or **Saved versions** from the browser app's Project tools. Check the version date before restoring it.

1. Save your current project before opening its saved versions.
2. Open **Saved project versions…** for the project you mean to restore. Select **Previous saved version** by its date and time.
3. Select **Restore selected version**. This opens the earlier work as an unsaved copy.
4. Check its results, then use **Save project as…** to keep it under a separate name. **Save project** writes the restored work back to the current project file.

5.4 Share a project with care

A project can contain data, paths, logs, and R code. Share only information you mean to share. Open projects from people or sources you trust, because running tasks can run code and write files.

Before sending a project, save it, check its report, and try opening your saved copy. If the recipient only needs the results, send a report or CSV instead of the full project.

6 Run tasks and check results

Start with one task while you are learning. Once each step works, run the whole flow.

6.1 Run, Preview, and Save do different jobs

Action	What it does
Preview	Lets you inspect a trial result where the task supports it
Run or Run & Save	Runs the task and keeps its result in the project
Run flow	Runs connected tasks in the active flow
Save project	Writes your project to its .igpx file

After a Preview, use Run to keep the result. After running, use Save to keep the project on disk. A Script can write outside files even during a preview, so preview only code you trust.

6.2 Check the task status

A successful status means the task finished. Open its result to make sure it answered your question. An error means it needs attention. Expand **Logging** and read its recorded error entries.

A stale result means an input or setting has changed since that result was made. Run the affected task and the later tasks that use it. An old report may still open while it waits to be rebuilt.

A skipped task may belong to a branch that was not chosen. A waiting task may need an earlier task to finish. Check inputs and branch settings before treating either state as a failure.

6.3 Run the right part of a project

Use the flow's play button to run the active flow. Check which flow is selected if your project has more than one. Right-click a task for **Run to here** or **Run from here** when those actions are available. These follow task connections, rather than the positions of shapes on the screen.

1. In the desktop app, choose **Task > Validate active flow...** before a long run. In a browser, select **Validate active flow** in the app header.
2. Read the listed problems. Close the dialog and correct any missing inputs or invalid settings.
3. Validate again. Once the setup is ready, choose **Task > Run active flow** or use the flow's play button.

Validation checks the setup. It cannot prove that your data or conclusions are correct.

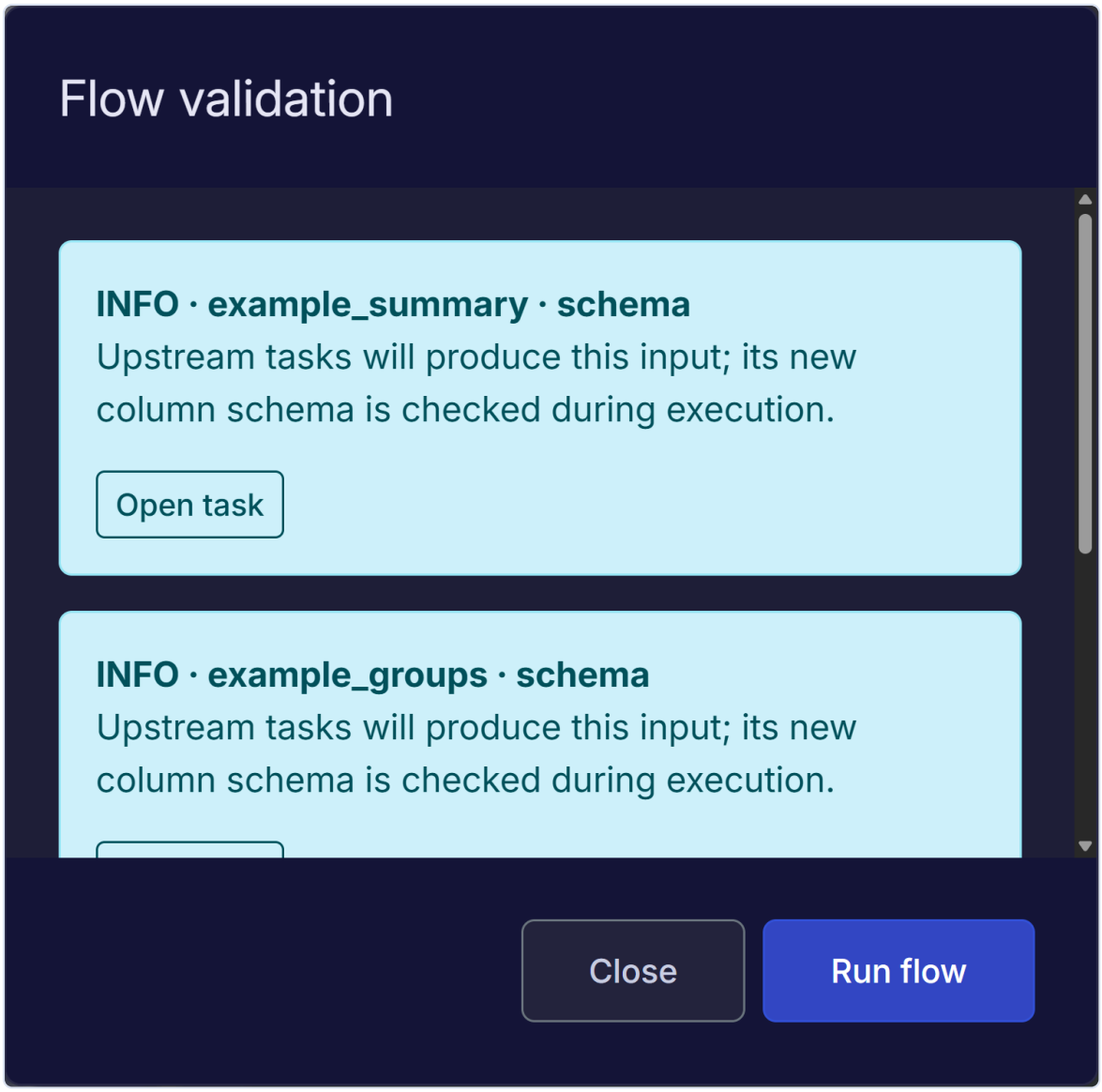


Figure 10. Validation lists problems or information about checks that will run later. Read each message and resolve blocking items before continuing.

6.4 Stop a run

Use **Stop** or **Task > Stop run** in the desktop app. Some work may need to finish its current step before it can stop. Wait for the status to change before starting another run.

Stopping does not undo files that tasks already wrote. Check export folders before running again. If you want to avoid replacing files, use new filenames or turn off Overwrite.

7 Importing your data

For a first import, use a small CSV or Excel sheet with one header row. Give every column a name. Keep titles, footnotes, and totals outside the data area.

7.1 Import Data

Use **Import Data** to bring a file or database table into the project.

1. Select **Add Task > Data > Import Data**. Under **Setup**, choose **File** in **Source**.
2. Select **Browse...** and choose your file. The app detects its type from the selected file.
3. For CSV or other text files, check **Delimiter** and **First row is header**. A delimiter separates columns, often with a comma or tab. Open **More file settings** only if you need to change options such as text encoding.
4. For Excel, choose the sheet and any row or range settings shown after selecting the file.
5. Select **Preview**. Each field should be in its own column, with the right names and values.
6. Enter a clear **Output table name**, such as `sales_raw`, then select **Run import**.

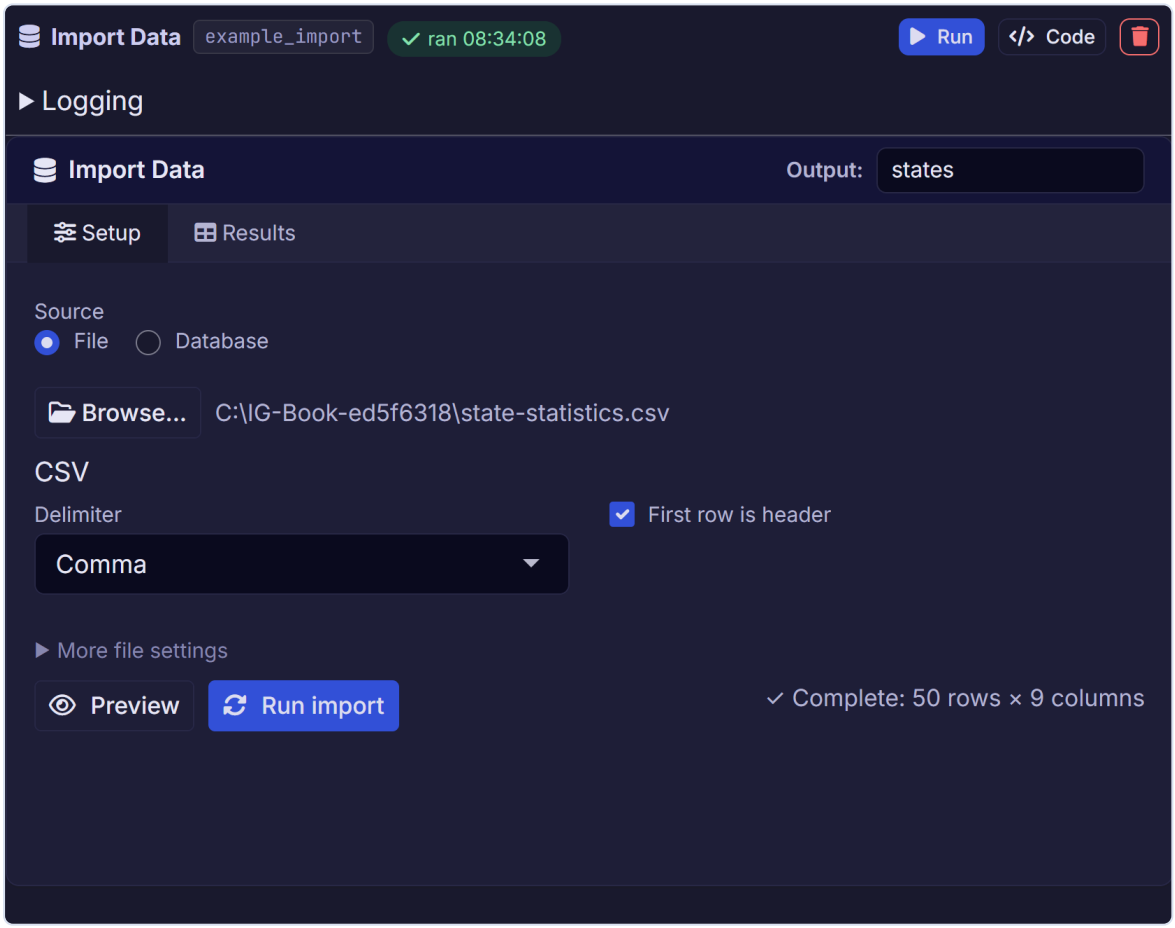


Figure 11. Import Data detects the selected CSV and offers delimiter and header settings. Preview it before running the import.

Open the new table and check its row count. Check dates, missing values, and identifiers such as postal codes. If an ID starts with zero, keep it as text so the zero is not lost.

Selecting a different file suggests a new output name based on that filename. Check **Output table name** again before rerunning an existing import. Later tasks still need the name they were set up to use.

If all values land in one column, check the separator. If the names become the first data row, check the header setting. If letters look wrong, try the file's text encoding. The **US states - import, join and filter** example uses a real bundled CSV that you can inspect without finding your own file.

7.1.1 Keep zeros at the start of an ID

Use this when a CSV contains IDs such as `001` and you need to preserve their exact text.

1. Select the CSV, then open **More file settings**.
2. In **col_types**, type one lowercase `c` for each column, without spaces or quotes. Use `cc` for a two-column file or `cccc` for four columns. Each `c` tells the app to read that column as text.
3. Preview the file and check that `001` still has its zeros. Confirm the output name, then select **Run import**.
4. Use **Change Column Types** afterwards to convert measurement columns to numeric. Leave the ID column as text.

Use this option only when you need it. Reading every column as text means measurements need conversion before numeric charts or models can use them.

7.2 Work with a database

Ask the person who manages the database for its connection details and a small table to practise with. A **driver** lets the app connect to that kind of database. A **DSN** is a named connection set up on your computer.

1. In **Import Data > Setup**, choose **Database** in **Source**.
2. If a suitable **Saved connection** exists, select it and choose **Load**. Otherwise, choose **Form** under **Connection** and enter a clear **Connection name**.
3. Choose **Driver** using your database contact's advice. For a SQLite file, choose **RSQLite::SQLite()** and enter its full path in **Database / file**. For ODBC, choose **odbc::odbc()** and use the supplied **DSN** or connection details.
4. Open **More connection settings** if you need **Host**, **Port**, or **User**. Enter a password only when the app requests it during connection.
5. Enter **Table** and **Schema**, if required. Use **Use SQL query** only when you have a suitable query to paste into **SQL**.
6. For a small, fixed practice copy, check **Collect into memory immediately**. This loads the result and closes the connection. Leave it off only when you intend to keep a live database reference.
7. Set **Output table name**, select **Preview**, and check the columns. Select **Run import**, then check the saved table and row count.

Start with a small table or query. A live database result still needs its connection when later tasks use it. Record the date of a fixed copy when that matters to your report. A failed connection may mean an incorrect address, missing driver, or account restriction; use the exact error when asking your database contact for help.

The Windows app includes its normal app packages. An R-only installation may need a database package and a separate system driver. The optional R setup explains where to get packages. Do not put passwords in a report, log message, or shared Script.

8 Select, join, and calculate data

These tasks help you choose the rows and columns you need. Run the input task first so column lists can show its results.

8.1 Query Builder

Use **Query Builder** when you need to choose columns, join tables, filter rows, or make grouped totals in one task.

1. Select **Add Task > Data > Query Builder**. In **Select Data**, choose the **Primary input table**.
2. Use the plus buttons under **Available columns** to add columns to **Selected columns**. Use **All** if you need every column.
3. If you need another table, open **Join Tables > Add join**. Choose the **Right table**, **Join type**, and matching key columns, then select **Add join** in the dialog.
4. Use **Filter Data > Add condition** to keep matching rows. Use **Sort Data > Add sort** to choose their order.
5. Enter an **Output table name**. Select **Preview (1 000 rows)** and inspect **Results**, then select **Run & save** to keep the full result.

A join matches rows using a shared key, such as a customer number. A left join keeps the rows from the first table and adds matches from the second.

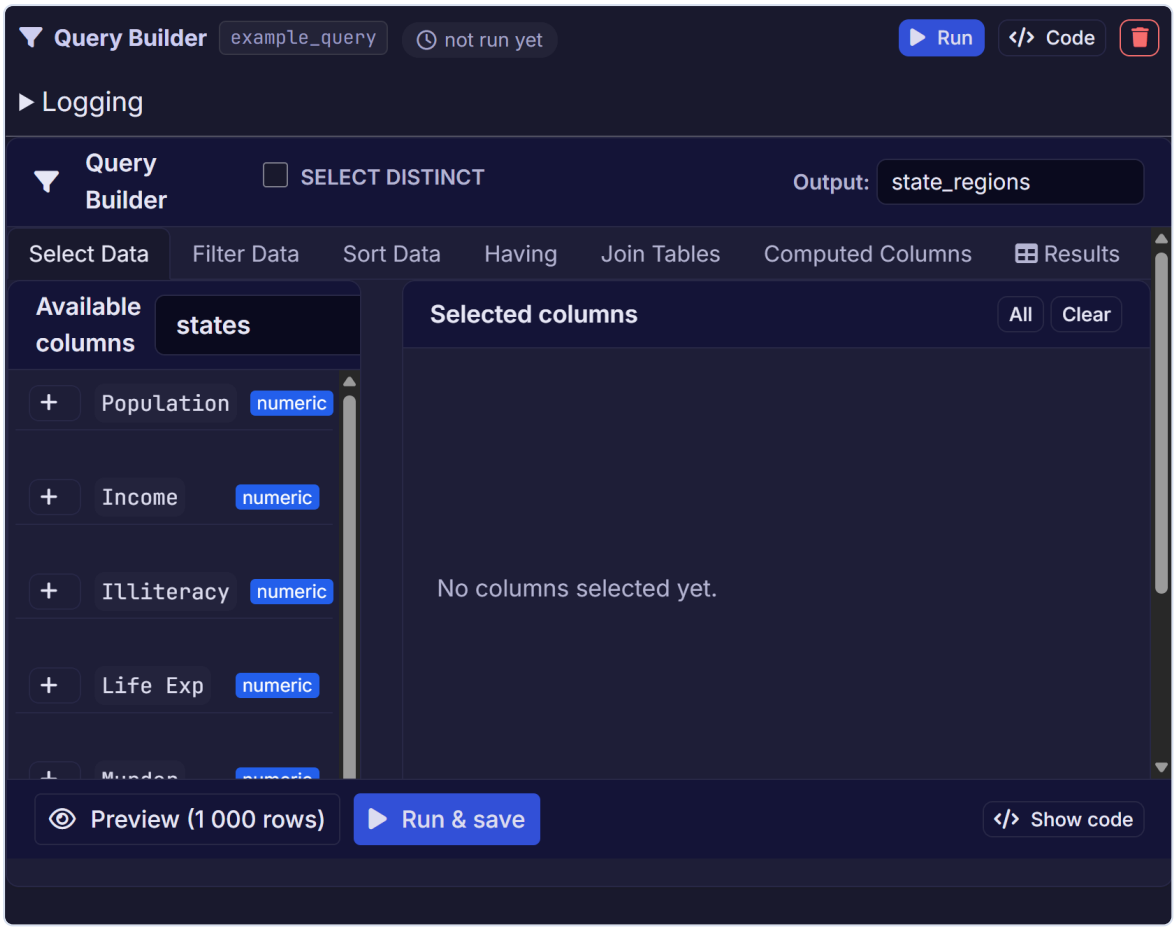


Figure 12. Query Builder separates column selection, filters, sorting, and joins into tabs. Add wanted columns to Selected columns before previewing.

Preview the result and check its row count. When it looks right, run the task to keep the result. If a key appears several times in both tables, a join can produce more rows than you expected. Check keys before calculating totals.

For grouped results, set a selected value column's calculation to **SUM** or **AVG**. AVG means mean. The other selected columns without a calculation define the groups. **Filter Data** chooses input rows; **Having** filters the calculated group results. Start with the **US states** example to inspect a saved join result.

8.2 Filter and Sort

Use **Filter and Sort** to keep matching rows and put them in order.

1. Select **Add Task > Data > Filter and Sort**, then choose **Input table** in the task header.
2. Under **Filter Data**, select **Add condition**. Choose the column, operator, and value. For example, an amount greater than 100 keeps rows above that amount.
3. If you add more conditions, check their **AND** or **OR** choice. AND needs both rules to match; OR needs at least one.
4. Under **Sort Data**, select **Add sort**. Choose the sort column and direction. Ascending goes from smaller to larger, or A to Z; descending reverses that order.
5. Enter an **Output table name**, select **Run & Save**, and open **Results**. Check a few rows at the top and bottom.

Missing values need their own rule. A missing value is an unknown entry, not the same as zero or an empty word. Use **IS NULL** to select missing values, or **IS NOT NULL** to keep known values.

Release 0.4.0 note: opening the saved Filter and Sort step in **US states - import, join and filter** can disconnect the workspace. Inspect its saved output instead, or add a fresh Filter and Sort task using the steps above. Save first. If needed, use the restart steps.

8.3 Derive Columns

Use **Derive Columns** to add a calculated column, such as a total or a value converted to new units.

1. Select **Add Task > Data > Derive Columns** and choose **Input table**.
2. Select **Add**. Enter **New column name** and **R expression for new column**. For example, if your table has `price` and `quantity`, enter `total` and `price * quantity`.
3. Enter an **Output table name** and select **Run & Save**.
4. Open **Results** and check the new column against a few values you calculate yourself.

Start with one new column. Run the task and check a few values by hand. Then add other calculations if needed. Keep units clear: changing the column name alone does not convert centimetres to metres.

Formulas use R expressions, so names and punctuation matter. Copy a working formula from a matching example when you are unsure. The **Cars - prepare an analysis table** example shows a calculated field. You can skip this task until you need a calculation that simpler tasks do not provide.

9 Optional: saved code steps

9.1 Script

Use **Script** when you have trusted R code to run as part of a flow. You can complete many projects using the other task forms instead.

1. Select **Add Task > General > Script**. Under **Setup**, choose any **Input tables / special outputs** the code needs. These choices tell the flow which earlier results to wait for.
2. Paste trusted code into **R code**. The first-project exercise supplies a complete two-line example.
3. In **Outputs**, put each result on a separate line as `name: kind`. For a table called flowers, type `flowers: table`. For a model called fit, type `fit: model`.
4. Select **Run**, then check the saved results and recorded entries under **Logging**.

Run the task and check Results and Log. If a result is missing, first compare its declared name with the code. Names are case-sensitive: `Flowers` and `flowers` are different.

The first-project exercise includes a two-line Script you can copy. The examples also contain prepared Scripts with notes. Read those notes before changing their code. Use the R Console for a quick trial, then keep repeatable code in a Script task. A console command on its own does not become a saved step in the flow.

10 Clean and reshape your data

Keep an unchanged input table, then give each cleaned result a new name. This makes it easier to check what changed. After each task, compare row counts, column names, and a few values.

Before you start, run the tasks that supply your input tables. Choose **Add Task**, then **Data**, to find the tasks in this chapter. The task controls guide explains where to run a task and name its output.

10.1 Append Table

Use **Append Table** to put rows from one table below rows from another. For example, you could combine January sales with February sales.

1. In **Tables to append**, select at least two tables.
2. Check that matching columns use the same units and compatible types. Use **Fill missing columns with NA** only if you want blanks where an input lacks a column.
3. Leave **Source ID column (.id)** blank, or enter a name if you want to identify which table supplied each row.
4. Enter an output table name and click **Run**.

The result contains the combined rows. Check its row count and any missing entries introduced by the fill option.

Appending adds rows. It does not match customer details across tables; use Query Builder for that. Open **Iris - reshape and compare** to see two species subsets combined into 100 rows.

10.2 Compare Data

Use **Compare Data** to check two tables for differences. It is useful after cleaning, importing a new extract, or reshaping data and changing it back.

1. Select **Table A (baseline)** and **Table B (comparison)**.
2. If the tables should be sorted before comparison, select their shared **Key columns**. Leave this blank to compare their current order.
3. Enter an output name and click **Run**.
4. Open **Results** and read the reported differences.

Selecting keys sorts the tables for comparison; it does not join their rows. Column types and attributes can differ even when the displayed values look similar.

If the tables have different row counts, first find why. A filter may have removed rows on purpose. The **Iris - reshape and compare** example compares a table before and after a reshape. Use its notes to see which differences are expected.

10.3 Stack Columns

Use **Stack Columns** to turn several measurement columns into rows. This is often called making data long.

1. Select the **Input table**.
2. In **Columns to pivot (stack)**, select the measurement columns. Leave identifiers, such as `flower_id`, unselected so they remain beside the stacked values.
3. Enter the **New names column** and **New values column** names.
4. Name the output, click **Run**, and check the long table in **Results**.

In **Iris - reshape and compare**, four measurements for each of 150 flowers become 600 rows. A flower identifier shows which rows belong together. Keep that identifier if you may want to turn the table back into columns later. Extra rows are expected here; they do not mean you have more flowers.

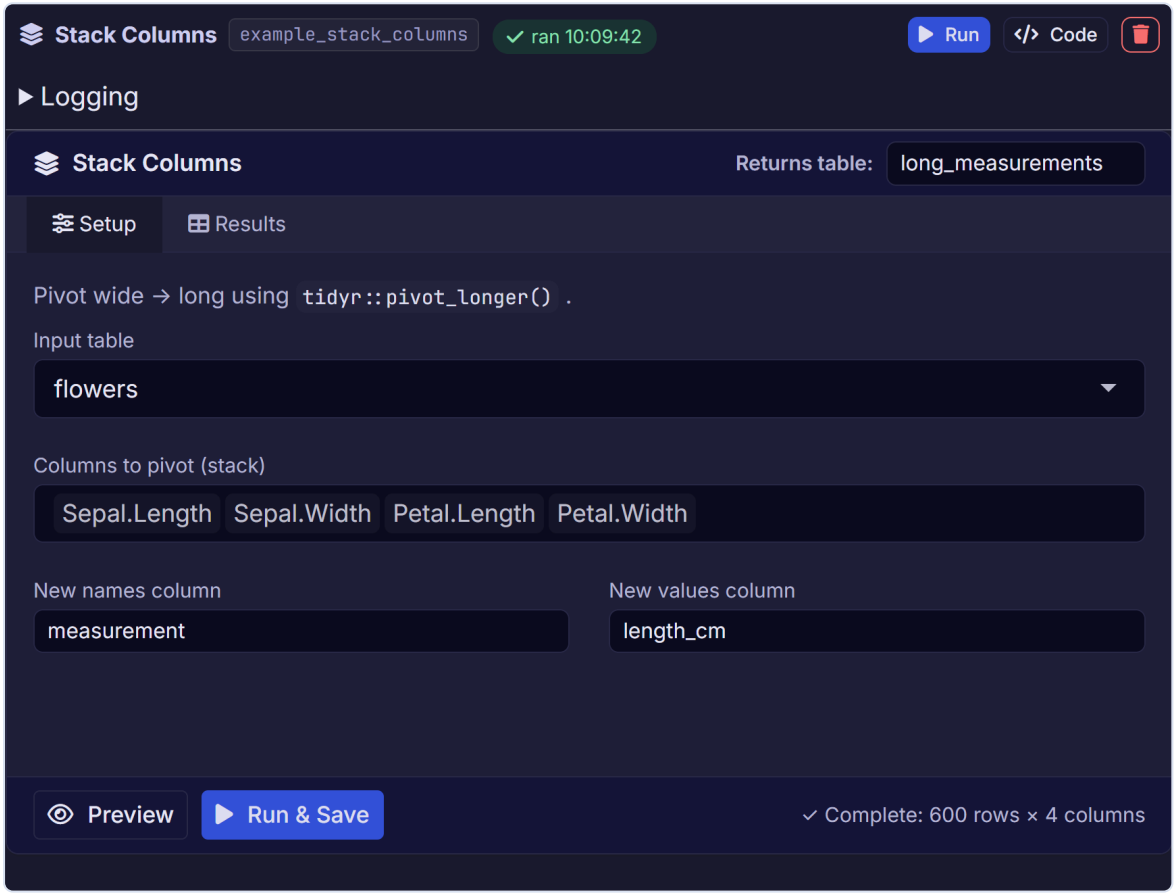


Figure 13. Stack Columns keeps unselected identifier columns and puts the selected measurements into two new columns: their names and values.

10.4 Split Columns

Use **Split Columns** to turn long-form values back into separate columns. This is often called making data wide. Here, Split Columns means reshaping a table; check its settings rather than assuming it splits words at commas.

1. Select the **Input table** and its **ID columns (keep as rows)**.
2. In **Names from**, choose the column whose values should become headings.

3. In **Values from**, choose the measurement column.
4. Name the output and click **Run**. Check the new headings and row count in **Results**.

Each identifier and new-column pair should describe one value. If several rows claim the same place, inspect those rows before continuing. They may need a summary first. The **Iris - reshape and compare** example shows a full long-to-wide round trip using a flower identifier.

10.5 Transpose

Use **Transpose** to turn rows into columns and columns into rows. It is most useful for a small table that needs a different display shape.

1. Prepare a small input first. Use **Keep / Drop Columns** and **First / Last N Rows** if you need a subset.
2. In Transpose, select the **Input table**.
3. Optionally select a **Row ID column** to supply the new headings.
4. Name the output and click **Run**. Inspect the new table and its headings.

Transpose uses the selected input table; it does not offer a separate measurement-column picker. Prepare that selection in an earlier task.

Start with a few rows. Transposing a large table can create a very wide result that is hard to view or export. Mixing text and numeric values may also change how values are stored. The **Iris - reshape and compare** example deliberately transposes a small numeric subset.

10.6 Rank

Use **Rank** to add positions such as first, second, and third based on a column's values.

1. Select the **Input table** and **Column to rank**.
2. For simple positions, choose **Minimum rank** or **Dense rank** in **Method**. Ties occur when rows have equal values. Other methods can return fractions or break ties by row order.
3. Set the **New column name**. Use **Group by** only if ranks should restart within each group.
4. Name the output and click **Run**.

The form ranks values in ascending order; it has no separate descending-order control. Minimum rank leaves gaps after a tie; Dense rank does not.

Check the highest and lowest values and at least one tie. A rank is a position, not the original measurement. Keep the original column if later work needs its values. The **Cars - rank, scale and sample** example shows ranked fuel economy.

10.7 Standardize Data

Use **Standardize Data** when numeric columns need a common scale before a later analysis. For example, weight and length can have very different ranges.

1. Select the **Input table** and **Numeric columns to standardize**.
2. Choose **Z-score** or **Min-Max** under **Method**.
3. Leave **Group by** blank for the whole table, or select groups to scale separately.
4. Name the output and click **Run**.

Z-score subtracts the mean and divides by the standard deviation, a measure of spread. Min-Max puts the observed range on a scale from zero to one. Both change the units of the selected columns.

Check the output and keep the original table for reference. A constant column has no spread, so it needs attention before some scaling methods work. Open **Cars - rank, scale and sample** for an example. Scaling is a choice for a later analysis, not a routine fix for every dataset.

10.8 Random Sample

Use **Random Sample** to choose part of a table without simply taking the first rows. It can help you try a flow on less data.

1. Select the **Input table**.
2. Under **Sample by**, choose a row count or proportion, then enter the amount. A proportion of 0.1 means 10%.
3. Enter a **Seed** for a repeatable choice with the same data and order.
4. Choose whether to **Sample with replacement**, name the output, and click **Run**.

With replacement means the same source row may be picked more than once. Without replacement, choose an amount that fits the available rows.

Check the sample count. Do not treat a small practice sample as if it were the full dataset. The **Cars - rank, scale and sample** example picks ten rows with seed 2026. Try a different seed in a saved copy and compare which rows were chosen.

10.9 Remove Duplicates

Use **Remove Duplicates** to remove repeated rows based on selected columns.

1. Select the **Input table**.
2. Under **Scope**, choose **All columns (exact row duplicates)** or **Key columns only (keep first by key)**.
3. If you chose keys, select the **Key columns** that define a duplicate.
4. Name the output and click **Run**.

The key option keeps the first matching row in the current order. Sort first if that choice matters. Rows with the same customer number can still describe different real events.

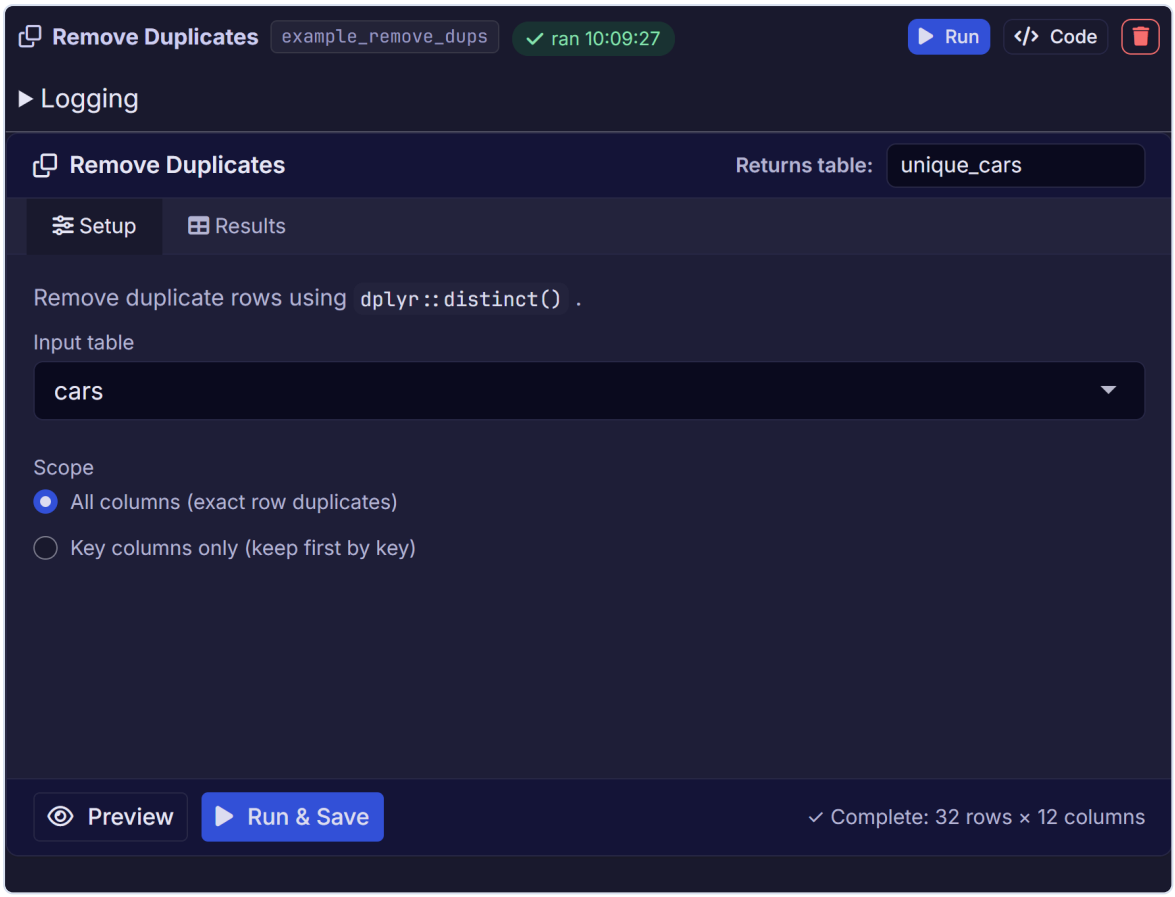


Figure 14. Remove Duplicates can compare complete rows or selected keys. Choose the scope before checking how many rows were removed.

Compare the row counts before and after. Inspect a few repeated cases if you are unsure. Two rows can look similar and still describe separate events, so do not remove them just to make the table smaller. **Cars - prepare an analysis table** adds one duplicate on purpose: its 33 source rows become 32 after this task.

10.10 Keep / Drop Columns

Use **Keep / Drop Columns** to choose which columns remain in a table.

1. Select the **Input table**.
2. Under **Action**, choose Keep or Drop.
3. Select the **Columns**. Keep retains only this selection; Drop removes it.
4. Give the output a new name and click **Run**.

Open the result and check its column names. Keep an ID column if later joins or checks need it. Removing a column from this output does not remove it from the unchanged source table. If a later task says a column is missing, check whether this step removed it. The **Cars - prepare an analysis table** example shows a smaller analysis table.

10.11 Rename Columns

Use **Rename Columns** to make headings easier to understand.

1. Select the **Input table**.
2. Under **Rename pairs**, click **Add**.
3. Select the column to rename and enter its new name. Add another pair for each additional change.
4. Name the output, click **Run**, and check the headings in **Results**.

Use unique names. Short names with letters, numbers, and underscores are easy to use in later formulas.

Renaming changes the label, not the values or their units. A weight column does not become kilograms because you call it `weight_kg`. Add a calculation when you need a real conversion. Check later tasks after renaming because their selected columns may need to change too. **Cars - prepare an analysis table** includes a rename you can follow.

10.12 First / Last N Rows

Use **First / Last N Rows** to keep a set number of rows from one end of a table.

1. Select the **Input table**.
2. Under **Rows to keep**, choose First or Last.
3. Enter the number of rows in **N**.
4. Name the output, click **Run**, and check the row count.

The table's current order decides which rows come first or last. Sort first if you want, for example, the ten largest sales.

This task is useful for a quick check or a short display. It is not a random sample. If the first rows all come from one date or group, they may give a poor picture of the whole table. See **Cars - prepare an analysis table** for a small selection.

10.13 Replace Missing Values

Use **Replace Missing Values** when you have chosen a reason to fill unknown entries.

1. Select the **Input table**.
2. Select **Columns to fill**. A blank selection means all columns, so choose columns explicitly while learning.
3. Under **Fill with**, choose the method. Enter a **Replacement value** if you choose a fixed value.
4. Name the output and click **Run**. Check both the changed values and the number of missing entries left.

Keep the raw data and note the method you chose. Filling a gap does not recover the unknown original value. It can change summaries and models. **Air quality - clean and compare** uses mean replacement to teach the controls. That example is a practice choice, not a recommendation to fill every dataset that way. You can inspect the missing-value audit before changing anything.

10.14 Recode Values

Use **Recode Values** to replace selected values with new values or labels. For example, you might change 0 and 1 to **Automatic** and **Manual** in a display column.

1. Select the **Input table** and **Column to recode**.
2. Enter an **Output column name** if you want a separate labelled column.
3. Add the old-to-new value mappings. Check the Else setting for values you did not list.
4. Name the output table and click **Run**. Count the categories to check the changes.

Keep the original codes when you may need them for a later model. A model may expect numbers even when a chart is easier to read with words. **Cars - prepare an analysis table** shows labelled transmission values and how they relate to the original codes.

10.15 Change Column Types

Use **Change Column Types** when values are stored in the wrong kind of column. Common types are text, number, date, and factor. A factor stores a set of categories.

1. Select the **Input table**.
2. Under **Type conversions**, click **Add**.
3. Select a **Column to convert** and its **Target type**. Repeat for other columns.
4. Name the output and click **Run**. Check types, values, and any new missing entries.

The form has no custom date-format field. Standard date text such as `2026-10-19` is the simplest input. If dates use another format, prepare them before converting or use a trusted Script for that conversion.

Keep identifiers as text when leading zeros matter. Use numeric types for real measurements and categories for group labels. **Cars - prepare an analysis table** changes a category's type. If a column is absent from a chart or model list, its current type is one of the first things to check.

11 Get to know your data

Look at your data before fitting a model. Check how many rows you have, which values are missing, and how each column is stored. These tasks help you spot surprises early.

Run the input task first. Find these tasks under **Add Task > Describe**. Their analysis controls and results appear in the same task panel; task controls explains the shared layout.

11.1 Characterize Data

Use **Characterize Data** for an overview of a table's columns.

1. Select the **Input table**.
2. Set the **Output table** name if you want to retain its summary.
3. Click **Analyze**, or use the task header's **Run** button.
4. Read **Results** for column types, counts, and missing-value information.

This task describes all columns in its input; it has no column-selection control. Check that measurements are numeric, group labels are categories or text, and date columns contain dates.

This is a good first step after Import Data. If a number column is stored as text, look for entries such as currency symbols or words before trying to convert it. Use Change Column Types once you know the intended meaning. The **Tooth growth - descriptive analysis** example starts with checks that help you understand the data before summarizing it.

11.2 List Data

Use **List Data** to page through a table in a readable view.

1. Select the **Input table**.
2. Set **Rows per page** to a comfortable display size.
3. Enable and name the output table if you need a retained copy.
4. Click **Load** or the task header's **Run** button. Page through the result.

Rows per page changes the view, not the number of source rows. This form has no column picker. Use Keep / Drop Columns first when you want a smaller set of columns.

A short listing helps you spot misspelled categories, unexpected dates, and values in the wrong units. Check the displayed row count or limit before assuming the listing shows every record. Keep your full input table for calculations. See **Tooth growth - descriptive analysis** for a listing alongside summary tasks. Use Export when you need to deliver a data file rather than a view inside the project.

11.3 One-Way Frequencies

Use **One-Way Frequencies** to count how often each category appears in one column.

1. Select the **Input table** and the category under **Variable**.
2. Choose **Sort by frequency** if the most common values should appear first.
3. Name the output and click **Analyze** or the task header's **Run** button.
4. Check the counts, percentages, and their total in **Results**.

The form does not offer separate percentage or missing-value switches. If a category seems to be missing, inspect blank values and spelling in the input.

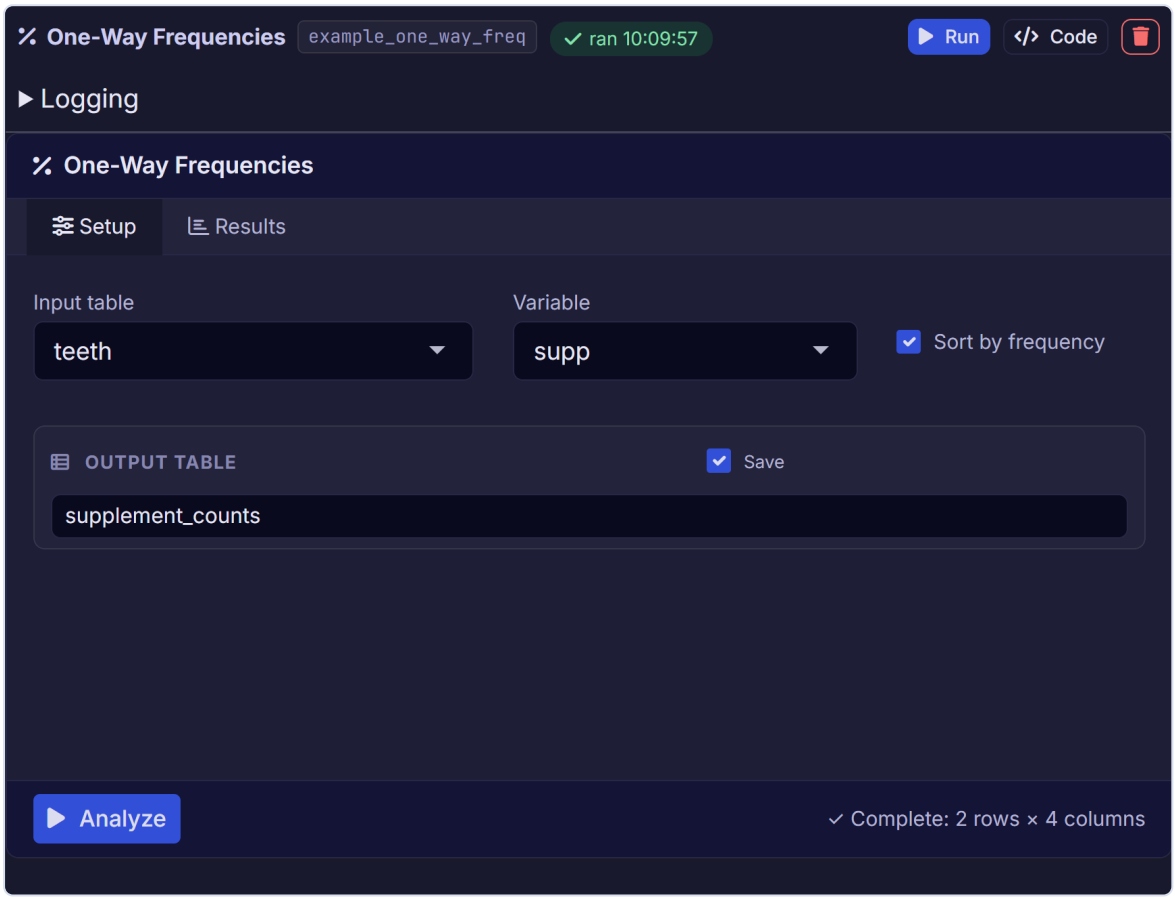


Figure 15. One-Way Frequencies counts the selected Variable. Sort by frequency changes the order, while Results lets you check the counts.

A count tells you how many records are in a group. A percentage also needs a denominator, such as all rows or only rows with known values. Check which rows the task includes before reporting percentages. **Tooth growth - descriptive analysis** counts supplement groups. You can also count the Iris species to check that each has 50 flowers.

11.4 Summary Statistics

Use **Summary Statistics** for numbers such as mean, median, minimum, maximum, and spread.

1. Select the **Input table**.
2. Under **Numeric columns**, select the measurements to summarize. Blank means all numeric columns.
3. Enable and name the **Output table** if you want to retain it.
4. Click **Analyze** or the task header's **Run** button, then read **Results**.

Read the usable and missing counts as well as the mean. Ten known values and forty missing entries do not describe fifty complete observations.

In **Data**, `n` is the number of known values and `n_miss` is the missing count. `sd` means standard deviation, a measure of spread. These checks help you judge how much information supports a mean.

The mean is the average. The median is the middle value after sorting. A few very large values can pull the mean away from the median, so comparing both can help you spot an uneven pattern. **Iris exploration** gives you a simple check: mean sepal length is about 5.8433 centimetres. Keep units in the headings when you put the results in a report.

11.5 Summary Tables

Use **Summary Tables** to calculate a value for each group, such as mean sales by region.

1. Select the **Input table** and **Row variable** for grouping.
2. Leave **Column variable (optional)** blank, or choose a second grouping column.
3. Choose the numeric **Value variable** and **Summary statistic**.
4. Name the output and click **Analyze** or the task header's **Run** button.

For the flower example, choose `Species`, `Petal.Length`, and Mean. The result has one mean for each species.

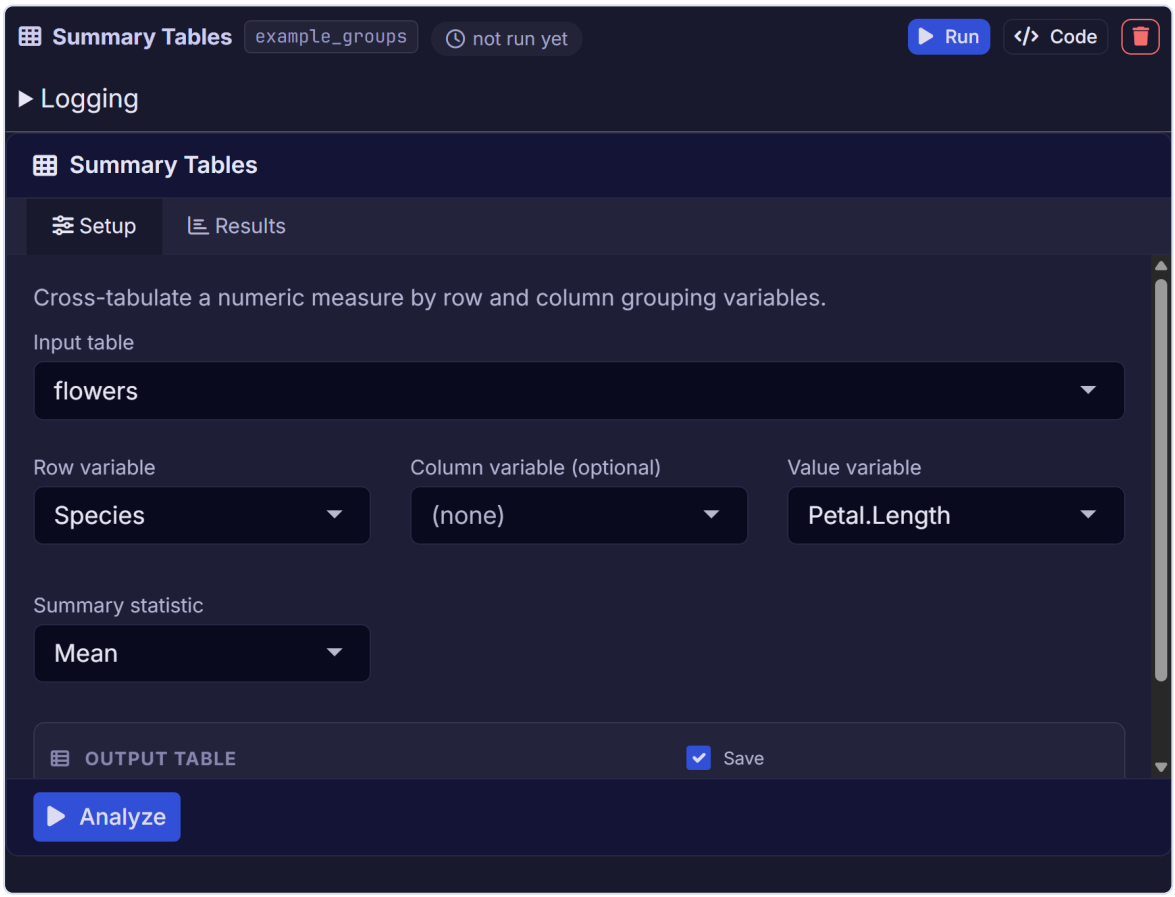


Figure 16. Summary Tables groups a Value variable by the Row variable. Leave Column variable blank for one grouping column.

Check how many groups appear and whether their labels match the source. Groups with missing measurements may have different numbers of usable values. Check the input and use a separate count summary when needed; a table of means does not also supply those counts. **Iris exploration** shows three species means, and **Tooth growth - descriptive analysis** shows another grouped summary. A grouped table is also a useful input for a report or bar chart.

11.6 Table Analysis

Use **Table Analysis** to compare two category columns in a cross-table. Each cell counts rows that match one category from each column.

1. Select the **Input table**, **Row variable**, and **Column variable**.
2. Set **Show percentages** to the view you need: none, row, column, or total.
3. Check the output-table and chi-square output switches and names.
4. Click **Analyze** or the task header's **Run** button. Read the counts before the test results.

If you enable a chi-square test, it checks for evidence of a relationship between the categories. Small or empty cells can make that test unsuitable. A **p-value** helps judge evidence against the idea that the categories are unrelated. A small value does not tell you how large or useful the relationship is. **Tooth growth - descriptive analysis** cross-tabulates supplement and dose. It helps you see how the observations are divided among the study groups.

11.7 Distribution Analysis

Use **Distribution Analysis** to examine how a numeric column's values are spread out.

1. Select the **Input table** and numeric **Variable**.
2. Set **Bins**, the number of ranges used in the histogram.
3. Choose **Density** or **Q-Q plot** if you need those displays, and check the plot dimensions.
4. Click **Analyze** or the task header's **Run** button. Inspect **Results**, **Plot**, and **Data**.

Look for a long tail, several peaks, extreme values, and missing entries. A long tail means a few values extend far beyond most of the data. The Q-Q view compares the observed pattern with a normal, or bell-shaped, reference pattern.

Use what you see to guide your next question. Do not delete a large value just because it looks unusual; check whether it is a mistake or a real observation. **Tooth growth - descriptive analysis** includes this task beside counts and grouped summaries. Together, those views help you understand both the measurements and the groups that produced them.

12 Make charts people can read

Choose a chart based on the question you want to answer. Start with simple settings, then add a clear title, axis labels, and units.

The **X** setting usually controls the horizontal axis. **Y** controls the vertical axis. Group or colour separates categories. After running, check both the chart and the number of rows it uses.

Before following a task page below, run the task that creates your input table. Add charts from **Add Task > Graph**, then open **Setup**. Give each wanted output a unique name. The task controls explain how to keep results for later steps.

12.1 Keep the output you need

Chart tasks can offer plotting data, an interactive chart, and a static image. An interactive chart may show details when you hover. A static image is a still chart that works well in Word reports and image exports.

Enable the static image output if you want to export PNG. Then use an **Export** task to write the file. Making a chart in the project does not by itself write a picture into your chosen folder.

Some chart views and chart data outputs show only the first 1,000 rows. Check the task's row limit before using a chart output as if it were the complete source table. Use a summary or sample on purpose when there are too many points to read.

12.2 Bar Chart

Use **Bar Chart** to compare category counts or a selected numeric summary.

1. Choose **Input table**, then select the category in **Category / X**.
2. Leave **Value (optional)** blank to count rows. To plot an existing numeric value, select that column instead. For a mean or other grouped calculation, make the summary table first; this form has no summary-method picker.
3. Add **Group / colour** only if it helps the comparison. Set **Title** and **Y label** to explain what the bar height means.
4. Select **Run**, then open **Results > Plot**. Compare the bars with the source counts or summary table.

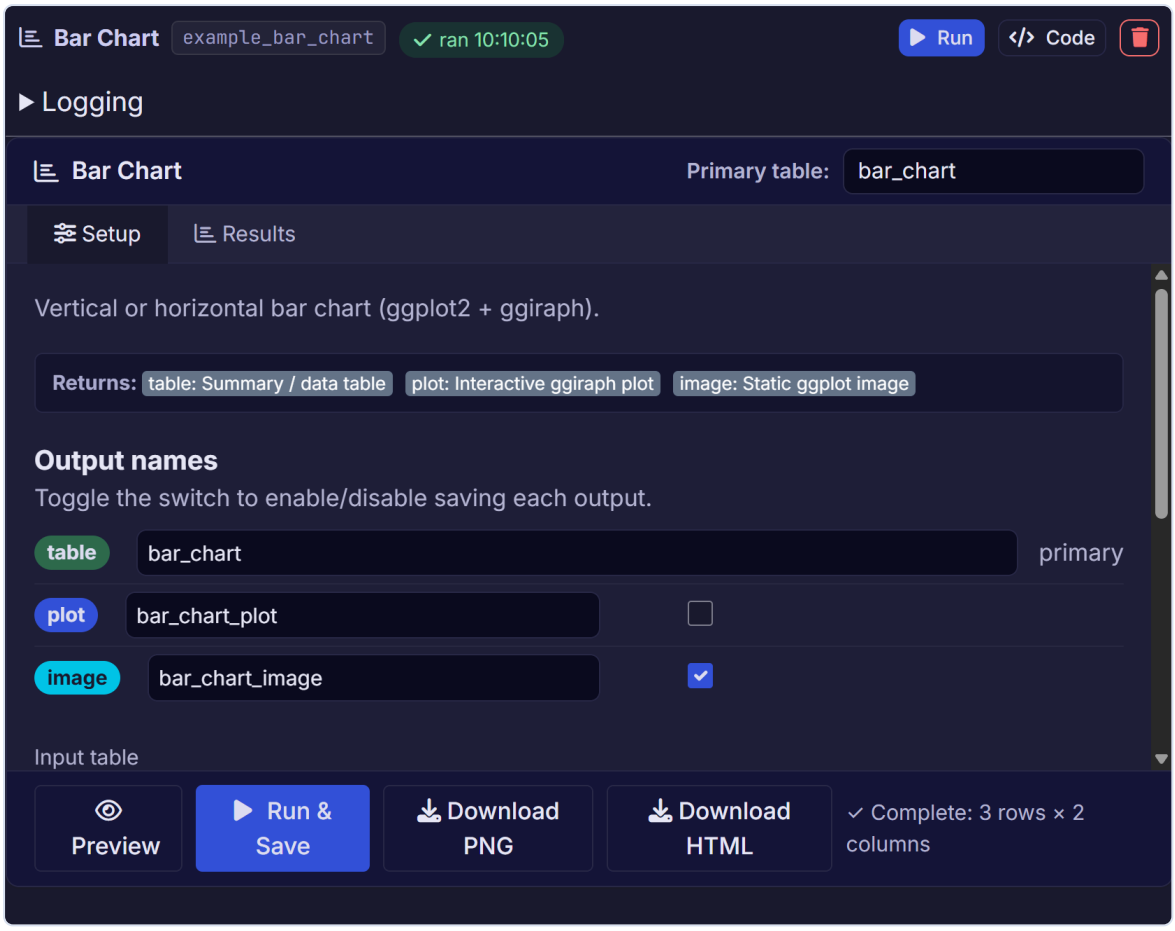


Figure 17. Leaving Value blank makes Bar Chart count observations. Select Species as Category / X to compare the three flower groups.

Keep category names readable. If the labels overlap, reduce the number of categories or change the layout. **Flowers - categorical chart gallery** counts the three species. Each bar should represent 50 flowers. Compare it with the frequency table if you are unsure what the chart is counting.

12.3 Bar-Line Chart

Use **Bar-Line Chart** to show bars and a line on the same category or time axis.

1. Choose **Input table**. Set **X / category / date** to the shared category or time column.
2. Choose **Y / value**. The bars and line use this measurement unless you also select **Secondary Y (optional)** for the second series.
3. If you use a second series, fill in **Secondary Y label**. Add a clear **Title** and axis labels.
4. Select **Run**, then open **Results > Plot**. Check which measurement each axis and series represents.

If the two values use different units or scales, explain both. Similar-looking heights on two axes do not mean equal values. Avoid using the chart to imply a relationship just because the shapes line up. **Nottingham temperatures - time charts** compares annual averages with observed ranges. The range is a difference between observed values, not a measure of uncertainty around the average.

12.4 Box Plot

Use **Box Plot** to compare the spread of a numeric measurement across groups.

1. Choose **Input table** and select the grouping column in **X / category / date**.
2. Select the measurement in **Numeric value**. Add **Fill group** only if needed.
3. Select **Run**, then open **Results > Plot**. Check the group labels and measurement units.

The line inside the box marks the median, the middle value. The box covers the middle half of the data. Points beyond the whiskers highlight values that may deserve a closer look.

Read the plot alongside group counts. A small group gives less information than a large one, even if their boxes look similar. Check unusual points before deciding they are errors. **Flowers - categorical chart gallery** compares flower measurements by species. Use Summary Tables if you also need exact group values in a report.

12.5 Bubble Plot

Use **Bubble Plot** to show two numeric measurements, with bubble size showing a third.

1. Choose **Input table**. Select two numeric columns in **X / category / date** and **Y / value**.
2. Choose a third numeric column in **Size / weight**. Add **Group / colour** if useful.
3. Select **Run** and open **Results > Plot**. Check that the size legend explains what larger bubbles mean. Keep units clear for all three measurements.

Overlapping bubbles can hide smaller ones. Try less data or a simpler Scatter Plot if the picture is hard to read. Use this task when size needs to change visibly; a Size field shared with other chart forms may not have that effect in Scatter Plot. **Flowers - numeric chart gallery** lets you compare both charts using the same observations.

12.6 Contour Plot

Use **Contour Plot** to see where pairs of numeric values are concentrated.

1. Choose **Input table** and select two numeric columns in **X / category / date** and **Y / value**.
2. Set **Title** and the axis labels so they identify the measurements and units.
3. Select **Run**, then open **Results > Plot**. Read the contour lines as a guide to where more observations lie. Compare them with a Scatter Plot of the same columns.

The smooth shape depends on the data and chart settings. It does not show a measured boundary around a real group. A small sample can give a pattern that looks more certain than it is. **Flowers - numeric chart gallery** includes contours beside the individual flower points, which makes it easier to see what the smoothing adds and what it leaves out.

12.7 Histogram

Use **Histogram** to show how one numeric measurement is distributed.

1. Choose **Input table**, then select the measurement in **Numeric column**.
2. Set **Bins**, the number of ranges counted together. Start with the default if you are unsure. Leave **Overlay density (density scale)** off for a count chart.
3. Select **Run**, then open **Results > Plot**. Check the horizontal axis and counts. Inspect the input or Characterize task for missing values.

Try a few sensible bin choices. Very wide bins can hide patterns, while very narrow bins can make random variation look important. A histogram's bars represent numeric ranges; a Bar Chart usually represents separate categories. **Flowers - numeric chart gallery** includes a histogram you can compare with its Summary Statistics. Use both when you want a picture of the spread and exact values.

12.8 Line Plot

Use **Line Plot** to show change across an ordered column, often time.

1. Choose **Input table**. Select the date or ordered column in **X / category / date** and a measurement in **Y / value**.
2. Add **Group / colour** if you need separate lines. Leave **Secondary Y (optional)** blank for a single measurement.
3. Check the source order and repeated time values. If you need one total or mean per time, prepare a summary table first.
4. Select **Run**, then open **Results > Plot** and check the time range and gaps.

A line joins observations. It does not prove what happened between them, and it is not a forecast. Check gaps and the time range so readers can see where data are missing. **Nottingham temperatures - time charts** shows 240 monthly observations. Read the date column and units before comparing years or describing a seasonal pattern.

12.9 Pie Chart

Use **Pie Chart** for a simple view of category shares when there are only a few groups.

1. Choose **Input table**, then choose the category column in **Slice category**.
2. Leave **Donut chart** off for a filled pie, or select it for a ring with the same counts.
3. Select **Run**, then open **Results > Plot**. Check each category's count and share against a frequency table.

The slices count observations by category. A numeric amount column does not supply slice weights.

Use a frequency table to confirm the totals. Many small slices are hard to compare, so a Bar Chart may be easier to read when you have many groups. **Flowers - categorical chart gallery** uses the three Iris species. Its slices should be equal because each species has 50 observations. Changing colour affects the display, not the counts behind it.

12.10 Scatter Plot

Use **Scatter Plot** to look at the relationship between two numeric columns.

1. Choose **Input table**, then select numeric columns in **X / category / date** and **Y / value**.
2. Add **Group / colour** if helpful. Set **Title**, **X label**, and **Y label** with the measurement units.
3. Select **Run** and open **Results > Plot**. Check the axis labels and units. Each visible point represents an observation, although points can overlap.

Look for a trend, curved pattern, separate groups, and unusual points. A visible trend does not prove that one measurement causes the other. Use Bubble Plot if you need a third measurement to control point size. **Iris exploration** compares sepal length with petal length by species. Try changing Y to petal width in a saved copy, then rerun the report so it shows your new chart.

12.11 Scatter Plot Matrix

Use **Scatter Plot Matrix** to compare several numeric columns in one grid.

1. Choose **Input table**, then select three or four **Numeric columns**.
2. Set **Matrix view** to **Pairwise scatterplots**. Choose **Distributions** instead when you want a separate histogram for each selected column.
3. Select **Run** and open **Results > Plot**. In the pairwise view, read each panel's row and column labels to identify its measurements.

Start with three or four columns. A large grid becomes hard to read, and display limits may show only part of a large dataset. If one pair is interesting, open a separate Scatter Plot for a clearer view. **Flowers - numeric chart gallery** compares the four Iris measurements. Use its matrix to choose a relationship to inspect, rather than trying to explain every panel at once.

13 Compare groups and fit models

A model describes a pattern in data. You do not need to write R to use these forms, but you do need to choose a method that fits your question. Start with an example whose question is close to yours.

A **response** is the result you want to explain or predict. **Predictors** are the columns you use to explain it. A **coefficient** describes part of the fitted relationship. A **fitted value** is the model's estimate for a row it used.

Run the input first. Check missing values and column types before selecting a model. After fitting, read warnings and the number of rows used. Use summaries and charts to check the data as well as the model output.

Use the Task finder to find each task's Add Task category. In its **Setup** tab, choose **Input table** before choosing columns. Near the top of the panel, keep the **Save** controls checked for the result tables you need. See Task controls if you are adding your first analysis task.

13.1 One-Way ANOVA

Use **One-Way ANOVA** to compare the means of a numeric response across groups.

1. Choose **Input table**, then select the measurement in **Response (numeric)**.
2. Select one **Group variable**. Check the group names and counts in the input table.
3. Select **Run** and open **Results**. Read the ANOVA table and the saved pairwise comparisons, if enabled.

The p-value in the ANOVA table helps assess evidence that the group means are not all equal. That overall test does not identify which pairs differ; the Tukey comparisons address pairs. Read the size of differences as well as their p-values.

The method assumes a suitable study design and data pattern. Repeated measurements on the same person, for example, need more care than unrelated observations. **Plant growth - compare experimental groups** uses 30 plant weights in three groups. Open it to see the response, group choice, summary, and report together before trying your own comparison.

13.2 t Test

Use **t Test** to compare two groups, paired measurements, or a single mean against zero.

1. Choose **Input table** and **Response (numeric)**.
2. For two unrelated groups, choose **Group (optional — two levels)** and leave **Paired measurements** off. The input must contain exactly the two groups you mean to compare.
3. For matched pairs, select **Paired measurements** and **Second measurement (same row)**. Each row must contain one matched pair. Leave the group blank.
4. For a one-sample test against zero, leave the group blank and paired setting off. There is no control for a different comparison value.
5. Select **Run**, then open **Results**. Read the mean difference and interval alongside the p-value.

Check the paired setting carefully. For paired data, choose the second measurement column; each row must contain a matched pair, such as before and after values from the same person. Two unrelated groups are not paired. Run the task and read the mean difference and interval alongside the p-value. **Plant growth - compare experimental groups** includes a two-group example using a prepared subset.

13.3 Linear Models

Use **Linear Models** to explain a numeric response using one or more predictors, including groups.

1. Choose **Input table** and select the numeric **Response**.
2. Select **Predictors**. A group column should use a category type when its values name groups; check it with Characterize or Change Column Types first.
3. Select **Run**, then open **Results**. Check the coefficients, model fit, and number of observations used.

Read the coefficient and fit tables together. A coefficient for a group is usually a comparison with a reference group; it is not that group's raw mean in every model. The **Plant growth** example shows a model beside its group comparisons. Keep the model description in your report so a reader knows which predictors you included. A completed fit alone does not show that the model is useful.

13.4 Linear Regression

Use **Linear Regression** to model a numeric response from selected predictors.

1. Choose **Input table**, then select **Response** and **Predictors**.
2. Keep the coefficient and model-fit outputs. Select **Save Fitted values (augment)** if you also want row-level fitted values and residuals. A residual is the observed value minus the fitted value.
3. Select **Run**, then open **Results**. Check the coefficients, fit, warnings, and the saved result tables.

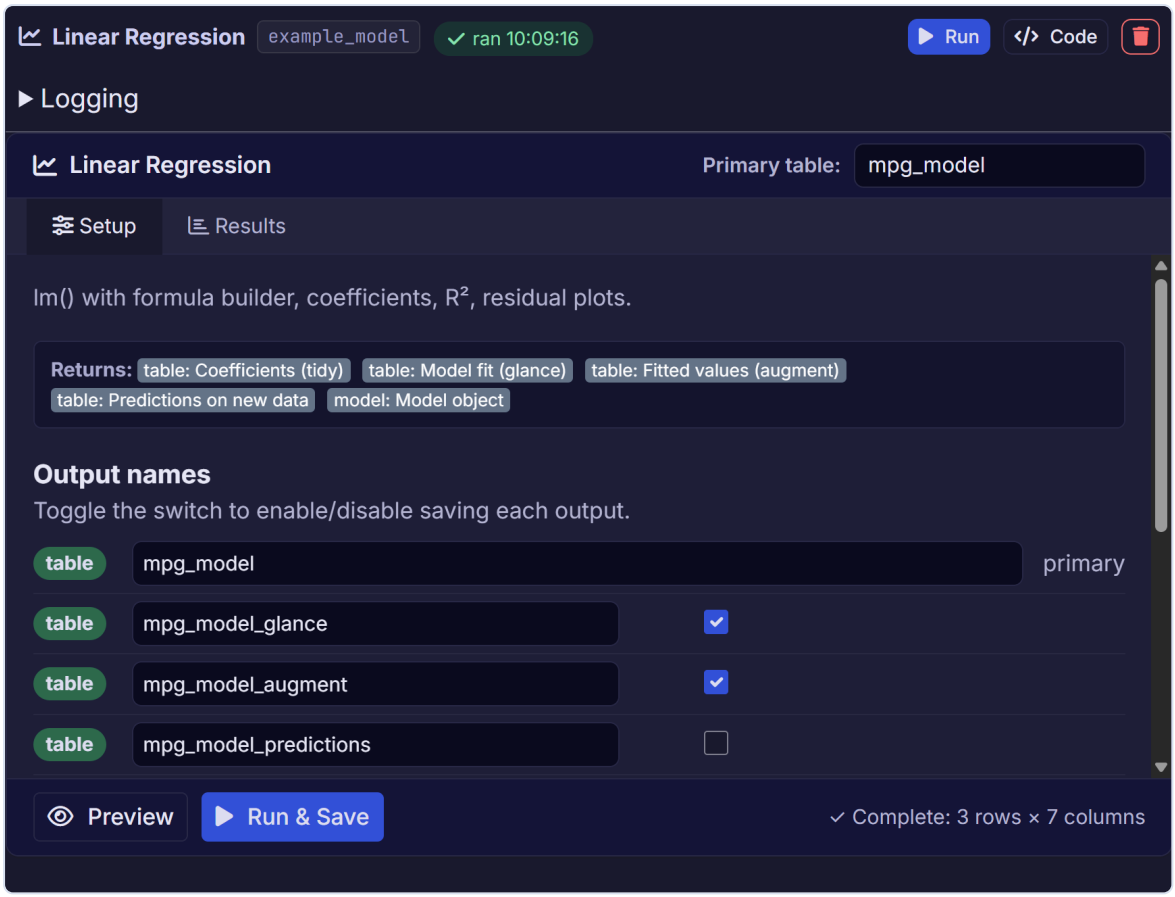


Figure 18. Linear Regression uses a response and selected predictors. Optional saved outputs include fitted values and a model object.

The **Fuel economy model** example uses 32 cars and predicts fuel economy from weight and horsepower. In that dataset, weight is in thousands of pounds. Check those units before explaining a coefficient. A fit to these observations is not proof that changing one car feature will cause the predicted change. Try removing one predictor in a saved copy and compare both the model and its warnings.

13.5 Logistic Regression

Use **Logistic Regression** for a response with two possible outcomes, such as 0 and 1.

1. Choose **Input table**. In **Response (0/1)**, select a column coded with 0 and 1. Confirm what 1 means.
2. Choose **Predictors** and keep the output tables you need. Enable fitted values if you want row-level fitted results.
3. Select **Run**, then open **Results**. Inspect the coefficients and fit information; read warnings before interpreting the result.

Fitted probabilities range from zero to one. The coefficients use a different scale, so they are not direct percentage-point changes.

Cars - binary outcome model treats 1 as manual transmission and 0 as automatic. It is a useful way to check what a probability refers to. Read warnings about convergence or separation. These can mean the model cannot estimate the relationship reliably. A green task status does not make extreme estimates safe to use without checking them.

13.6 Generalized Linear Models

Use **Generalized Linear Models** when the response needs a model family other than ordinary numeric regression.

1. Choose **Input table**, **Response**, and **Predictors**.
2. Choose **GLM family** using a matching example or analysis plan. For example, **poisson** is used for counts in the Loom breaks example.
3. Select **Run**, then open **Results** and inspect warnings, coefficients, and fit information.

The family describes the kind of outcome. The form uses that family's standard link, which controls how predictors connect to the expected outcome. You do not choose a separate link here.

Loom breaks - a count model uses a Poisson model for 54 counts of loom breaks. Counts are whole numbers of events, not percentages or rates. If exposure time differs between rows, the simple count example may not suit your question. Run the task and read warnings and fit information. A successful fit does not by itself show that the chosen family matches the data.

13.7 Nonlinear Regression

Use **Nonlinear Regression** to fit one of the supported curve forms to numeric measurements.

1. Choose **Input table** and the numeric **Response (y)**.
2. Choose one numeric column in **Predictor (x — first selected used)**, then choose **Curve preset**. Start with the matching example's preset.
3. Check for zeros and negative entries; some curve forms require positive values. The task prepares its own starting values.
4. Select **Run**, then open **Results** and compare the fitted curve with the observed points.

Puromycin - fit a nonlinear curve uses positive concentration and reaction-rate values from a prepared subset. It shows a power curve as a teaching example. A different preset changes the model; it is not simply a repair button for a failed fit. If fitting fails, check valid input values and the selected form first. Be cautious about predictions beyond the observed range.

13.8 Correlations

Use **Correlations** to measure how pairs of numeric columns vary together.

1. Choose **Input table** and select the wanted **Numeric variables**. Blank means all numeric columns, including any numeric IDs, so an explicit selection is often safer.
2. Select **Run** and open **Results**. The task calculates Pearson correlations using available pairs when values are missing.
3. Compare the table with Scatter Plots of interesting pairs. Values near 1 or -1 show a strong straight-line relationship. Values near zero do not rule out a curved pattern.

A correlation does not show that one value causes another. Groups and unusual points can also change the result. **Flowers - multivariate structure** compares the four Iris measurements. Inspect the species groups as well as the combined result. Check the number of usable observations when missing values mean different pairs use different rows.

13.9 Principal Components

Use **Principal Components** to combine several numeric columns into fewer summary directions. The task is often called PCA.

1. Choose **Input table**. In **Numeric variables**, select only the measurements you want to combine. Leave IDs and group codes out.
2. Keep the wanted outputs near the top of the panel. Enable **Save Component scores (augment)** if you need a component value for each row.
3. Select **Run**, then open **Results**. Check variance explained and loadings. The task scales the measurements automatically.

Variance explained shows how much of the data's spread each component represents. Loadings show how the original columns contribute.

Flowers - multivariate structure uses the four flower measurements and keeps species labels out of that input set. It uses scaling inside PCA. A component is a calculated direction, not a new measurement with a simple original unit. Use its scores to explore patterns, then return to the original columns to explain them.

13.10 Cluster Analysis

Use **Cluster Analysis** to group rows with similar numeric measurements.

1. Choose **Input table** and the wanted **Numeric variables**. Standardize them first if a common scale is needed; this task does not do that automatically.
2. Choose **Method** and set **Clusters** to the requested number of groups.
3. For **k-means**, set **Random seed** so a rerun with the same data can repeat its random start. For **Hierarchical**, choose **Linkage** using your analysis plan or example.
4. Select **Run** and open **Results**. Inspect group sizes and the saved cluster assignments.

Run the task and inspect group sizes and assignments. Very small groups or results driven by one large-scale column need attention. **Flowers - multivariate structure** creates three clusters. Three requested clusters do not prove that the data contain three natural groups, and clusters need not match the known species. Compare the result with the source measurements and charts before giving the groups names.

14 Check patterns and work with time

These tasks answer more specific questions. Read the matching example first, and keep the meaning and order of your data clear.

Run the input task first. Find each analysis task's category in the Task finder, add it, then choose **Input table** under **Setup**. Keep the output controls enabled for results you want to use later.

14.1 CDF Plots

Use **CDF Plots** to see what share of observations is at or below a value. CDF stands for cumulative distribution function.

1. Choose **Input table** and the measurement in **Numeric variable**.
2. Add **Group / colour (optional)** only when you need separate group curves.
3. Select **Run**, then open **Results** and inspect the plot. Pick a value on the horizontal axis and read the share on the vertical axis. A share of 0.75 means about 75% of observations are at or below that value.

This view can help you compare thresholds without choosing histogram bins. Check the units and missing-value count. It describes the data you supplied, not a guarantee about future measurements. **Nile flow - distribution and stability** includes this plot alongside a probability plot and control chart, so you can compare different views of the same 100 annual observations.

14.2 Probability Plots

Use **Probability Plots** to compare a numeric column with a normal distribution, a common bell-shaped pattern.

1. Choose **Input table**, then choose the measurement in **Numeric variable**.
2. Keep the plot output enabled. This task uses a normal reference; there is no reference-distribution picker.
3. Select **Run**, then open **Results** and inspect the plot. Points near the line suggest a similar pattern. Curves and distant points suggest differences, especially near the extremes.

The plot is a check, not a pass-or-fail certificate for every later analysis. Read it with a histogram and the study context. Do not remove observations just to make the points straighter. **Nile flow - distribution and stability** gives you a worked normal probability plot. Use it to learn how the line and points relate before interpreting your own data.

14.3 Control Charts

Use **Control Charts** to inspect measurements in their recorded order and flag unusual patterns.

1. Sort the input into observation order before adding the task. Choose **Input table** and **Measurement**.
2. Leave **Subgroup** blank for an individuals chart. Select a subgroup column only when the rows form meaningful subgroups for an Xbar chart of subgroup means.

3. Check **Western Electric run rules** if those extra pattern checks suit your use. These rules flag certain sequences, not just single extreme points.
4. Select **Run**, then open **Results**. Inspect the plot and the table of points with control limits and flags.

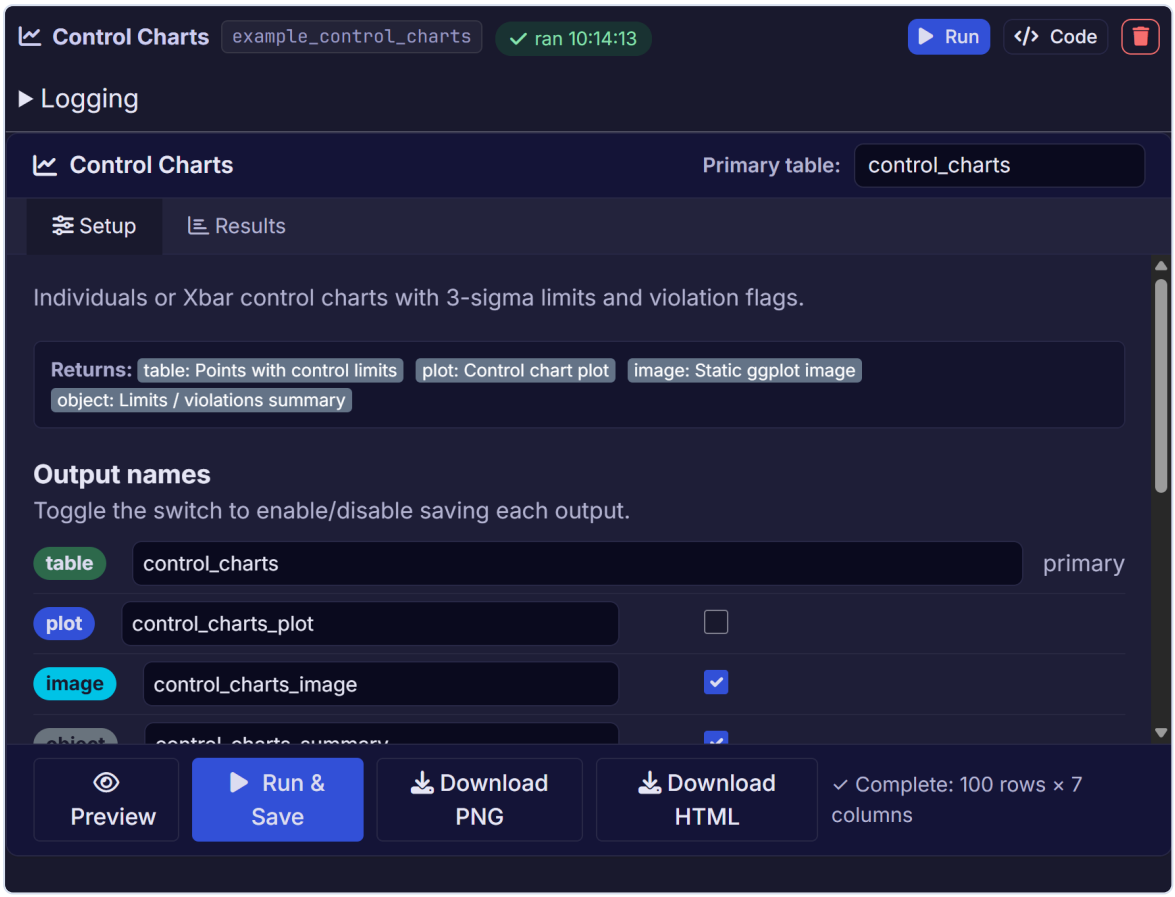


Figure 19. Control Charts uses the row order of the input. A blank Subgroup setting selects an individuals chart; run rules add pattern checks.

The app estimates limits from the data. Those limits are different from an outside target or an allowed tolerance set by a customer. A point inside a limit is not proof that it meets a product requirement. **Nile flow - distribution and stability** uses an individuals chart for practice. River observations are a teaching dataset, not a controlled factory process. Read the example's notes before borrowing its settings for another purpose.

14.4 ARIMA / Basic Forecasting

Use **ARIMA / Basic Forecasting** to estimate future values from an ordered numeric series.

1. Prepare one numeric value per equal time interval, such as one month. Sort the table into time order. The task needs at least eight values and no missing series values.
2. Choose **Input table**, then select **Series value (rows in time order)**.
3. Set **Forecast horizon (steps ahead)**. Enter 12 for twelve future intervals.
4. Select **Run**, then open **Results**. Inspect the fitted model, forecast values, and intervals.

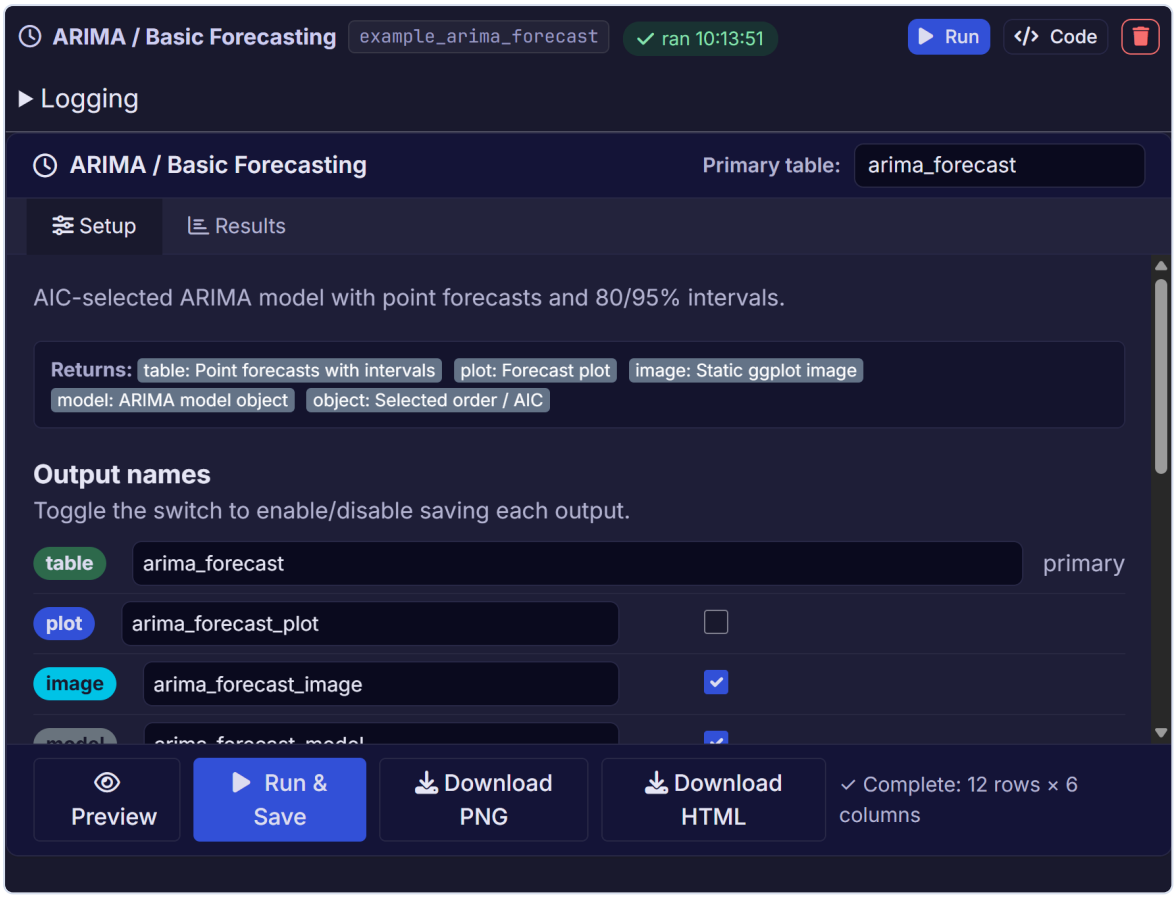


Figure 20. The forecasting form uses an ordered value column and a horizon measured in steps. It does not infer the calendar interval from a date column.

Air passengers - forecast a monthly series starts with 144 monthly values and forecasts 12 more steps. The task does not automatically turn every dated table into a tuned seasonal model. Check the settings and model used. The intervals show model uncertainty; they do not promise that future events will stay within them. Keep the time labels clear when reporting the result.

15 Time until an event

Survival tasks use a time column and an event indicator. An event might be a failure, a repair, or another outcome. A censored observation means the event had not been seen by the end of that observation's follow-up.

For numeric event columns, check that **1 means the event occurred** and **0 means it was not observed during follow-up**. Convert other numeric codes first. For text or category columns, use **Event level** to name the value that means an event. The example prepares 0/1 codes and explains the change in its preparation task.

15.1 Life Tables

Use **Life Tables** to estimate how the chance of remaining event-free changes over time.

1. Choose the prepared **Input table**. Select **Time to event** and **Event indicator (1 = event, 0 = censored)**.
2. If the event column contains text or categories, set **Event level** explicitly. For a numeric 0/1 column, keep that coding.
3. Add one **Strata** column if you want separate curves. Only the first selected column is used.
4. Select **Run**, then open **Results**. Check the number of usable rows and events. Confirm that time uses one unit and has no unexplained negative values.

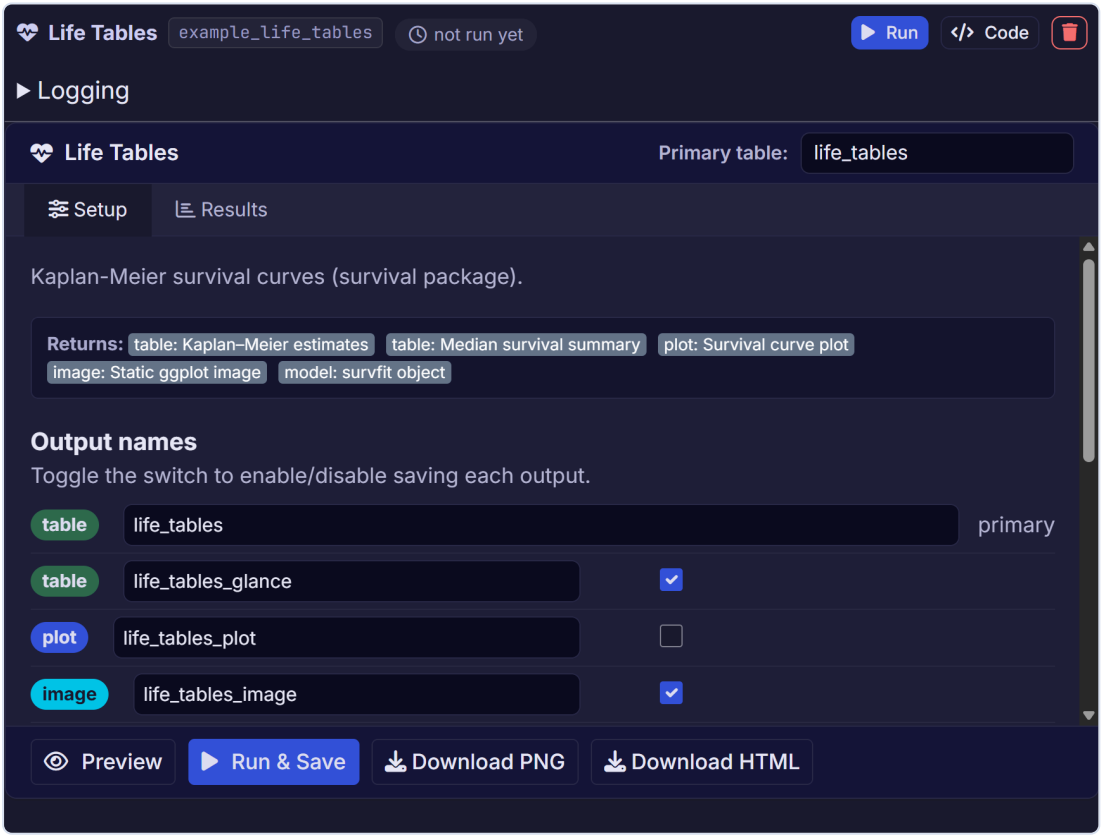


Figure 21. Life Tables separates follow-up time, event coding, and optional strata. Check the prepared event column before reading the curves.

Lung cancer - survival analysis keeps the 228-row original data and prepares a separate input for analysis. Its original event code of 2 is converted to 1. Open the preparation task before reading the curve. The example teaches app controls and coding checks. It is not a guide for making care decisions. Use suitable analysis advice before applying these methods to important decisions.

15.2 Proportional Hazards

Use **Proportional Hazards** to model the relationship between predictors and the rate at which an event occurs over time. This is often called a Cox model.

1. Choose the prepared **Input table**, **Time to event**, and **Event indicator (1 = event, 0 = censored)**.
2. Select the predictors in **Covariates**. Set **Event level** if the event column uses text or categories.
3. Check missing values and event coding, then select **Run**.
4. Open **Results**. Read the coefficients, intervals, warnings, and number of included observations together.

Lung cancer - survival analysis includes an illustrative model. A hazard ratio compares fitted event rates; it is not a person’s absolute event probability. Use the example to learn the controls. Ask for method advice before relying on the model’s assumptions.

16 Run steps conditionally or repeatedly

Most first projects can use a straight sequence of tasks. Add a Branch when a decision should choose a path. Add a Loop when the same steps should repeat. Try the supplied example before building a large version.

Before starting, save a copy of your project and run its input tasks. Both tasks are under **Add Task > Control Flow**. A **path token** is a name that tells later tasks which route they belong to. Selecting a table supplies data; selecting a path token controls whether or how often a task runs.

16.1 Branch

Use **Branch** to run tasks only when a condition is met. For example, you could make a summary only when a table has enough rows.

1. Add **Branch**. Under **Setup**, select the source table in **Inputs / dependencies**.
2. Set **Number of outgoing paths**, then expand the first path's settings. Enter a **Path label** and a unique **Path token**.
3. For a row-count check, choose **Dataset row count** in **Condition**. Choose **Dataset**, **Operator**, and **Row count**. In the flower example, `>=` and `100` mean at least 100 rows.
4. Set another path's **Condition** to **Fallback / else path** if you need a route when no ordinary condition matches. Select **Validate**, then **Evaluate**, and read the decision.
5. Open each task that should follow that decision. In **Run after branch/loop path**, select the intended path token.
6. Run the active flow. Expand **Logging** to check the chosen route and any skipped tasks.

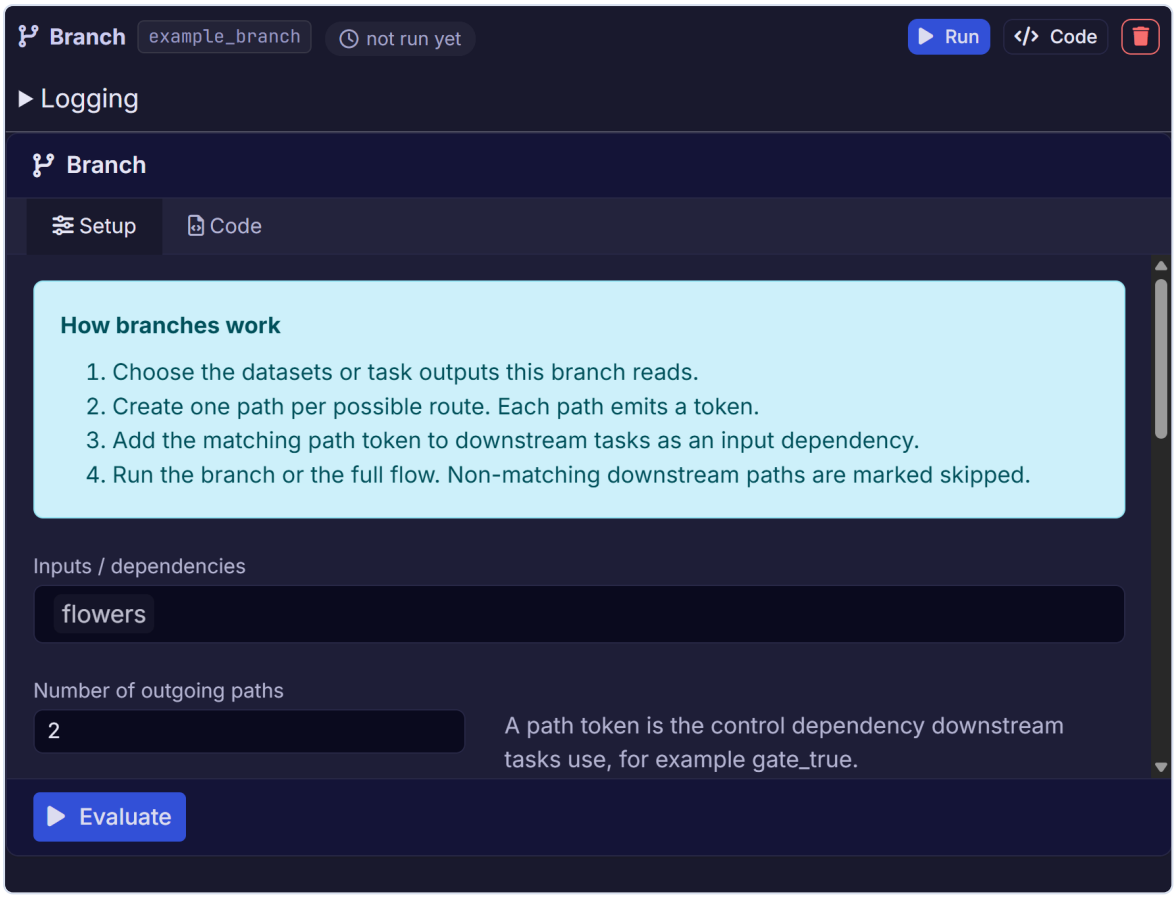


Figure 22. Branch settings name each path and its condition. The following task must also select that path in Run after branch/loop path.

Choosing an input table alone does not attach a task to a branch path. That separate path choice is needed. **Flowers - conditional quality gate** checks for at least 100 flowers. Change the threshold in a saved copy and run again to see which path is taken.

16.2 Loop

Use **Loop** to repeat tasks for several values, files, tables, or groups.

Start with **Flowers - repeat by species**. Its group loop runs one summary for each species. Follow the connection from the Loop path to the repeated Script. Inspect the three saved summaries and the iteration entries in the log.

To practise without writing code, save a copy of **Flowers - repeat by species** and use its prepared tasks. An **iteration** is one trip through the repeated steps.

1. Open the example's **Loop** task. Under **Setup**, check that **Iteration type** is **Groups**.
2. Choose **Source dataset / input alias** and **Group / value column**.
3. Keep the **Loop variable** name used by the prepared Script. This name lets that code use the current group. Do not change it in only one place.
4. Keep the example's **Path token** and **Output naming rule** while learning. The rule's `{index}` part keeps a separate result for each iteration.
5. Select **Preview loop**, then inspect **Preview**. Check that the listed groups are the ones you expect.

6. Open the repeated task and select the loop token in **Run after branch/loop path**, alongside its normal inputs. Run the active flow and check the per-group results and log entries.

When building your own loop, add it from **Add Task > Control Flow > Loop**. Use a unique path token and an output rule such as `{task}_{index}_{output}`. Check later Report and Export inputs after changing output names.

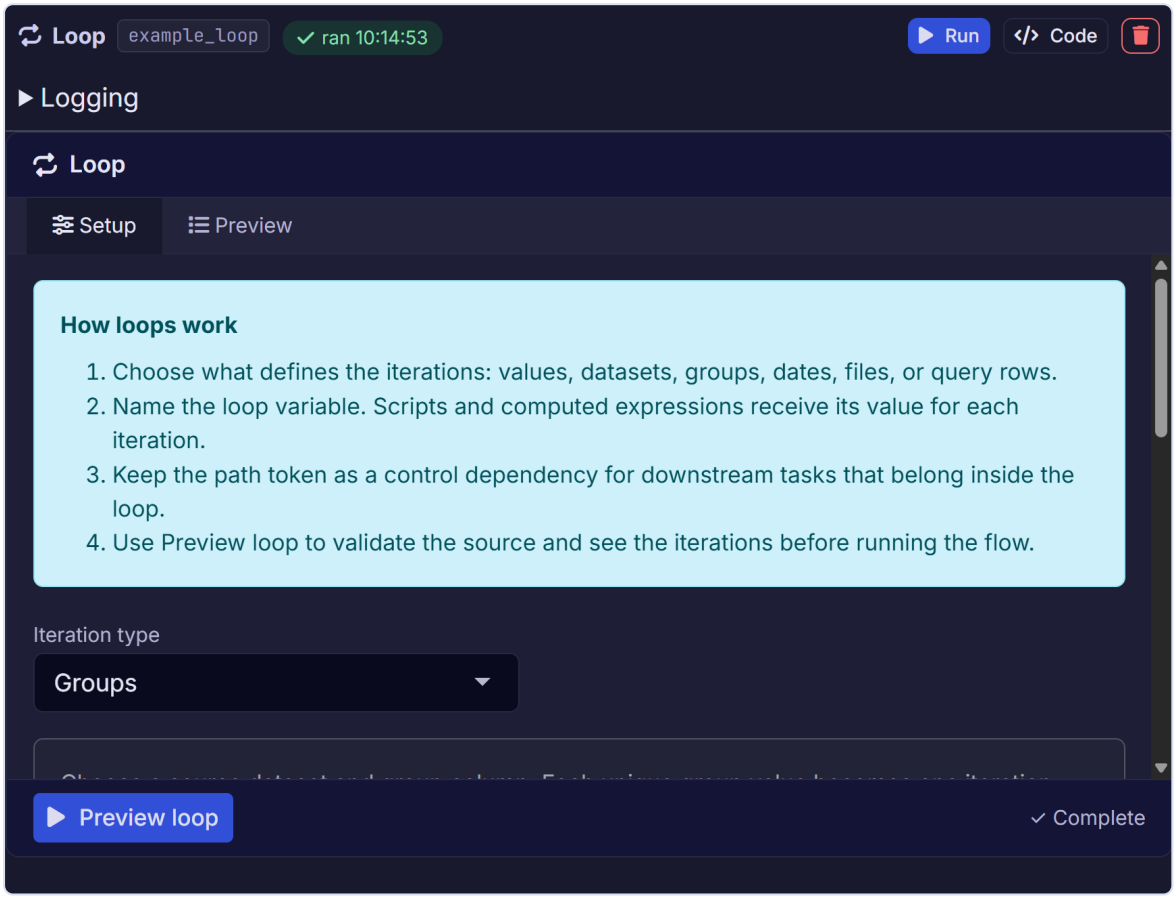


Figure 23. Loop settings choose the source of repeated work. Preview loop lets you check the planned groups before running the flow.

For exported files, give each iteration a different filename. A supported `{index}` placeholder can add the iteration number. If every iteration writes the same file with Overwrite on, only the last file may remain.

16.3 Choose a loop source

Source	What you provide
Parameter list	One value per line in Iteration values
Files	File paths or matching file patterns
Datasets	All loaded tables; Source dataset names the input alias used by repeated visual tasks
Groups	A source table and grouping column
Date ranges	One date or range per line, such as `2026-01-01` or `2026-01-01/2026-01-31`

Source	What you provide
Database query results	A prepared result table whose rows supply the items

Check the preview before running. A missing file or a mistaken group can otherwise repeat the wrong work many times. If the repeated step uses code, begin with the working example and keep the loop variable name consistent with that code.

16.4 Start small

Try two or three iterations first. Check output names, file paths, row counts, and logs. Once those are right, expand the loop.

Place a final Report or Export Log outside the repeated path if it should run once at the end. Attach it inside the path only when you want one per iteration. A stopped or failed loop does not remove files that earlier iterations already wrote.

17 Keep a useful log

A log helps answer three questions: What ran? What changed? What went wrong? The app records task events, and you can add messages that explain your own checks.

17.1 Choose how much to record

Open a configured task and expand **Logging**, near the top of its panel. Keep **Record task logs** enabled, then choose **Minimum severity to record**. Severity means how serious a message is. The app calls these choices levels.

Level	Use it for
DEBUG	Extra details while finding a problem
INFO	Normal progress and results
WARN	Something to check, even if the task continues
ERROR	A failure or serious problem

INFO is a useful starting point. A minimum of WARN keeps warnings and errors but leaves out normal INFO messages. This choice affects which new messages are kept; raising the detail later cannot recover messages that were never recorded.

17.2 Customize a task's message

1. Expand **Logging**. Find the rule whose **When** setting is the event you need, such as task completion.
2. Check **Enable this rule** and choose its **Level**. The level must also pass **Minimum severity to record**.
3. Edit **Message**. Use **Field to insert into a message** and the rule's insertion button for values such as task name or output row count.
4. Set **Preview an event** to the same event. Use **Example values** to try sample counts, then read **Rendered log entries**. This preview does not run the task.
5. Run the task and inspect its recorded entries under Logging. Check the timestamp so you read the new entry.

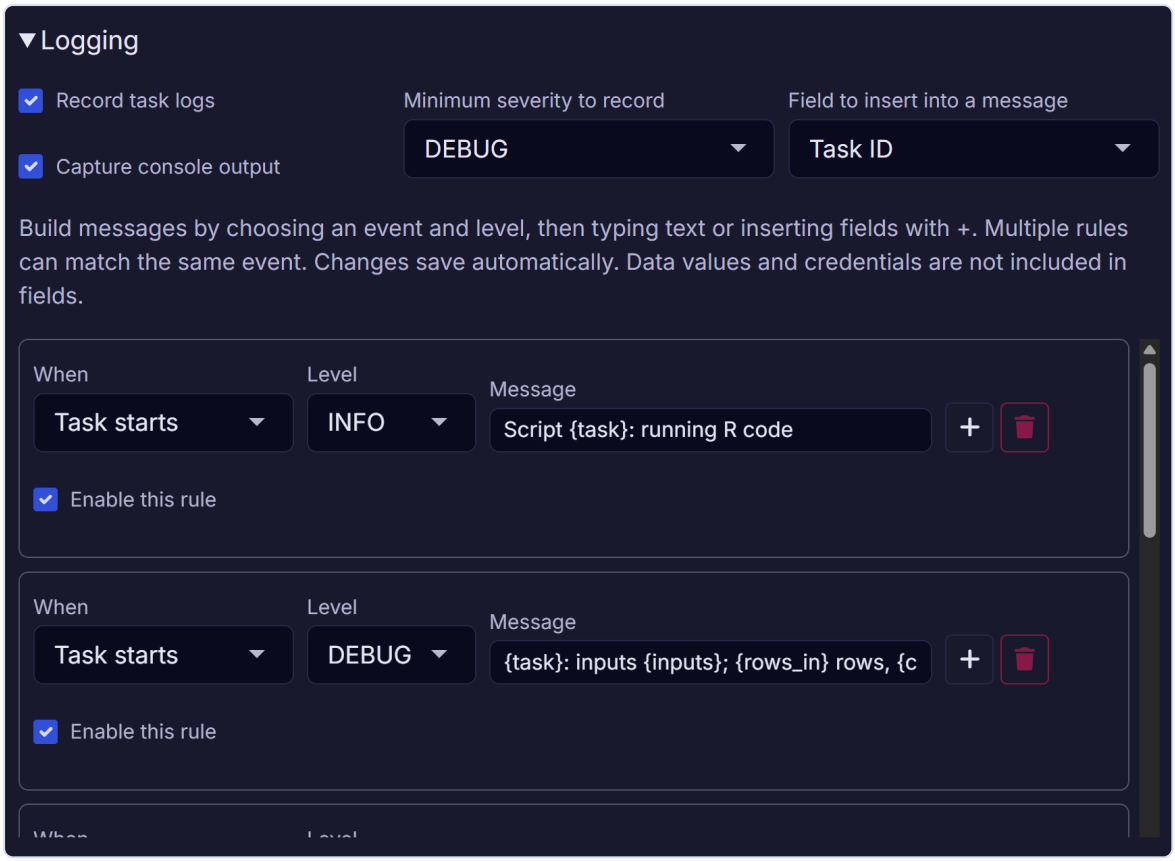


Figure 24. Task Logging lets you choose events, levels, and message fields. Test the wording in the preview before running the task.

For a table-cleaning task, you could use:

```
{task_name}: {rows_in} rows in; {rows_out} rows out.
```

The words in braces are placeholders. The app fills them with values from the run. Use the picker so you do not need to memorize their spelling. An output count may not exist at task start, so use it in a completion message. The practice text above stays identical in both editions; you can translate the words around the braces in your own messages.

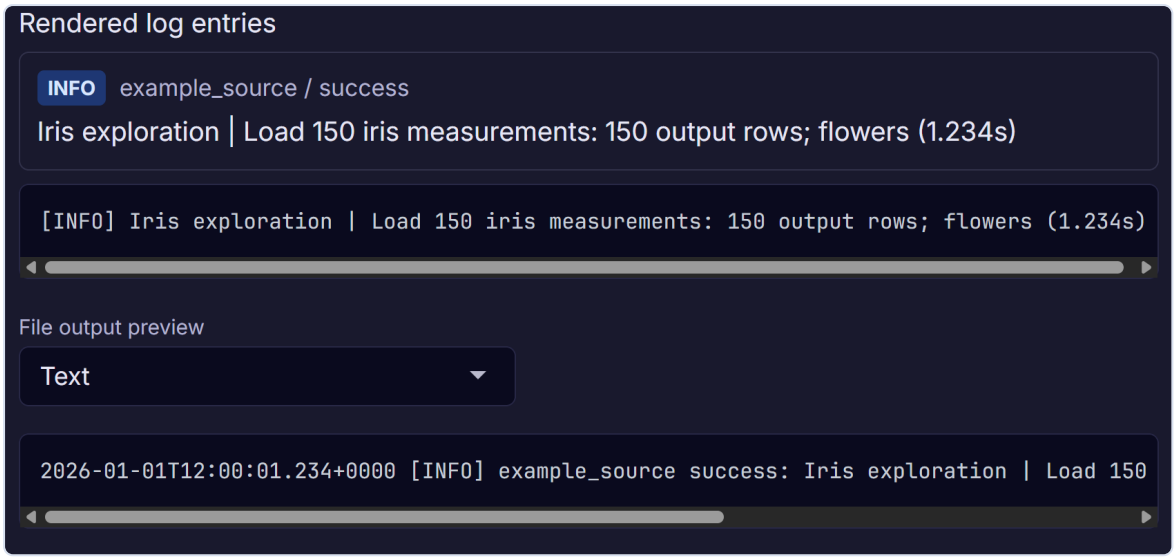


Figure 25. Read the rendered entry before running the task. File output preview lets you inspect Text, CSV, or JSON without writing a file.

Keep the original-message field in warning and error rules. That way, your friendly wording does not hide the reason something failed. If a placeholder stays visible, check the field and event in the preview.

17.3 Add a message inside a Script

This is optional. A Script can add a plain message with the supplied `log_event()` helper:

```
log_event("Checked the source data before making the report", "INFO")
```

Use messages to explain decisions, such as a filter or missing-value choice. Avoid printing private row-level information just to show that a step ran. The **Cars - a report with an execution audit** example includes a custom warning you can inspect.

A warning message alone does not stop a Script. If you need a quality condition to choose a different path, use a Branch. If you need code to fail on a bad condition, ask the code's author to make that check explicit.

17.4 Export Log

Use **Export Log** to write selected task logs to a file as part of your flow.

1. Add Export Log from Output.
2. Choose the tasks in **Run after and include these tasks**. Include Report if you want its completion recorded. Blank means all other tasks in the active flow.
3. Choose **Text**, **CSV**, or **JSON** in **Format**, then set **Minimum severity to export**. Leave **Show example entries instead of retained logs** off when checking your own run.
4. Enter a full **Destination file path** in a folder you can write to. Include the matching file ending, such as `/pipeline-log.csv` after the folder address for CSV.
5. Check **Overwrite an existing file** only if reruns should replace it. Select **Export log**, or **Run** in the task header. Open the file outside the app and check its dates and messages.

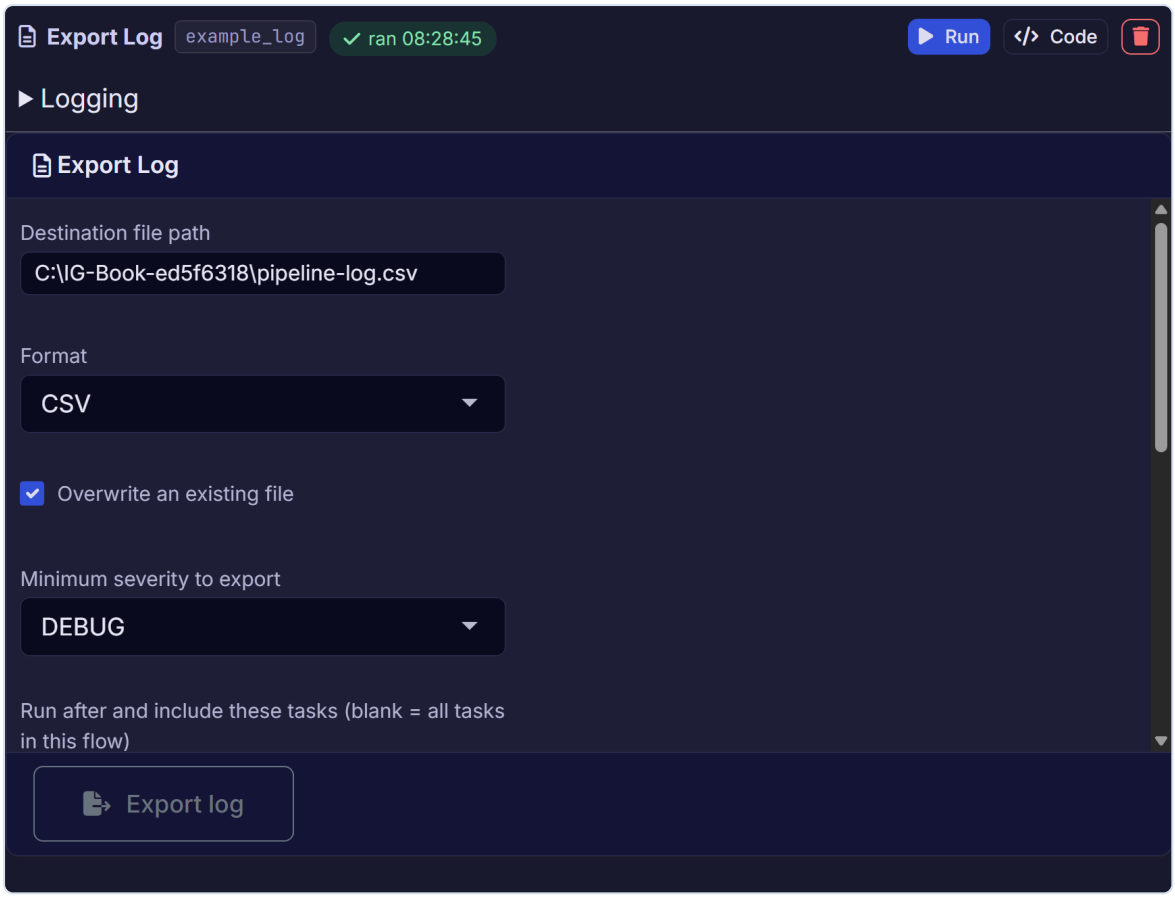


Figure 26. Export Log needs a complete file path, a format, and the tasks to wait for. Its severity filter applies to messages already retained by those tasks.

TXT is easiest to read as plain text. CSV opens in a spreadsheet with separate fields. JSON is useful when another program will read the records. Export Log writes a file rather than making a new data table.

If the flow stopped at an earlier error, run Export Log by itself afterwards when possible. It can help you collect the error messages. Its file cannot include the exporter's own later completion message.

17.5 Put the log in a report

Built-in headings in saved reports keep their original language. This guide quotes **Pipeline execution log** exactly so you can find it in the downloaded file. You can write your own report title, notes, and log messages in French or English.

1. Open a Report task after running the tasks whose logs you need.
2. Under **Appearance**, check **Include pipeline log in the report**. Set **Minimum log level** to INFO for normal progress, or WARN for warnings and errors only.
3. Select **Generate Report**, then **Download HTML** or **Download DOCX**. Open the file and look for **Pipeline execution log**.

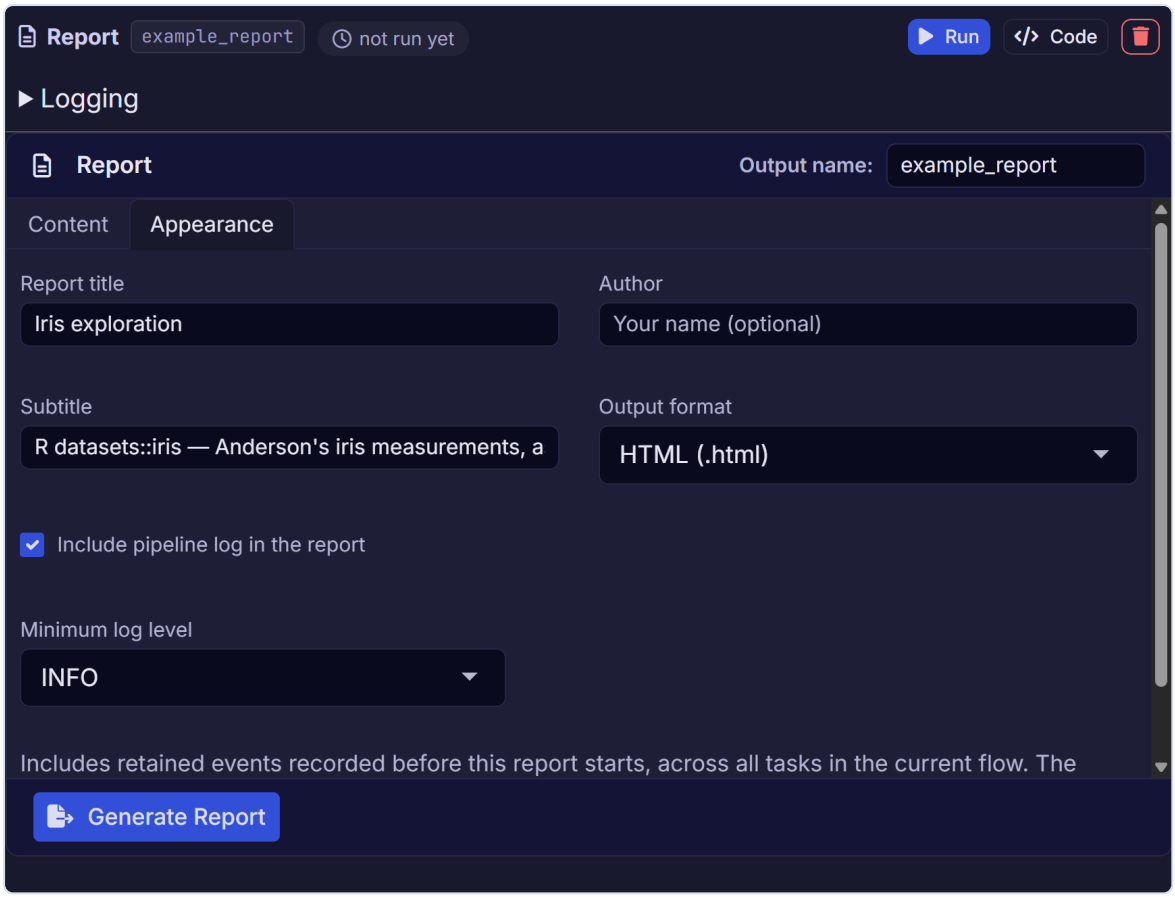


Figure 27. Report Appearance controls the log appendix and its minimum level. Generate the report again after changing this setting.

The report takes a snapshot of other tasks' available logs before it is made. It cannot include its own completion at that point. Add Export Log after Report if you need that event in a separate file. You can also make a report containing only a log by enabling the log option without selecting ordinary report items.

17.6 Read dates as well as messages

Tasks keep up to 1,000 recent entries each, across runs. A saved project keeps those available entries. Running a flow again does not erase its older messages.

Use the timestamps and run identifiers to compare runs. Old warnings may still appear after you fix a problem. Long or very chatty runs can push old entries out of the retained log, so export important logs in time.

17.7 Use startup logs for startup problems

The desktop **Session** menu offers **Open startup log**, **Open logs folder**, and **Save diagnostic log** for connection and startup problems. Task logs help with the analysis itself. Before sharing a log, remove private data, passwords, and file paths you do not want to publish.

18 Share reports and files

Choose what your reader needs. Send a report for reading, a CSV for working with data, or a project for editing the flow. You can send more than one when each has a clear purpose.

18.1 Report

Use **Report** to bring tables, charts, and notes into an HTML or Word document.

First run the tasks that supply the results. For a chart in Word, keep its static image output enabled and run the chart again if needed.

1. Add Report from Output.
2. Under **Content**, use the plus button beside a result in **Available outputs**. Check that it appears under **Report contents**. Add items in the order you want readers to see them.
3. For each item, fill in **Section heading** and **Note**. Explain the data source, units, and important choices. Leave **Render style** at Auto unless you need a specific display.
4. Under **Appearance**, choose HTML or Word in **Output format** and enter **Report title**.
5. Turn on **Include pipeline log in the report** if you want an execution-log section.
6. Select **Generate Report**. When it finishes, select **Download HTML** or **Download DOCX**. Save the document and open it outside the app.

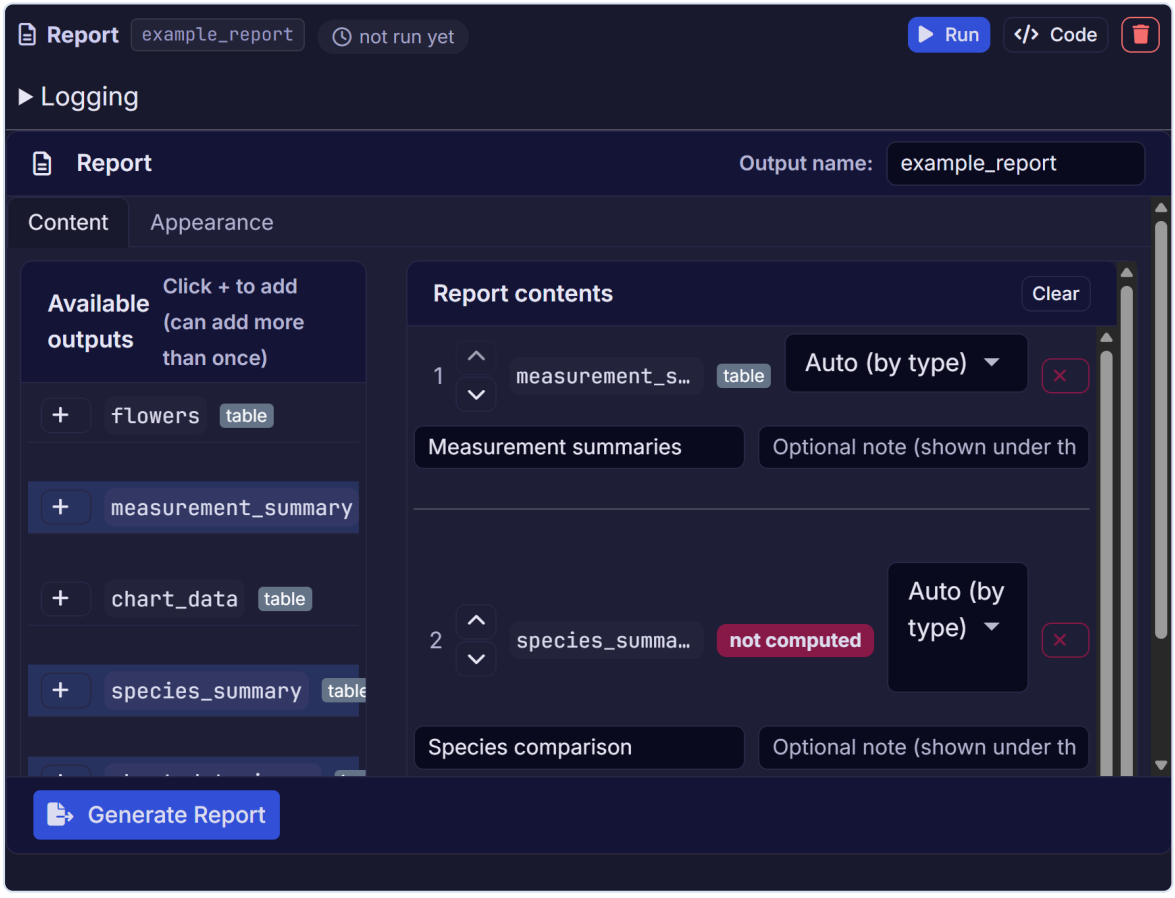


Figure 28. Add outputs on the left to build Report contents on the right. Each item can have its own heading, note, and render style.

Check the document itself before sharing it. A missing result can lead to a warning and an incomplete section. Run its source task, then regenerate the report. **Iris exploration** shows tables and a chart together. **Cars - a report with an execution audit** focuses on logging.

18.2 Choose HTML or Word

HTML opens in a web browser. It is a good first choice and can include interactive tables where supported. Download it and check it outside the running app.

Word produces a .docx file that you can edit in a compatible word processor. Use static charts and tables; a Word page cannot keep browser hover controls. If your chart is missing, enable its static image output, run the chart, and select that output in the report.

The Windows Setup includes the normal report tools. An R-only installation needs extra reporting packages, and Word also needs Pandoc. See Optional: install with R if that is how you run the app.

18.3 Pick a content style

Start with **Auto**. It chooses a display based on the actual result. Use a table style for a table and a figure style for a static chart. Print style shows an object's text when that is what you need.

Give a reader enough context to understand a result. For example: **Mean petal length by species, 150 flowers, measurements in centimetres** is more useful than **Output 2**. If you change the data or chart, run Report again; a previously saved document will not update on its own.

18.4 Export

Use **Export** to write a table, chart, or supported object to an outside file.

Before starting, run the task that creates the result you want to export.

1. Select **Add Task > Output > Export**. Under **Setup**, choose **Output to export**, then a compatible **Format**.
2. Select **Choose folder...** and choose a destination you can find again.
3. In **File name (without extension)**, enter a name such as `species-summary`. Check the full file address shown by the app; it adds the format's file ending.
4. For an image, check its dimensions and **DPI**. Check **Overwrite if the file already exists** only if you mean to replace that file.
5. Select **Export**. Open the finished file from the chosen folder and check its contents.

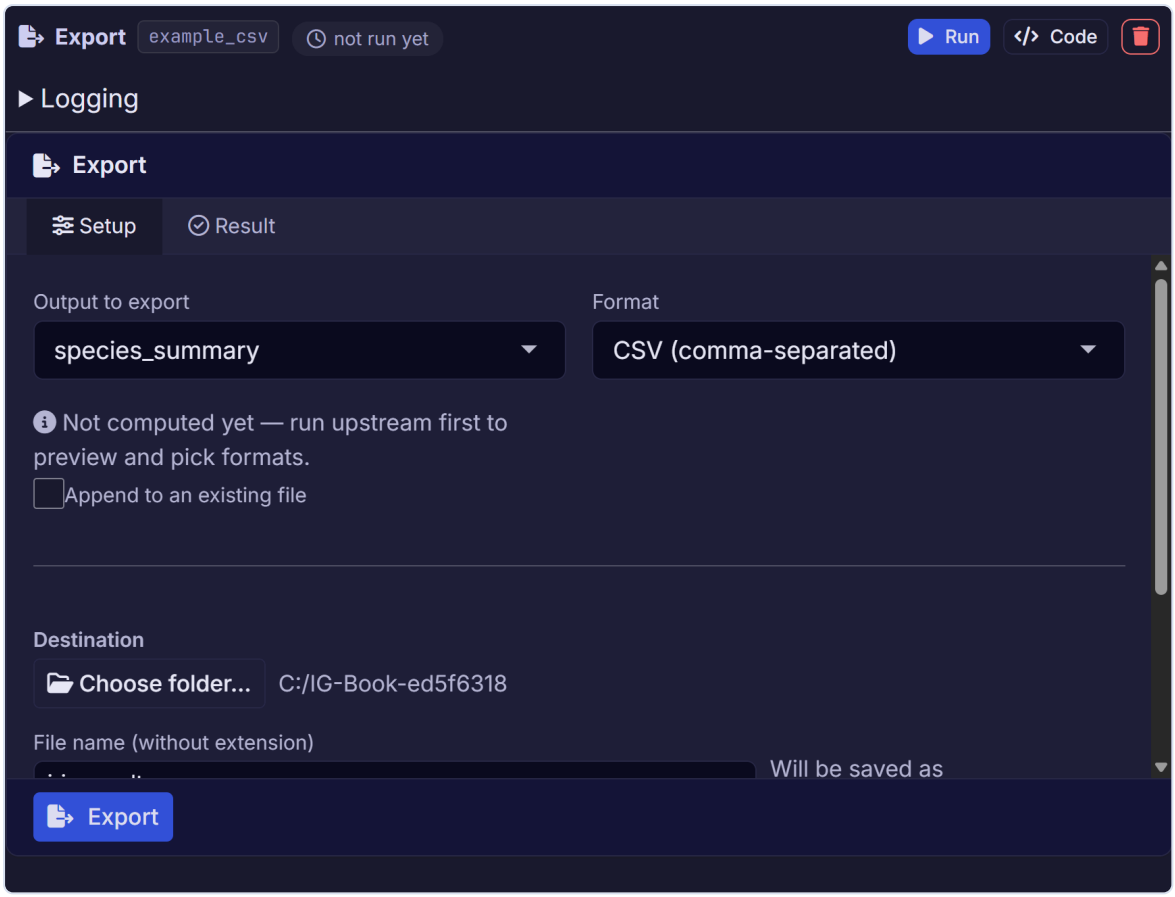


Figure 29. Export separates the destination folder from the filename. Check the complete file address before writing or replacing a file.

Start with CSV for data and PNG for a still chart. The source must be the right kind of result: exporting chart data as CSV is different from exporting the chart's static image as PNG.

Check Overwrite before rerunning. Append is available for CSV and TSV when supported; it adds rows to an existing file. It is not a general way to add a sheet to Excel. An Export task writes a file and does not create a new table for later tasks. The original result remains the source for those tasks.

18.5 Choose an export format

What you need	A useful choice	What to check
Data for many other apps	CSV or TSV	Dates, text IDs, and missing values after opening
A spreadsheet	XLSX	Column types and the receiving spreadsheet's limits
A still chart	PNG	Readable labels, dimensions, and resolution
A chart that can scale	PDF or SVG, when offered	That the receiving app can open it
Typed data for compatible tools	Parquet or Feather	Support in the receiving tool
Structured data for a program	JSON	The format expected by that program
An R result for another R user	RDS or RData	The recipient's R and package support

You do not need to learn every format. Choose the one your reader or next tool can use. If leading zeros disappear in a spreadsheet, import the ID column as text in that spreadsheet. If a CSV opens as one column, use the spreadsheet's text/CSV import command and choose comma as the separator.

18.6 Check a finished delivery

Save the project. Open the exported report and data files outside the app. Check titles, units, row counts, charts, and the log's dates. Make sure you are sending the latest files from the folder you actually chose.

If you need a record that the report finished, send a final Export Log made after Report. Its events are later than the snapshot inside the report.

19 Optional: try the R Console

You can use the app without the console. It is there for quick questions and small trials when you want more control.

R is the calculation language behind the app. A **command** is an instruction you ask R to carry out. The console shows the command and its result. Start with this calculation:

```
2 + 3
```

The result is 5. You do not need to install anything to try this inside the desktop app.

19.1 Open the console and run a command

1. Choose the **R Console** tab. In the desktop app, you can also use **View > R console**.
2. Select the command area, type `2 + 3`, and press Enter. Check that the transcript shows 5.
3. Use Shift+Enter to add a line without running it. Use Ctrl+Enter to run a block of lines.

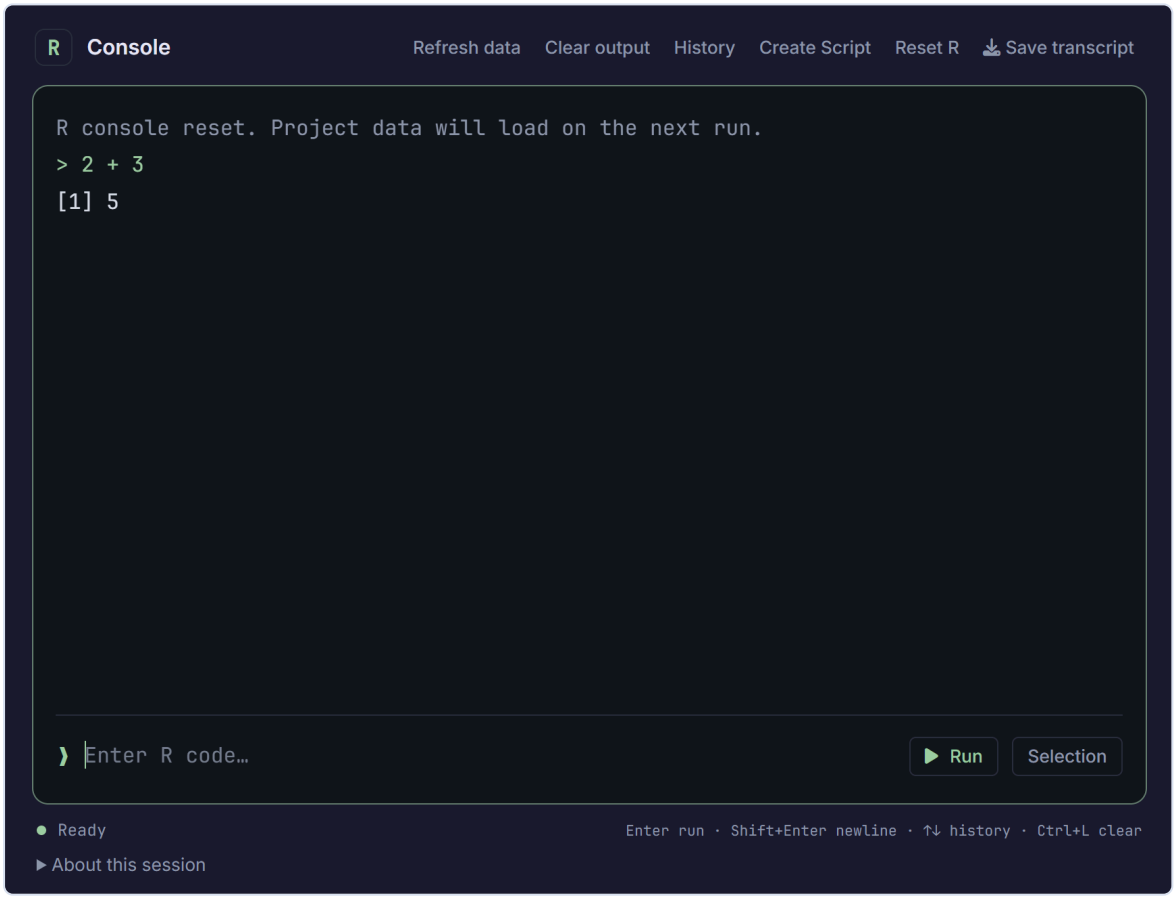


Figure 30. The R Console has a command area and transcript. Project-data controls supply working copies for trials; they do not add a step to the flow.

Commands, R output, and console messages keep their original text. The buttons use your chosen language.

The console loads copies of available project data when you first run a command. Select **Refresh data** after changing data in the flow. This replaces the imported copies while keeping other console values. Changing a console value does not automatically change the project's table or flow.

19.2 Read a command before running it

Commands can calculate, change values, read files, or write files. Use code you understand or trust. For a simple practice with R's built-in flower data, try:

```
mean(datasets::iris$Sepal.Length)
```

The result is about 5.8433. Here, `mean` asks for an average. The rest identifies the sepal-length column in the built-in flower data. You do not need to memorize that punctuation to use the task forms.

19.3 Find and reuse a command

Use the history controls to find earlier commands. Up and Down move through history when the cursor is at the command boundary; Alt+Up and Alt+Down also help. Ctrl+R searches command history.

Tab or Ctrl+Space offers completions when available. A completion helps type a name; it does not check that the finished command answers your question. Read errors and edit the command before trying again.

19.4 Clear, Stop, and Reset

Action	When to use it	What to expect
Clear output or Ctrl+L	You want a cleaner display	Clears the transcript and plots; working values remain
Stop	A command is taking too long	Stops console work and clears its working values; the transcript and history remain
Reset R	You want a fresh console	Clears its working values, transcript, history, draft, and plots
Restart R server	The app's calculation service has stopped	Restarts the app session; save first if possible

After Stop, the next command loads fresh project-data copies. Values created only in the console are gone. These actions do not turn a trial command into a saved task.

19.5 Keep useful code in a task

1. Select the useful code in the console, or leave it as the current draft or last command.
2. Select **Create Script**. Open the new Script task and check the copied code.
3. Check **Input tables / special outputs** and **Outputs**. An inferred name still needs to match the result the code creates.
4. Run the Script and check its saved results. See the Script task guide for the output-line format.

20 Optional: export the flow as R code

The **Code** tab can export the active flow as a complete R script or a code bundle. This is useful when someone needs to rerun the analysis without opening Individual Guide.

A script is a text file of R commands. A bundle includes the script and supporting files with instructions. Choose a complete-flow export for a handoff; an individual task's code may depend on earlier steps.

You can skip code export for everyday app use. A saved project is usually the easiest way to return to your work.

20.1 Share code that someone can run

1. Run the flow and check its results. Select the flow you mean to share.
2. Open **Code**, or **View > Generated R code** in the desktop app. Choose **Complete script** in **Code view** if you want to read all the exported instructions together.
3. Select **Download .R** for one complete script, or **Download bundle** for a ZIP containing the pipeline, its supporting functions, and a README.
4. Save the download. Include any needed input files separately and tell the recipient which outputs to expect. The code bundle does not collect your outside data files or database credentials.

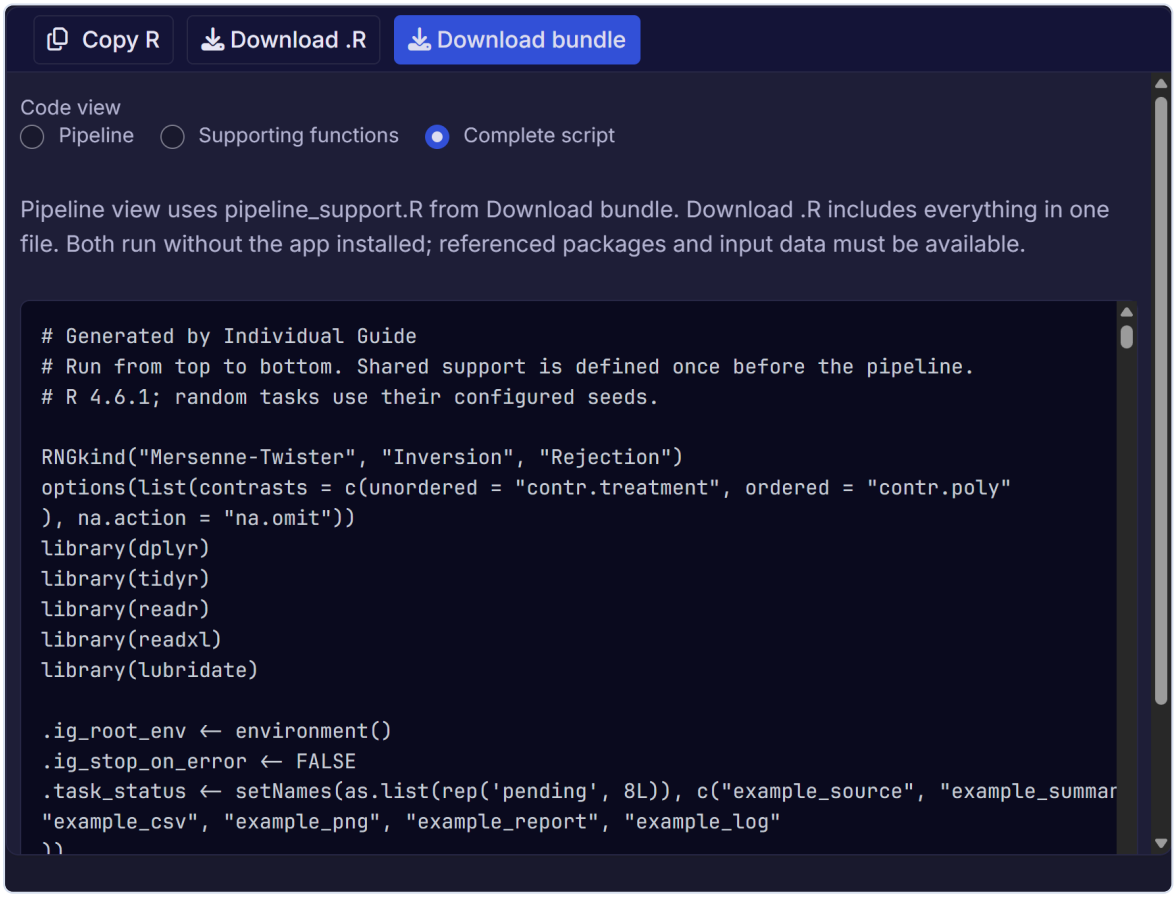


Figure 31. The Code tab separates the pipeline from supporting functions. Download .R includes both in one file; Download bundle keeps them together in a ZIP.

The recipient needs R, the packages requested by the script, and access to its inputs. The complete generated pipeline does not need the Individual Guide package itself. Database access and file locations still need to be set up on the receiving computer.

Open the bundle's README and follow its instructions in a separate working folder. Check output destinations before running. Running a saved script can replace files just as running Export inside the app can.

21 Explain and review your work

A useful project tells another person what you did and why. A few clear notes can save much more time than a long trail of unexplained task names.

21.1 Add notes at decision points

Use task notes or project comments to explain choices. Record why you removed rows, filled missing values, chose a model, or set a threshold. Write for someone who sees the project for the first time.

1. Right-click a task in the flow. Enter **Note (shown as node tooltip)**, then select **Save note**. Point to the task later to read it.
2. For a longer discussion, choose **View > Comments...** in the desktop app, or the browser app's **Comments** button.
3. Choose **Attach to**, enter an optional title, and type your comment. Select **Comment** to keep it, or **Raise issue** when it needs review.
4. Save the project so the notes travel with it.

For example: **We kept orders from January through March and excluded cancelled orders.** That tells a reader more than **Filter updated**.

21.2 Check before sharing

1. Choose **View > Review dashboard...** in the desktop app, or the browser app's review control. Check outstanding issues before marking work ready for review.
2. Choose **View > Change history...** to inspect saved project changes. Use its filters to find the task or time you need.
3. Choose **Task > Run records...** to inspect previous executions. In a browser, these records are under **Project tools > Run records**.
4. Compare records for the task you are checking. A run record helps identify changes in settings, inputs, and software versions; it does not replace reading the result.

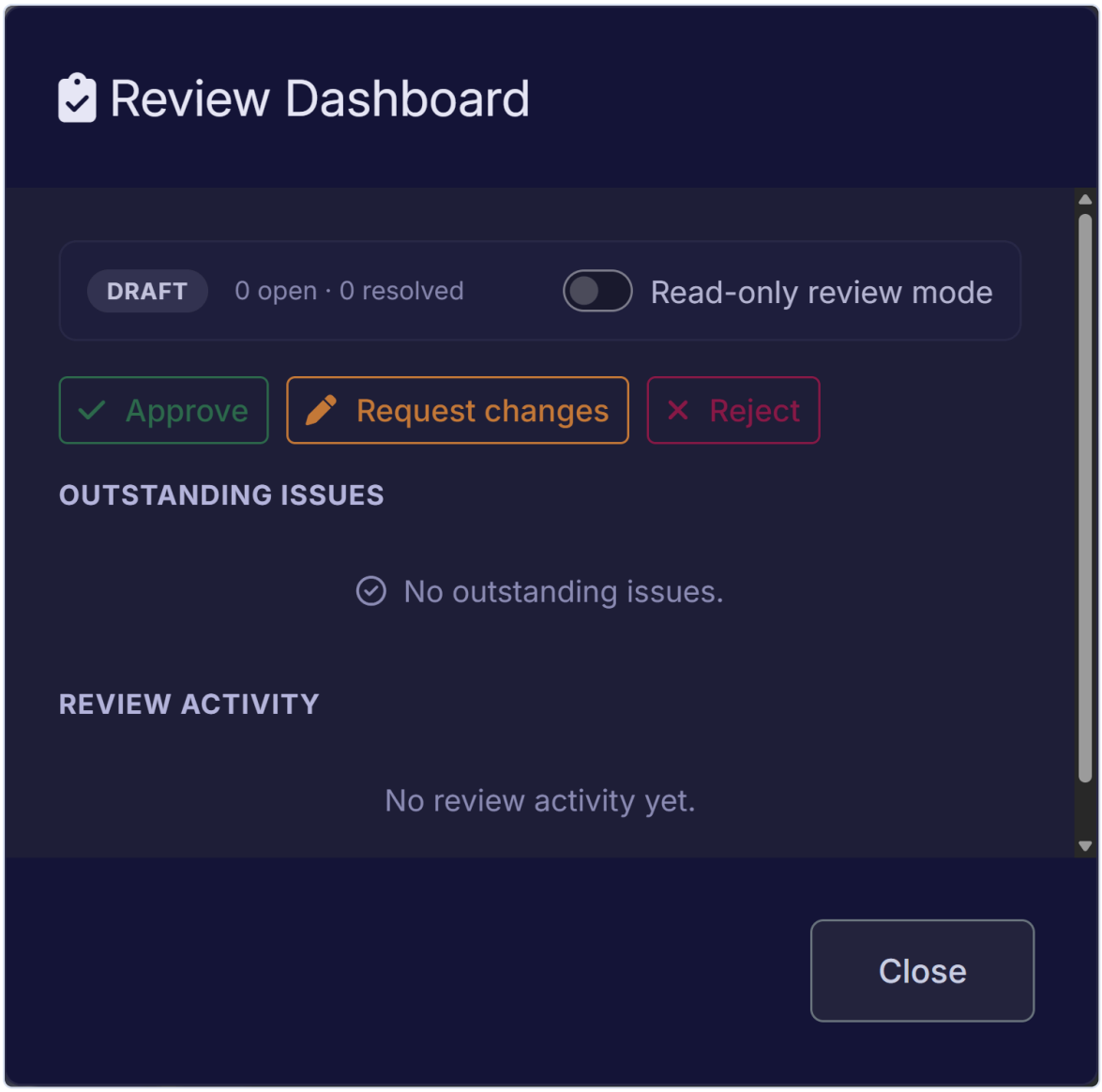


Figure 32. The review dashboard collects review status and issues. Notes and comments help another reader understand the choices behind the results.

Check the current results too. Old success messages do not prove that a changed task has run again. Update stale results and the report, then save.

21.3 Find something quickly

1. In the desktop app, press Ctrl+Shift+F to open **Global Search**. In a browser, select the **Global search** button in the app header.
2. Type part of a task name, column name, or comment. Select a result to open it.
3. For a task-only view, choose **View > Task list** in the desktop app, or **Project tools > Task list** in a browser. Enter part of its name in **Find a task**, then open the matching row.

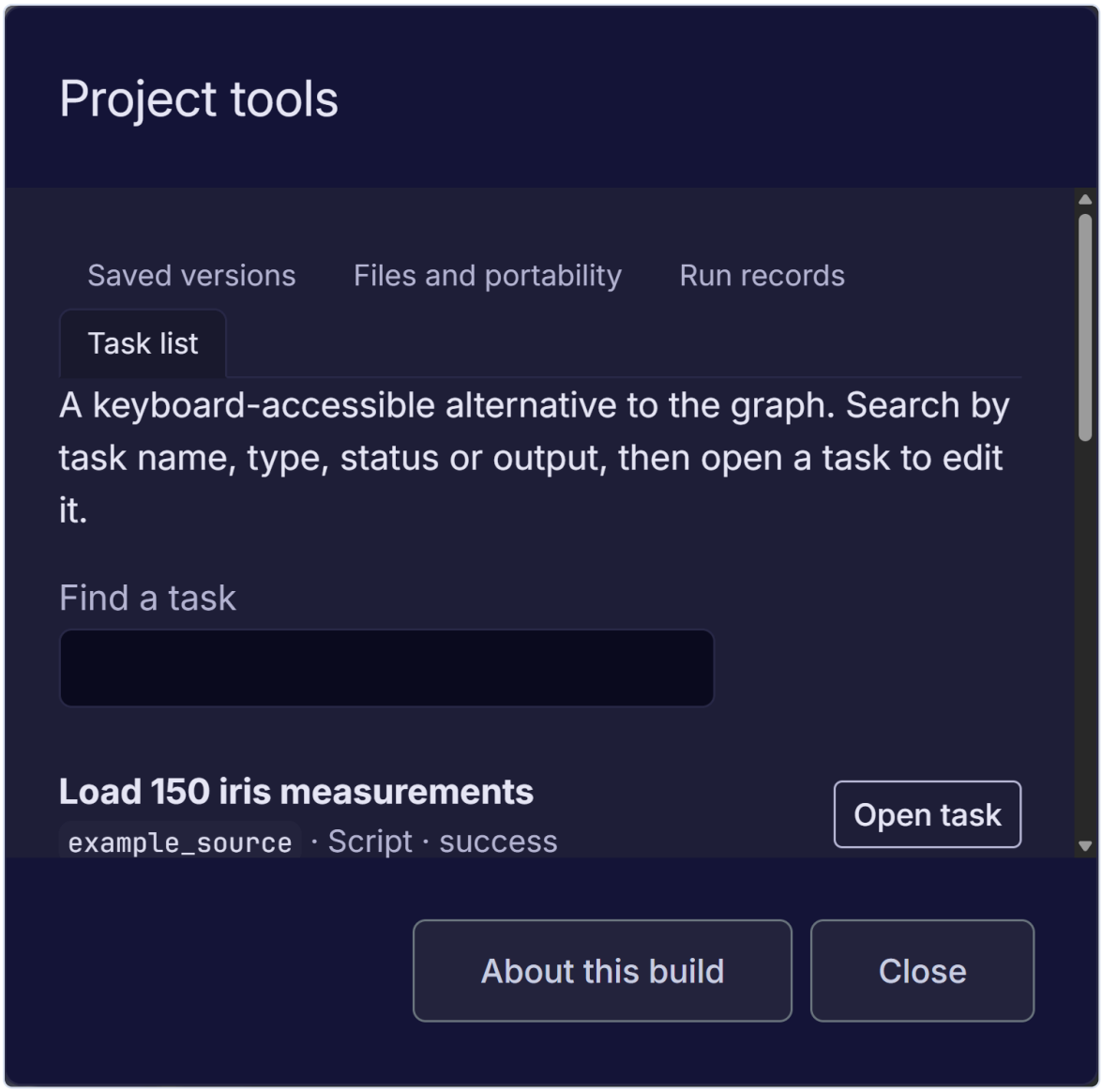


Figure 33. Task list provides a searchable alternative to the flow diagram. Find a task by its name, type, status, or output.

Undo can restore supported project changes, but it does not rewind every action. In particular, it cannot bring back the old contents of an outside file that Export overwrote. Save a separate copy before a large change.

21.4 A simple review checklist

- Can you name the data source and the date of the copy you used?
- Do row counts and missing values make sense after cleaning?
- Do charts show the correct columns, labels, and units?
- Have you checked model warnings and the number of rows used?
- Does the latest report match the latest results?
- Are logs and exported files in the folder you expect?
- Have you saved the project and checked the copy you will share?

22 Make the app comfortable to use

1. Open a project, then choose **File > Settings...** in the desktop app. In a browser, select **Settings** in the app header.
2. Choose **Global defaults** or **This project**, then search for a setting or select a category. Start with **Appearance** or **Preview rows per page** if the workspace is hard to read.
3. Select **Edit** beside the option, change its value, then select **Save option**. Check the workspace. Save the project to keep project overrides.

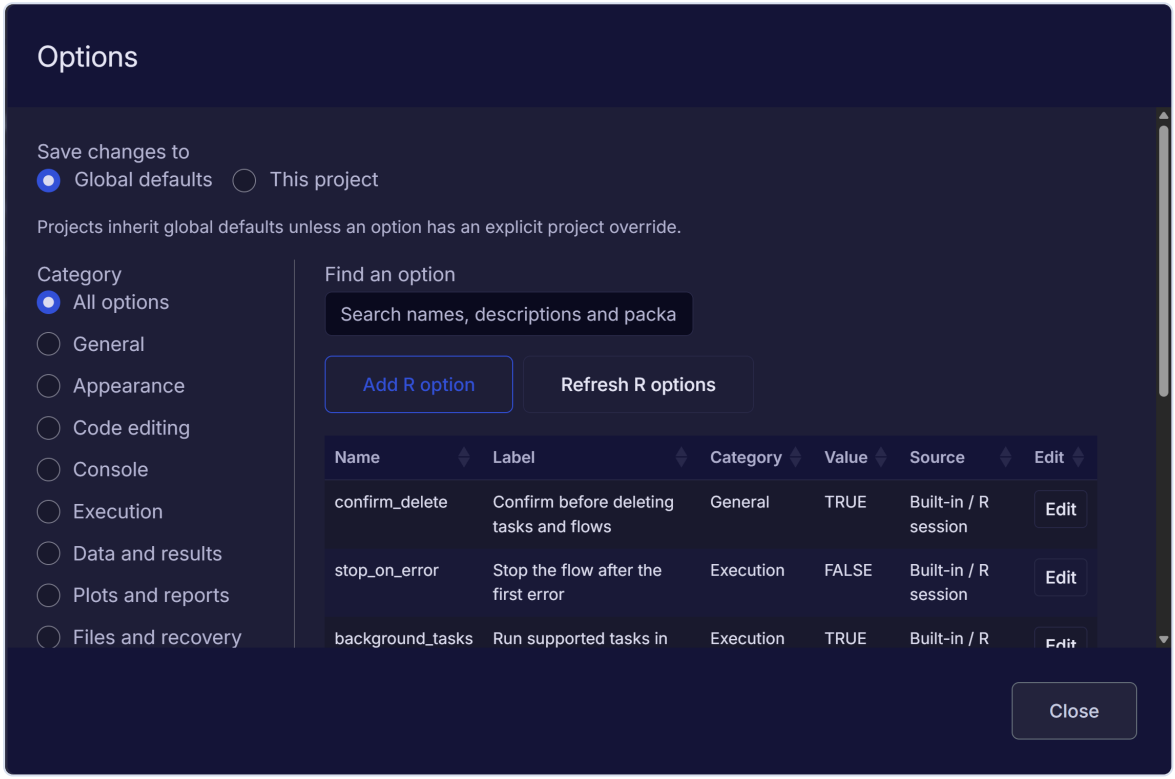


Figure 34. Options groups appearance, code editing, the console, execution and recovery settings. Choose a scope, find an option, then select Edit.

22.1 Global defaults and project overrides

Global defaults apply to projects that inherit them. **This project** saves only the values you override. Select **Use global default** to remove one project override. An explicit `NULL` R value removes an option; it does not mean inheritance. Older projects keep their saved preferences when opened.

The R-options categories list the bundled R reference and options registered by the app or console. Use **Add R option** for a package option that is not listed. Enter its name, choose a value type, and select **Save option**. The expression editor accepts vectors, lists and functions; expressions run when you save. Use the option's documentation to choose a valid value. Some R options need a restart. If a saved R setting prevents startup, choose **Session > Restart with default R options...**, repair the setting, then restart normally.

22.2 Package versions

Choose **Session > R packages...**, or **R package manager** in Options. IG uses its own library. Enter a package name and an exact version, or leave the version empty for the latest available version. **Pin installed version** keeps the selected version. Export a lockfile to record the package profile; restore it to request those versions again. Save your project and restart R after package changes. The manager can restore a previous profile or return to bundled packages.

22.3 Display and navigation

Use **Appearance** to choose Dark or Light. **Default plot theme** and **Default plot palette** affect newly added charts; they do not restyle every existing chart. Code-editing and console categories control fonts, wrapping and other display choices. Use the desktop View menu's zoom controls if text is too small. For a large flow, combine **Fit view** with **Task list**.

Keyboard navigation can help when a diagram is difficult to use. Tab moves focus through controls. The shortcut guide lists common desktop keys.

22.4 Saving and confirmation choices

Previous saved versions to keep sets the number of earlier explicit saves retained for a project. The default is 20; zero turns these versions off. Crash recovery is separate. Keep your own backups for important projects.

Enable **Stop the flow after the first error** while learning, so an error does not pass unnoticed. **Run supported tasks in background workers** helps keep the interface available during supported work; it does not make every task run at once. **Recovery autosave interval (seconds)** defaults to 30.

Leave useful confirmations on while learning. Read any warning about unsaved work before opening another project or example. Saving takes a moment and is easier than rebuilding changes later.

22.5 Trust and private information

Opening a project can make its saved tasks and code available to run. Accept projects from sources you trust. Review input paths, export destinations, and Script tasks before running someone else's flow.

Treat a project as a file that may contain private information. A report or log can contain it too. Check the actual files before putting them in a public folder or attaching them to an issue.

23 Open other Shiny apps

A Shiny app is an interactive page that uses R behind the scenes. It may show a dashboard, a chart, or a form. You can open an app someone has shared with you inside Individual Guide. You do not need to copy its code into a pipeline.

Your Individual Guide project stays open. The other app runs in a separate copy of R. Its view fills most of the window, with an Individual Guide bar across the top.

23.1 What you need

Ask the app's author for its complete folder and any setup instructions. A supported folder has either:

- One **app.R** file, plus any data and helper files it uses.
- Both **ui.R** and **server.R**, plus any other files it uses.

Keep the folder together. If it came in a ZIP file, extract it first. Selecting a file does not upload it or install packages. The app runs on this computer and can read or change files with your permissions. Use apps only from a source you trust.

23.2 Try the practice apps

1. Download the Shiny practice apps for this release.
2. Right-click the ZIP in Windows and choose **Extract All**. Choose a folder you can find again.
3. Follow the steps below. Select **iris-explorer/app.R** for the flower app. Select the **local-sales** folder for the two-file app, which reads a CSV and can download a filtered copy. Both use packages included with Individual Guide.

The flower app has its own **Language / Langue** list. Choose **Français (Canada)** there to see French labels inside that app. That list changes the practice app only.

23.3 Open and run an app

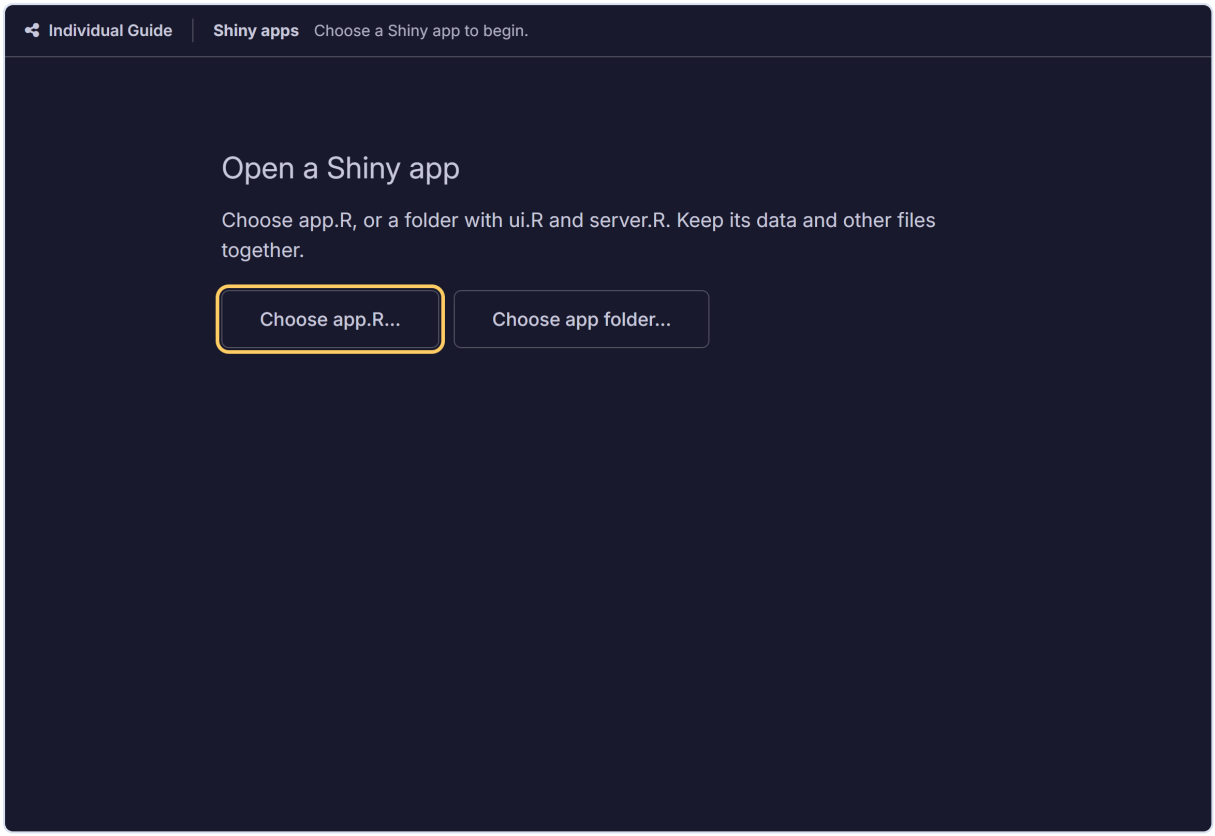


Figure 35. Choose a single app.R file or a folder containing both ui.R and server.R. Your Individual Guide project stays open behind this view.

1. On the welcome screen, choose **Open Shiny app**. From your workspace, use the window icon beside the app name. In the desktop menu, **File > Open Shiny app...** opens the same view.
2. Choose **Choose app.R...** for a single-file app, or **Choose app folder...** for a folder. For the two-file format, choose the folder that contains both files.
3. Check the selected path. Keep any data, helper scripts, and **www** folder in their original places.
4. If you trust the source, select **I trust this app**, then **Run app**.
5. Wait for the app to appear. Use its own controls to work with its data. The Individual Guide bar stays above it.

23.4 Return to your project

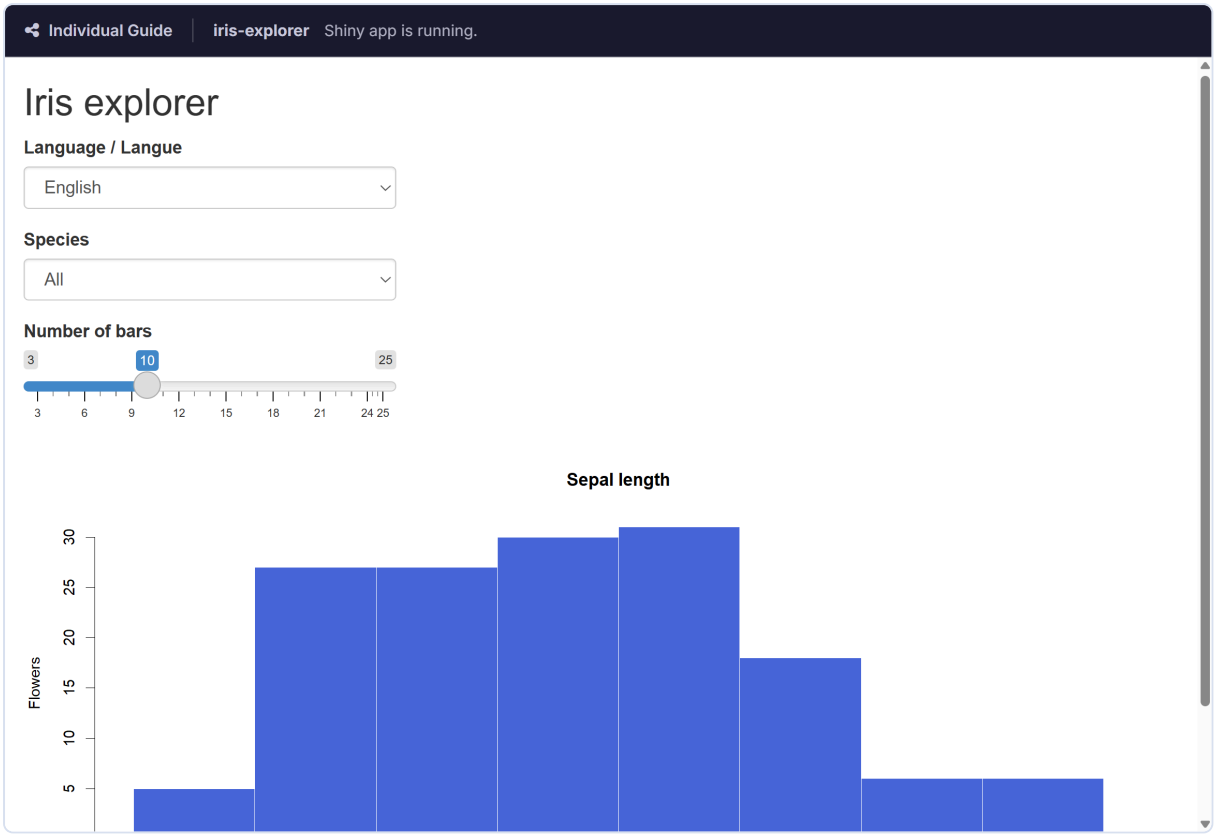


Figure 36. The loaded app fills most of the window. The Individual Guide bar keeps the workspace, app switcher, log and close controls within reach.

Choose **Individual Guide** at the left of the top bar. Your project is still there, with its inputs and results. The other app keeps running in the background.

Use the window icon again to return to the other app. Its inputs stay as you left them. In the desktop app, **View > Individual Guide workspace** also brings you back to your project.

Switching views does not restart either R process. If you are finished with the other app, close it to free its resources.

23.5 Close an app or open another

To stop an app, choose **Close app** in its top bar. Save any work using that app's own controls first. Read the confirmation, then choose **Close app** again. **Cancel** keeps it running. Closing it leaves Individual Guide and your project open.

To switch apps, choose **Open app...**, then select another file or folder. Picking a file alone does not close the running app. You can choose **Back to running app** if you change your mind. When you select **Run app**, the previous app closes and the new one starts. Save work in the previous app before that step.

Closing Individual Guide stops the loaded app too. Its temporary session is not saved inside your **.igpx** project. Each app decides how to save its own data and settings.

Restarting the main R server also closes the loaded app. Save work in both apps before restarting.

23.6 If an app will not start

Choose **App log** to read its messages. The same control works while an app is running. Logs keep the app's original language. Copy the exact error when asking its author for help.

- If a file is missing, check that you extracted and kept the complete folder.
- If a package is missing, follow the author's public package instructions. Individual Guide includes its own packages, but it cannot include every package another app might need. Individual Guide does not install extra packages for that app.
- If the app stops or takes too long to start, the top bar stays available. You can run it again, choose another app, or return to your project.

The language choice in Individual Guide changes its own top bar and controls. The loaded app keeps the language and appearance supplied by its author. Its R code runs separately, with the package libraries available to Individual Guide. Apps that need another R version, external software, or a database still need their own setup.

This feature opens local apps. In the ordinary browser version, use a browser on the same computer that runs Individual Guide.

24 Keep work running smoothly

Try a small version of a flow before using a large dataset. Check its settings, then run the full version once you know what it will do.

24.1 When editing feels slow

Wait for an active run to finish, or use Stop if you do not need it. Close task panels you are not using and use the Task list to move around a large project. A smaller chart or sample can make exploration easier, but keep full data for calculations that need it.

If the app remains slow, save, close it, and open it again from its shortcut. Record which task and action cause the delay if you need help. A report such as **Typing the output name is slow in a 40-task project** is easier to investigate than **The app is slow**.

25 Troubleshooting

Start with the symptom below. Keep the exact error text when asking for help. It often points to a missing file, column, or package.

25.1 The app seems not to open

Use the installed Start menu or desktop shortcut. Wait for the startup stages, especially on the first launch. If you opened Setup, finish installation and then use the app shortcut.

If startup shows a failure, open its startup log. Save the error text. Check that the installer came from the public release page and that it matches your 64-bit Windows computer.

25.2 The app disconnects or the R server stops

1. If the workspace still responds, save the project and wait for saving to finish.
2. In the desktop app, choose **Session > Restart R server...**. If a reconnect screen is shown, its **Restart R server** button provides the same recovery route.
3. Read the confirmation and restart. This stops active work and can lose unsaved changes.
4. Wait for reconnection. Reopen your saved project, or select **Restore** if a **Recover unsaved work?** prompt offers the work you need. Save a recovered copy before continuing.
5. Check the input paths and results before running the flow again.

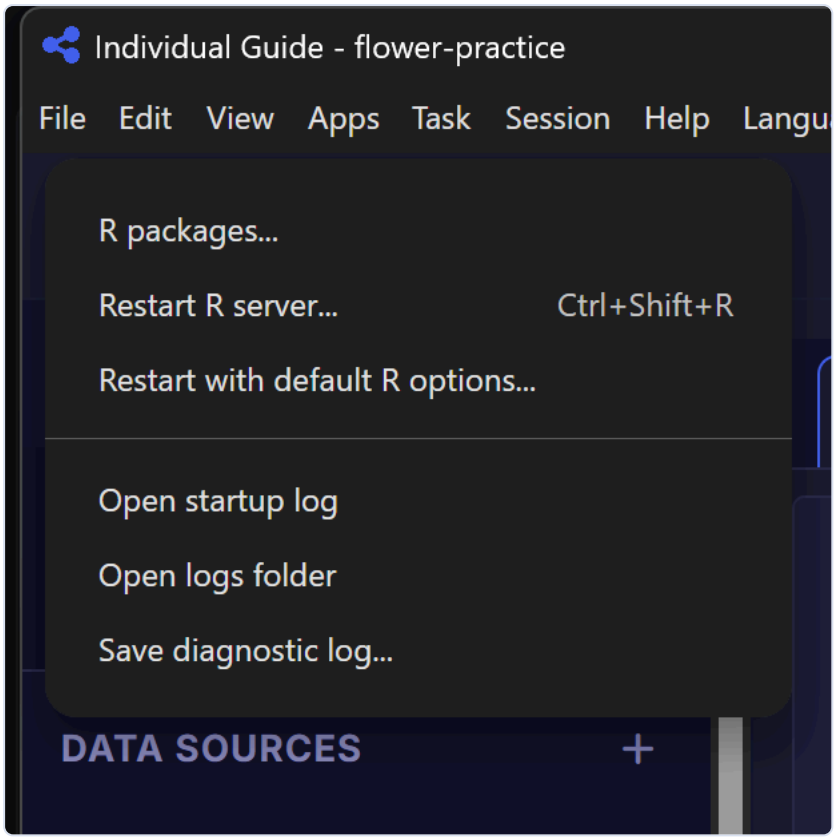


Figure 37. The desktop Session menu provides server restart and startup diagnostics. Use task logs instead when investigating an analysis result.

When running through R in a web browser, go to the R window that launched the app. Stop that session if needed and run the launch command again. See Optional: install with R. The browser version has no independent desktop Session menu.

25.3 A button or menu item is disabled

Finish any open dialog and check whether a flow is running. Open or create a project if you are still on the welcome screen. If the app is disconnected, reconnect or restart the R server first.

25.4 A file will not import

Check that the file still exists and that you can open its folder. For text files, check the separator and header. For Excel, check the selected sheet and range. Confirm the file type matches the import choice.

If the error names a missing package and you run through R, follow the public package instructions. If you use Setup, keep the full error and app version when reporting the issue.

25.5 A table or column is missing from a choice list

Run the task that creates the table. Check its output name and Results. Then check whether a cleaning step renamed or removed the column. A numeric-only list may hide a column stored as text; inspect its type before converting it.

25.6 A result is old, empty, or different from the preview

Check the selected input and filters. Use Run, rather than only Preview, to keep the result. Run later tasks again when their inputs change. Check branch decisions for skipped work and logs for failed earlier tasks.

Some views show only the first part of a large table. Read the row-count and limit information before assuming the full data are missing.

25.7 A model fails or looks strange

Read its first warning or error. Check response and predictor types, missing values, group counts, and the number of usable rows. Start with a matching example to check the setup. A model may need method advice even when all app settings are valid.

25.8 A report is missing a chart

Run the chart task and enable its static image output. Select that image in Report, then run Report again. Open the downloaded file to check the result.

If Word generation says Pandoc is missing, use the reporting setup in the optional R section. The Windows Setup already includes Pandoc. HTML is a useful first format to try.

25.9 An exported file is missing or will not update

Open its Export or Export Log task. Check the full destination, folder permissions, filename, and format. Run the task again. If the file already exists, decide whether to allow Overwrite or use a new name. Close the file in another program if it is locked.

25.10 A log seems empty

Check that task logging is enabled and that its rules are enabled. INFO messages will not appear with a minimum of WARN. Run a selected task again and check its own log before checking an export or report appendix.

The report's log section contains earlier recorded events, not its own later completion. Put Export Log after Report when you need that entry.

25.11 Ask for help

Use the public issue tracker for app problems. A GitHub account is needed to create an issue. Include the app version, how you opened it, the last action, the exact error, and what you expected.

For startup or connection problems, use **Session > Save diagnostic log...** in the desktop app. For an analysis problem, export the relevant task log. Check attachments for private data and passwords before posting them. A small example using public data is often the easiest way to explain a problem.

26 Optional: install with R

Skip this chapter if you use the Windows Setup. These steps are for people who want to launch the app from R instead. You do not need to learn general R programming first.

R is the software that performs calculations. An R **package** is an add-on. IndividualGuide is the app's package name. You install it once, then use a short command to open the app on later days.

26.1 Step 1: install R from its public website

1. Go to the R download site, CRAN. Choose your operating system.
2. On Windows, choose **Download R for Windows**, then **base**, then the current R installer. Download it and follow its instructions.
3. Open **R** from the Start menu. Check for a console with a `>` prompt before continuing.

That R console is where you paste the installation commands below. Do not type the `>` itself. Use the separate R application for installation, rather than a Script task or the R Console inside Individual Guide.

On macOS, use the CRAN macOS download and open the R application. Linux users should follow CRAN's instructions for their Linux version; extra system software may be needed for some packages.

26.2 Step 2: install the app from GitHub

You need internet access for installation. In the R console, paste these commands and press Enter. Installation can take several minutes and print many progress lines. Keep R open and wait for a new `>` prompt before moving on.

```
options(repos = c(CRAN = "https://cloud.r-project.org"))
install.packages("remotes")
remotes::install_github(
  "woodtho/IndividualGuide",
  ref = "v0.7.0",
  dependencies = NA,
  upgrade = "never"
)
```

This gets the app from its public GitHub release tag and its required add-ons from the public CRAN package service. It does not require you to download the source repository or find a local build folder. You normally do not need a GitHub account for this public download.

If R asks to create a personal package library, choose Yes. This means a folder for your installed add-ons. If Windows asks whether to compile packages from source when a ready-made version is available, choose No to use the ready-made version. If installation ends in an error, read it before trying to launch the app.

26.3 Step 3: add the features you need

The basic app installation includes its required packages. These extra public packages support reports and some analysis tasks. Install them once if you want to use those features or explore all the examples:

```
install.packages(
  c("rmarkdown", "knitr", "survival", "forecast"),
  repos = "https://cloud.r-project.org"
)
```

For Word reports, also install Pandoc using its official installer. Close and reopen R after installing it. You can check that R can find it with:

```
rmarkdown::pandoc_available()
```

`TRUE` means it was found. HTML reports have a simpler fallback, so you can start with HTML while sorting out Word support.

Only add database support if you need it. For a SQLite file, run:

```
install.packages("RSQLite", repos = "https://cloud.r-project.org")
```

For an ODBC connection, run:

```
install.packages("odbc", repos = "https://cloud.r-project.org")
```

ODBC may also need a driver from the database provider. Ask your database support contact which one to use. If you have trusted custom connection code, ask its author which extra packages it needs. These commands belong in the separate R application, with the app closed.

26.4 Step 4: open the app

In the R console, run:

```
IndividualGuide::run_app()
```

The app opens in a web browser. Keep the R window open while using it. That R session is running the app, so it is normal for its console to be busy.

Use the app's own open and save buttons. The desktop File and Session menus belong to the Windows Setup version and are not present here.

When you finish, save the project first. Close the app's browser tab, then stop the app in R with the Stop control or Esc as appropriate for your R window. Quit R when you no longer need it.

26.5 Open it next time

Open R and run only `IndividualGuide::run_app()`. You do not need to reinstall packages each day. If the browser does not open automatically, use the local address printed in the R console.

To update later, save and close the app, start a fresh R session, and follow the instructions for the release you want. The tag `v0.7.0` deliberately selects the version used in this guide.

26.6 If package installation fails

If an optional task names a missing package, close the app and return to R. Use one of the single-package installation commands above, replacing its first quoted name with the package named in the error. Keep the CRAN address as shown.

Check your internet connection and copy the first useful error. A message about a missing system library or compiler needs the setup for that package and operating system; repeating the same command will not fix it. On Windows, the public Setup is the easier route if you do not need an R-managed installation.

The commands above follow the official remotes installation reference. Keep using the project's public release notes for version-specific instructions.

27 Choose an example project

All 22 projects below come with data, configured tasks, saved results, logging, and output files. They use public sample data supplied with R or its packages. You do not need a separate data download to explore them.

27.1 Find an example by task

1. Leave the welcome screen with **Start with a blank canvas** if needed. In the desktop app, choose **Help > Application help**; in a browser, select **Help** in the app header. Open **Example projects**.
2. Search for a title or topic. Use the **Task type** filter to find a specific task, such as Rename Columns or Cluster Analysis.
3. Use the **Task category** filter to narrow the list to a group, such as Graph or Survival Analysis.
4. Choose **Open example**. If an unsaved-work prompt appears, cancel and save first. Choose **Discard** in the desktop prompt, or **Discard and open example** in a browser, only when you mean to replace that work. Use **File > Save project as...** in the desktop app to keep your own copy.

If nothing appears, clear the search and filters, then try just one filter. You can also use the Task finder at the end of this guide to see which projects contain a task.

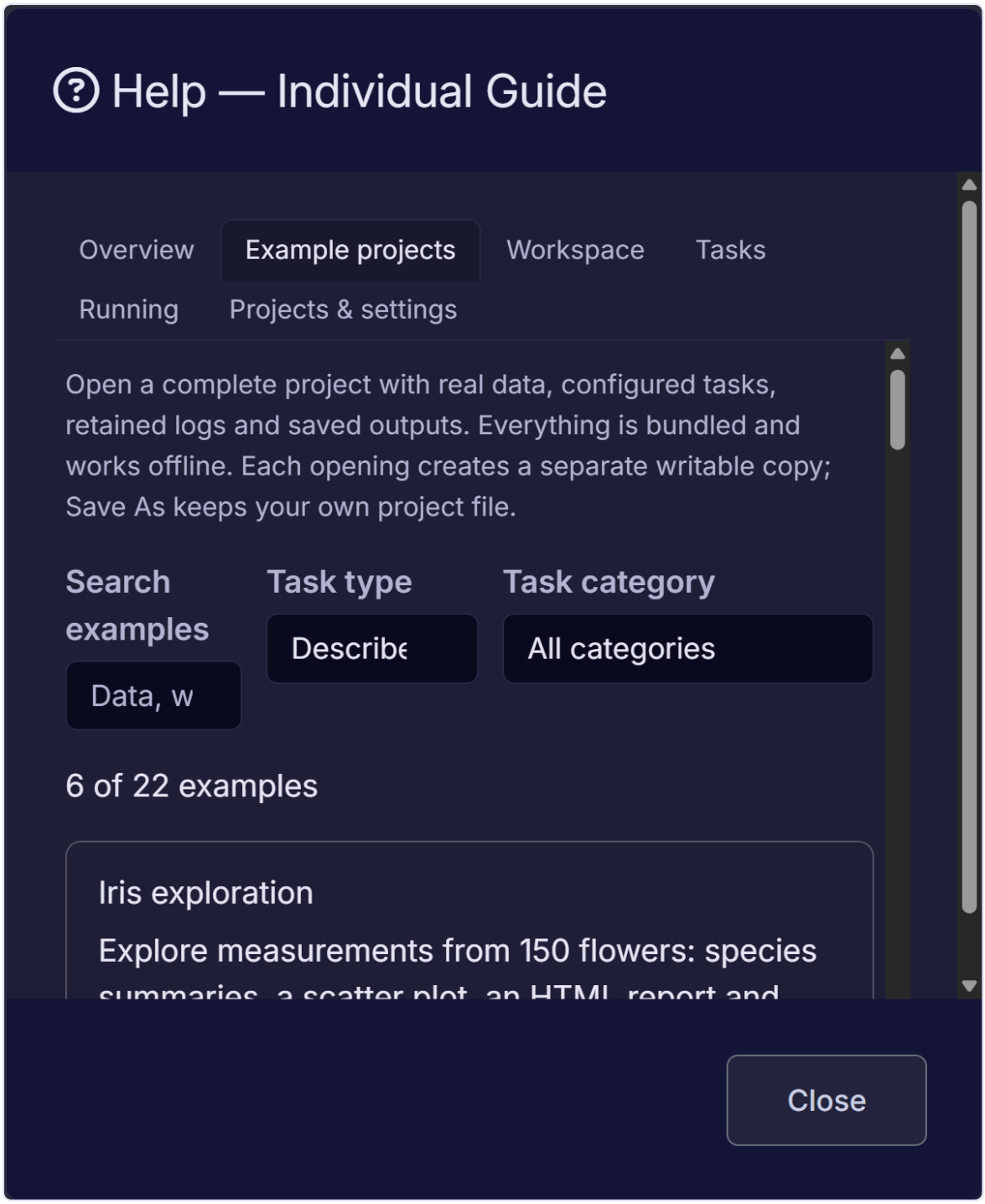


Figure 38. The Task type filter finds example projects that contain Summary Statistics. Clear a filter to return to the full list.

27.2 A simple learning route

Start with **Iris exploration**. Next try **Cars - prepare an analysis table** for cleaning, **Flowers - categorical chart gallery** for charts, and **Cars - a report with an execution audit** for reporting and logs. Choose models and control-flow examples when you need them.

27.3 Before you rerun an example

Read its notes and inspect its saved results first. Check Export and Export Log destinations before running the flow. Each opened example gets a separate working copy, but you should still choose folders and Overwrite settings that suit your own work.

These projects teach app controls. Their model and cleaning choices are examples, not a claim that those choices fit every dataset.

27.4 Iris exploration

Explore measurements from 150 flowers: species summaries, a scatter plot, an HTML report and exported results.

Data: 150 flowers, with 50 in each of three species.

Tasks to find: Script, Summary Statistics, Summary Tables, Scatter Plot, Export, Report, Export Log.

Check the result: Open the species summary. Mean petal lengths are about 1.462, 4.260, and 5.552 centimetres.

Try this next: Change the scatter plot's Y column to Petal.Width. Run the chart and report again.

Open its report and log exports, then save your project copy if you want to keep your changes.

27.5 Air quality - clean and compare

Clean 153 daily New York air-quality observations, inspect missing values and compare ozone measurements with temperature.

Data: 153 daily New York air-quality observations, including missing measurements.

Tasks to find: Script, Summary Statistics, Replace Missing Values, Scatter Plot, Export, Report, Export Log.

Check the result: Compare the raw table, the missing-value audit, and the cleaned result. The example uses mean replacement as a teaching choice.

Try this next: Read the replacement task's settings. In a copy, try another available method and compare the summary. Keep the raw data.

Open its report and log exports, then save your project copy if you want to keep your changes.

27.6 Fuel economy model

Use 32 Motor Trend cars to model fuel economy from weight and horsepower, inspect coefficients, fitted values and a model report.

Data: 32 cars, with fuel economy, weight, horsepower, and other measurements.

Tasks to find: Script, Summary Statistics, Linear Regression, Scatter Plot, Export, Report, Export Log.

Check the result: Open the model coefficients and fitted values. Weight is measured in thousands of pounds.

Try this next: Remove one predictor in a copy. Rerun the model and report and compare the results.

Open its report and log exports, then save your project copy if you want to keep your changes.

27.7 Cars - prepare an analysis table

Turn car measurements into a labelled, deduplicated analysis table and inspect each intermediate result.

Data: 32 original cars plus one duplicate row added for practice.

Tasks to find: Script, Remove Duplicates, Keep / Drop Columns, Rename Columns, Derive Columns, Recode Values, Change Column Types, First / Last N Rows, Characterize Data, Export, Report, Export Log.

Check the result: The duplicate-removal step reduces 33 rows to 32. Inspect the renamed columns and transmission labels.

Try this next: Open each cleaning task in order. Compare the table before and after it, then change one label in a saved copy.

Open its report and log exports, then save your project copy if you want to keep your changes.

27.8 US states - import, join and filter

Import a bundled CSV of state statistics, join a regional lookup, filter and sort the result.

Data: A bundled CSV with 50 US states, plus a state-to-region lookup.

Tasks to find: Script, Import Data, Query Builder, Filter and Sort, Export, Report, Export Log.

Check the result: The left join should retain the states and add region labels. The later filter keeps Income of at least 4500 and sorts it in descending order.

Try this next: In release 0.4.0, leave the saved Filter and Sort panel closed; see its task guide for the known disconnect issue. Add a fresh task to try an Income threshold, then check the rows and descending sort.

Open its report and log exports, then save your project copy if you want to keep your changes.

27.9 Iris - reshape and compare

Pivot flower measurements to long form and back, compare the round trip, append species subsets and transpose a small table.

Data: The 150 Iris flowers with an added flower identifier.

Tasks to find: Script, Stack Columns, Split Columns, Transpose, Compare Data, Append Table, Export, Report, Export Log.

Check the result: Stacking four measurement columns produces 600 rows. The identifier helps put the measurements back into columns. Two appended species subsets contain 100 rows.

Try this next: Compare the long and wide tables. Inspect Compare Data before assuming the round trip preserved every detail.

Open its report and log exports, then save your project copy if you want to keep your changes.

27.10 Cars - rank, scale and sample

Rank fuel economy, standardize predictors and draw a reproducible sample with an explicit seed.

Data: Measurements from 32 cars.

Tasks to find: Script, Rank, Standardize Data, Random Sample, Export, Report, Export Log.

Check the result: Inspect the rank and standardized columns before the ten-row sample. The sample uses seed 2026.

Try this next: Run the sample again with the same data, then change the seed. Compare row identities as well as the count.

Open its report and log exports, then save your project copy if you want to keep your changes.

27.11 Tooth growth - descriptive analysis

Inspect column structure, list observations, count supplement groups, cross-tabulate dose and summarize tooth lengths.

Data: 60 tooth-growth observations, with supplement and dose groups.

Tasks to find: Script, Characterize Data, List Data, One-Way Frequencies, Summary Statistics, Summary Tables, Table Analysis, Distribution Analysis, Export, Report, Export Log.

Check the result: Compare the frequency counts, dose-by-supplement table, and numeric summaries. Dose is stored as a category for the grouped examples.

Try this next: Choose a different grouping in a copy. Check what the counts and percentages refer to before describing them.

Open its report and log exports, then save your project copy if you want to keep your changes.

27.12 Flowers - categorical chart gallery

Compare species with bars, a pie chart and grouped box plots, then collect the figures into a report.

Data: 150 flowers in three equal-sized species groups.

Tasks to find: Script, Bar Chart, Pie Chart, Box Plot, Export, Report, Export Log.

Check the result: The species counts are 50 each. Compare bars, pie slices, and grouped box plots in the report.

Try this next: Change chart titles or category styles. Check that the report's static figures update after you rerun it.

Open its report and log exports, then save your project copy if you want to keep your changes.

27.13 Flowers - numeric chart gallery

Explore numeric relationships with bubbles, density contours, histograms, scatter plots and a scatter matrix.

Data: The four numeric measurements from the 150 Iris flowers.

Tasks to find: Script, Bubble Plot, Contour Plot, Histogram, Scatter Plot, Scatter Plot Matrix, Export, Report, Export Log.

Check the result: Compare the histogram, bubble plot, contour plot, scatter plot, and matrix. Each view answers a different question.

Try this next: Change histogram bins or a selected measurement. Check axis labels and the chart's data limit.

Open its report and log exports, then save your project copy if you want to keep your changes.

27.14 Nottingham temperatures - time charts

Chart twenty years of monthly temperatures and compare annual averages with their observed range.

Data: 240 monthly Nottingham temperature observations, with dates added for charting.

Tasks to find: Script, Line Plot, Bar-Line Chart, Export, Report, Export Log.

Check the result: Compare the monthly line with the annual average-and-range chart. The annual range is not a confidence interval.

Try this next: Inspect the date order. Change a title to include the temperature units and check both axes.

Open its report and log exports, then save your project copy if you want to keep your changes.

27.15 Plant growth - compare experimental groups

Compare plant weights across control and treatment groups, with ANOVA, a two-group t test and a general linear model.

Data: 30 plant weights in one control and two treatment groups.

Tasks to find: Script, One-Way ANOVA, t Test, Linear Models, Export, Report, Export Log.

Check the result: The ANOVA uses all three groups. The t test uses a prepared two-group subset, so the tests do not ask exactly the same question.

Try this next: Inspect the subset before the t test. Read group counts and means before the test tables.

Open its report and log exports, then save your project copy if you want to keep your changes.

27.16 Cars - binary outcome model

Model manual versus automatic transmission with a logistic regression and retain fitted probabilities and coefficients.

Data: 32 cars, with transmission coded as 0 for automatic and 1 for manual.

Tasks to find: Script, Logistic Regression, Export, Report, Export Log.

Check the result: The fitted probabilities describe the outcome coded 1. They use a different scale from the coefficients.

Try this next: Inspect the response coding, then try a different predictor in a copy. Read any fit warnings.

Open its report and log exports, then save your project copy if you want to keep your changes.

27.17 Loom breaks - a count model

Fit a Poisson model to counts of loom breaks by wool and tension, and retain model diagnostics.

Data: 54 loom-break counts, with wool and tension groups.

Tasks to find: Script, Generalized Linear Models, Export, Report, Export Log.

Check the result: The model uses a Poisson family for counts. Read the fit information rather than treating completion as proof of a good model.

Try this next: Inspect the selected response and groups. Compare count summaries with the model results.

Open its report and log exports, then save your project copy if you want to keep your changes.

27.18 Puromycin - fit a nonlinear curve

Fit a power curve to enzyme reaction rates in the treated sample and compare it with the observations.

Data: Puromycin enzyme measurements, filtered to the treated sample.

Tasks to find: Script, Nonlinear Regression, Filter and Sort, Export, Report, Export Log.

Check the result: The example fits a power curve to positive concentration and rate values. Read the preparation filter and compare the curve with the points.

Try this next: Inspect the selected curve and input range. Avoid explaining predictions far beyond the observed measurements.

Open its report and log exports, then save your project copy if you want to keep your changes.

27.19 Flowers - multivariate structure

Explore correlations, principal components and three clusters using the four flower measurements.

Data: Four numeric measurements from 150 flowers; species is kept as a known label.

Tasks to find: Script, Correlations, Principal Components, Cluster Analysis, Export, Report, Export Log.

Check the result: Compare correlations, scaled principal components, and three clusters. Clustering does not automatically use the same scaling as PCA.

Try this next: Inspect the cluster sizes and assignments. Compare them with species without assuming they must match.

Open its report and log exports, then save your project copy if you want to keep your changes.

27.20 Lung cancer - survival analysis

Prepare complete cases, translate the event code, inspect Kaplan-Meier estimates and fit an illustrative Cox model.

Data: 228 original lung-cancer observations, with a separate prepared analysis table.

Tasks to find: Script, Life Tables, Proportional Hazards, Export, Report, Export Log.

Check the result: The preparation converts the original status code 2 to event 1 and selects complete cases. Compare row counts before and after preparation.

Try this next: Read the time and event settings before the curves or model. Use this example to learn the controls, not to make care decisions.

Open its report and log exports, then save your project copy if you want to keep your changes.

27.21 Air passengers - forecast a monthly series

Forecast twelve months of airline passengers with the app's ARIMA task, retaining point forecasts and intervals.

Data: 144 ordered monthly airline-passenger values.

Tasks to find: Script, ARIMA / Basic Forecasting, Export, Report, Export Log.

Check the result: The horizon is 12 future steps. Inspect the model summary and intervals. This is a basic forecasting example, not an automatically tuned seasonal analysis.

Try this next: Change the horizon in a copy. Check the number of forecast rows and explain what one step means.

Open its report and log exports, then save your project copy if you want to keep your changes.

27.22 Nile flow - distribution and stability

Inspect annual river flow with an individuals chart, empirical CDF and normal probability plot.

Data: 100 annual Nile-flow observations in their original order.

Tasks to find: Script, Control Charts, CDF Plots, Probability Plots, Export, Report, Export Log.

Check the result: Read the control-chart flags beside the CDF and normal probability plot. The limits are estimated from these observations.

Try this next: Inspect unusual points and their years. Keep the time order and distinguish chart limits from outside requirements.

Open its report and log exports, then save your project copy if you want to keep your changes.

27.23 Flowers - conditional quality gate

Use a row-count condition to activate a summary path and record the gate decision in the audit log.

Data: 150 flowers and a condition requiring at least 100 rows.

Tasks to find: Script, Branch, Summary Statistics, Export, Report, Export Log.

Check the result: The summary follows the chosen branch path. Check its Run after branch/loop path setting, not only its table input.

Try this next: Raise the threshold above 150 in a copy. Run again and inspect the decision and skipped work in the log.

Open its report and log exports, then save your project copy if you want to keep your changes.

27.24 Flowers - repeat by species

Loop over the three species, compute a separate summary per iteration and keep iteration-aware logs.

Data: 150 flowers split into three species groups.

Tasks to find: Script, Loop, Export, Report, Export Log.

Check the result: Find species_summary_1, species_summary_2, and species_summary_3. Each summarizes a group of 50 flowers.

Try this next: Read the iteration entries in the log. Check that the final report and log export sit outside the repeated body.

Open its report and log exports, then save your project copy if you want to keep your changes.

27.25 Cars - a report with an execution audit

Create a results report with a pipeline-log appendix, a custom warning and separate text, CSV and JSON log files.

Data: 32 cars, a numeric summary, and a custom teaching warning.

Tasks to find: Script, Summary Statistics, Export, Report, Export Log.

Check the result: Open the report's Pipeline execution log and compare the TXT, CSV, and JSON log exports. The later export can include report completion.

Try this next: Set the report appendix to WARN and rerun it. Compare its shorter log with the INFO version and the full export.

Open its report and log exports, then save your project copy if you want to keep your changes.

28 Task finder

Find a task below, then follow its link for setup steps. The example links show up to three projects you can open from Help. Use the app's task filter to find all matches. Start by inspecting a completed example rather than rebuilding it from scratch.

Task	Where to find it in Add Task	Examples to open
Import Data	Data	US states - import, join and filter
Query Builder	Data	US states - import, join and filter
Script	General	Iris exploration; Air quality - clean and compare; Fuel economy model
Branch	Control Flow	Flowers - conditional quality gate
Loop	Control Flow	Flowers - repeat by species
Append Table	Data	Iris - reshape and compare
Compare Data	Data	Iris - reshape and compare
Filter and Sort	Data	US states - import, join and filter; Puromycin - fit a nonlinear curve
Stack Columns	Data	Iris - reshape and compare
Split Columns	Data	Iris - reshape and compare
Transpose	Data	Iris - reshape and compare
Rank	Data	Cars - rank, scale and sample
Standardize Data	Data	Cars - rank, scale and sample
Random Sample	Data	Cars - rank, scale and sample
Remove Duplicates	Data	Cars - prepare an analysis table
Keep / Drop Columns	Data	Cars - prepare an analysis table
Rename Columns	Data	Cars - prepare an analysis table
First / Last N Rows	Data	Cars - prepare an analysis table
Replace Missing Values	Data	Air quality - clean and compare
Derive Columns	Data	Cars - prepare an analysis table
Recode Values	Data	Cars - prepare an analysis table
Change Column Types	Data	Cars - prepare an analysis table
Characterize Data	Describe	Cars - prepare an analysis table; Tooth growth - descriptive analysis
Distribution Analysis	Describe	Tooth growth - descriptive analysis
List Data	Describe	Tooth growth - descriptive analysis
One-Way Frequencies	Describe	Tooth growth - descriptive analysis
Summary Statistics	Describe	Iris exploration; Air quality - clean and compare; Fuel economy model
Summary Tables	Describe	Iris exploration; Tooth growth - descriptive analysis
Table Analysis	Describe	Tooth growth - descriptive analysis

Task	Where to find it in Add Task	Examples to open
Bar Chart	Graph	Flowers - categorical chart gallery
Bar-Line Chart	Graph	Nottingham temperatures - time charts
Box Plot	Graph	Flowers - categorical chart gallery
Bubble Plot	Graph	Flowers - numeric chart gallery
Contour Plot	Graph	Flowers - numeric chart gallery
Histogram	Graph	Flowers - numeric chart gallery
Line Plot	Graph	Nottingham temperatures - time charts
Pie Chart	Graph	Flowers - categorical chart gallery
Scatter Plot	Graph	Iris exploration; Air quality - clean and compare; Fuel economy model
Scatter Plot Matrix	Graph	Flowers - numeric chart gallery
One-Way ANOVA	ANOVA	Plant growth - compare experimental groups
t Test	ANOVA	Plant growth - compare experimental groups
Linear Models	ANOVA	Plant growth - compare experimental groups
CDF Plots	Capability	Nile flow - distribution and stability
Probability Plots	Capability	Nile flow - distribution and stability
Control Charts	Control Charts	Nile flow - distribution and stability
Linear Regression	Regression	Fuel economy model
Logistic Regression	Regression	Cars - binary outcome model
Generalized Linear Models	Regression	Loom breaks - a count model
Nonlinear Regression	Regression	Puromycin - fit a nonlinear curve
Correlations	Multivariate	Flowers - multivariate structure
Principal Components	Multivariate	Flowers - multivariate structure
Cluster Analysis	Multivariate	Flowers - multivariate structure
Life Tables	Survival Analysis	Lung cancer - survival analysis
Proportional Hazards	Survival Analysis	Lung cancer - survival analysis
ARIMA / Basic Forecasting	Time Series	Air passengers - forecast a monthly series
Export Log	Output	Iris exploration; Air quality - clean and compare; Fuel economy model
Export	Output	Iris exploration; Air quality - clean and compare; Fuel economy model
Report	Output	Iris exploration; Air quality - clean and compare; Fuel economy model

29 Shortcuts and everyday terms

29.1 Useful Windows desktop shortcuts

Use the menus while learning. These keys can save time once the actions feel familiar. A text editor or open dialog may use a key for its own controls.

Keys	Action
Ctrl+Shift+O	Open Shiny app
Ctrl+Alt+I	Return to Individual Guide workspace
Ctrl+N	New project
Ctrl+O	Open project
Ctrl+S	Save project
Ctrl+Shift+S	Save project as
Ctrl+Shift+F	Search project
Ctrl+Alt+L	Open Task list
Ctrl+Alt+R	Run active flow, when focus is outside an editor
Ctrl+Shift+F5	Stop the flow run
Ctrl+Shift+R	Restart R server; save first if possible
F1	Open app help
Tab / Shift+Tab	Move focus through controls

For more keys, open **Help > Keyboard shortcuts...**. If you run in a normal web browser, use the app buttons; the browser may use these same keys for its own actions.

29.2 A few more terms

Term	Plain meaning
Column type	The kind of value stored, such as text, number, date, or category
CSV	A text data file whose columns are separated by commas
Input	A table or result that a task needs
Missing value	An unknown entry; not automatically zero
Object	A saved result that is not an ordinary table, such as a model
Overwrite	Replace an existing file with a new one
Package	An add-on that gives R more features
Path	The address of a file or folder

Term	Plain meaning
Predictor	A column used to help explain or predict a response
Response	The outcome a model tries to explain or predict
Seed	A setting for repeating a random choice with the same inputs
Stale	A result that needs to be rerun after a change

29.3 Public downloads and help

- Current handbook and downloads: the latest reviewed HTML, PDF, EPUB, and Word guide. Books attached to older releases may be earlier editions.
- Individual Guide releases: public Windows Setup and R package files.
- Release 0.7.0: the version used by this guide.
- Report an app issue: include the version and exact error; check attachments for private information.
- Download R: needed only for the optional R installation route.
- Install Pandoc: for Word reporting when using your own R installation.

The example data come from R's datasets package and the survival package. The example notes describe prepared fields and teaching choices. Individual Guide is an open-source project by Thomas Wood.

Individual Guide 0.7.0. Find downloads and help through the public links in this guide.